universidad de león

School of Industrial, Computer and Aerospace Engineering

BACHELOR'S DEGREE IN AEROSPACE ENGINEERING

Bachelor's Thesis

Cascaded hybrid control of a quadcopter: neural replacement of specialized outer-loop controllers

Control híbrido en cascada de un cuadricóptero: sustitución neuronal de controladores especializados del lazo externo

Author: Christian González Pérez

Supervisors: José Ramón Rodríguez Ossorio

Guzmán González Mateos

June, 2026

This page intentionally left blank.

Acknowledgements

I would like to start by thanking my family for always being there and for allowing me to study at university. Their constant support has been the foundation for facing this endeavor.
To my partner Alba, for the joint work and for the support during the first semester.
To my classmate Guillermo, for all the support during these four years. Thanks to him, I have participated in activities and built projects that, without his drive, would not have moved forward.
To my friends Alejandro and Alberto, companions in continuous `refinement`*. Thanks to them, we formed a trifecta with which we navigate technological developments month after month, perfecting our style, our procedures, and our way of improving. Because we do not add up to three, we multiply by eight. Thank you for all the long hours struggling with Gemini CLI or Opencode to get an image placed in LaTeX exactly as we wanted; for all the attempts with Chinese tools to convert images into selectable text; for the experiments creating custom agents; for the work with MCP connections and with CLI to expand the capabilities of artificial intelligence; and for all the attempts to achieve what we were told was impossible.*
To my thesis supervisors, Joserra and Guzmán, for allowing me to develop the work that I desired and for giving me the space to do so with cutting-edge tools, leveraging all the knowledge I had acquired. Their trust made it possible for this project to be highly ambitious, consistent with my way of moving forward, experimenting, and developing.
To my professor Andrés Merino, of Aeronautical Meteorology, for not only allowing, but motivating, the use of generative AI in programming during the practical part of his subject to achieve much more advanced results. All of this was supported by the same theory that would have underpinned conventional work, but thanks to the tools we were able to build more complex simulators, set up weather stations, and create applications deployed in the cloud worthy of being presented in competitions. Thank you for allowing me to develop my skills in a real engineering environment and for giving me space to perfect my techniques, my procedures, and my knowledge on how to manage these tools.
And, finally, to all the people who listen to me, who support me, and who are there in the most difficult moments. To all the people who told me "test it", "try it", "you can do it". Thank you, because we proved that it was indeed possible.

This page intentionally left blank.

UNIVERSITY OF LEÓN

School of Industrial, Computer and Aerospace Engineering

BACHELOR'S DEGREE IN AEROSPACE ENGINEERING

Bachelor's Thesis

STUDENT: Christian González Pérez

SUPERVISORS: José Ramón Rodríguez Ossorio
Guzmán González Mateos

TITLE: Cascaded hybrid control of a quadcopter: neural replacement of specialized outer-loop controllers

TITLE (EN): Control híbrido en cascada de un cuadricóptero: sustitución neuronal de controladores especializados del lazo externo

DEFENSE TERM: June, 2026

RESUMEN:

Este trabajo estudia si un controlador neuronal entrenado por imitación puede sustituir a un conjunto de controladores proporcional–derivativos (PD) especializados en el seguimiento de trayectorias de un cuadricóptero simulado. Para ello se desarrolla un entorno reproducible de simulación que modela la dinámica de cuerpo rígido en sistemas de referencia ENU/FRD, la actitud mediante cuaterniones, actuadores con saturación y retardo, perturbaciones y frecuencias diferenciadas de física, control y telemetría. Sobre este entorno se implementa una arquitectura en cascada con un lazo externo que genera la fuerza deseada y un lazo interno clásico que conserva la estabilización de actitud y la asignación a rotores. Primero se sintonizan controladores PD especializados para familias de trayectorias `hold`, círculo, Lissajous y `waypoint`; después se usan como expertos para generar datos de imitación. Se entrenan y comparan tres arquitecturas, MLP, GRU y LSTM, con las mismas entradas, objetivos y procedimiento. La evaluación distingue fidelidad supervisada, seguimiento en bucle cerrado dentro de distribución, transferencia clásica entre familias y escenarios fuera de distribución con variaciones, composiciones, y trayectorias nuevas. Los resultados muestran que la imitación neuronal es viable y puede sustituir a varios PD especializados en condiciones conocidas y en algunas variaciones no vistas. Sin embargo, no se observa una superioridad global de las redes. GRU y LSTM aproximan mejor la fuerza en evaluación supervisada, pero esa ventaja no se traduce de forma sistemática en menor RMSE de posición; MLP resulta competitivo en varias familias y LSTM se degrada en trayectorias geométrica o dinámicamente exigentes. El estudio concluye que la calidad del seguimiento depende de la arquitectura, de la distribución de entrenamiento, de los límites de actuación y del lazo clásico compartido, por lo que la generalización requiere un mayor nivel de evaluaciones.

ABSTRACT:
This work studies whether an imitation-trained neural controller can replace a set of specialized proportional–derivative (PD) controllers for trajectory tracking of a simulated quadrotor. For this purpose, a reproducible simulation environment is developed that models rigid-body dynamics in ENU/FRD reference frames, attitude using quaternions, actuators with saturation and lag, disturbances, and differentiated frequencies for physics, control, and telemetry. On this environment, a cascaded architecture is implemented with an outer loop that generates the desired force and a classical inner loop that preserves attitude stabilization and rotor allocation. First, specialized PD controllers are tuned for the `hold`, circle, Lissajous, and `waypoint` trajectory families; they are then used as experts to generate imitation data. Three architectures, MLP, GRU, and LSTM, are trained and compared with the same inputs, targets, and procedure. The evaluation distinguishes supervised fidelity, in-distribution closed-loop tracking, classical transfer between families, and out-of-distribution scenarios with variations, compositions, and new trajectories. The results show that neural imitation is feasible and can replace several specialized PD controllers in known conditions and in some unseen variations. However, no global superiority of the networks is observed. GRU and LSTM approximate the force better in supervised evaluation, but this advantage does not systematically translate into lower position RMSE; MLP is competitive in several families, and LSTM degrades in geometrically or dynamically demanding trajectories. The study concludes that tracking quality depends on the architecture, the training distribution, the actuation limits, and the shared classical loop, which is why generalization requires a higher level of evaluations.

Keywords: quadcopter, 6DOF simulation, PD control, imitation learning, neural networks.

Student signature:	**Approved by Supervisors:**

Contents

List of Figures vii

List of Tables x

Abbreviations and Acronyms xi

Symbols and Units xiii

Declaration of Generative AI Usage xvii

1 Introduction 1
1.1 Motivation 1
1.2 Hypotheses and Objectives 3
1.3 Structure of the Document 4

2 State of the Art 5
2.1 Quadcopter Simulation and Modeling 5
2.2 Programming and Machine Learning Environments 7
2.3 Classic Cascaded Control and Gain Scheduling 8
2.4 Imitation Learning and Hybrid Control 9
2.5 Neural Networks for Control 10
2.5.1 Feedforward Networks 10
2.5.2 Recurrent Networks 11
2.6 Evaluation, Generalization and Metrics 15
2.7 Positioning of This Work 16

3 Methodology 17
3.1 Physical Model and 6DOF Simulator 17
3.2 Classic Cascaded Control 28
3.3 Neural Control by Imitation 34
3.4 Datasets and Evaluation Metrics 39
3.5 Experimental Workflow and Reproducibility 40

4 Experimental Results 44
4.1 Execution Coverage and Validity 44
4.2 Classic Reference and Cross-Family Transfer 44

4.3 Neural Learning under Known Conditions 45
4.4 Out-of-Distribution Generalization 46
4.5 Representative Architecture Case 49
4.6 Actuation, Protections, and Shared Limits 49
4.7 Discussion of Results and Limitations 50

5 Conclusions and Future Work 54
5.1 Conclusions 54
5.2 Scope and Limitations 55
5.3 Future Work 56

References 59

Appendix 1: Commands and Experimental Traceability 62

Appendix 2: Code Architecture and Relevant Snippets 68

Appendix 3: Visual Simulation Evidences 72

List of Figures

1.1 Conceptual diagram of the experimental workflow of the work. It consists of trajectory generation, specialization of classic controllers, training of neural networks, and comparative evaluation. Source: own elaboration. 2

2.1 Actuation and motion mechanisms in a quadcopter. For each maneuver, the top view of the vehicle in layout (left) is shown with the rotational direction of each rotor and indications of thrust imbalance (+, −, =), and the resulting physical effect in side view (right). Tilting the vehicle (ϕ, θ) redirects the total thrust vector (T), generating the lateral (F_y) or longitudinal (F_x) force components that enable horizontal translation. Source: own elaboration. 6

2.2 Conceptual comparison of neural architectures for control. In (A), the purely static nature of the MLP is shown. In (B), the unrolled flow of the basic RNN and its vulnerability to gradient vanishing are detailed. In (C) and (D), the selective gated structure of the LSTM cell and the GRU cell are contrasted. Source: own elaboration. 13

3.1 Reference frames used by the simulator. The world frame follows the ENU convention and the body frame follows the FRD convention; total thrust is applied in the opposite direction of the vertical body axis (z_B). Source: own elaboration. 17

3.2 X-configuration of the motors and rotor rotational directions in the body reference frame. The physical direction of the rotors is associated with the sign of the reaction yaw moment. Source: own elaboration. 18

3.3 Logical schematic and decision-making of the rotor mixer under nominal and saturated conditions. Attitude control is prioritized by proportional scaling of moments (k) over deviation of the requested collective thrust. Source: own elaboration. 21

3.4 Analytical trajectory families of the simulator. Each geometry introduces different stimuli (stationary, variable speed, coupled acceleration in three axes, or discrete segments). Source: own elaboration. 24

3.5 Kinematic profiles generated during transitions between waypoints. On the left, the trapezoidal profile (for long segments) is illustrated, and on the right, the triangular profile (for short segments, where the commanded speed is not reached). All represented quantities are scalars along the progress axis of the segment. Source: own elaboration. 25

3.6 Multirate time execution sequence and data dependencies. The three frequency bands associated with physics integration (100 Hz), the digital control cycle (50 Hz), and periodic variable logging (10 Hz) are distinguished. Source: own elaboration. 26
3.7 Architecture of the implemented classic cascaded controller. The outer loop (position in ENU world frame), the geometric conversion of force to attitude, and the inner loop (attitude stabilization in FRD body frame) are highlighted. The neural substitution boundary is located at the desired force signal $\boldsymbol{F}_{d,W}$. Source: own elaboration. 28
3.8 Logical flow of optimization and progressive tuning of classic gains. Global random candidates are filtered by stability and safety conditions before local simplex/Nelder–Mead-like refinement begins. Source: own elaboration. 33
3.9 Structure of the hybrid neural controller. The neural network replaces only the translational decision of the outer loop to generate $\boldsymbol{F}_{d,W}$, delegating safety filtering, force-to-attitude conversion, and stabilization to the classic inner loop. Source: own elaboration. 34
3.10 Management of the temporal sliding window in recurrent networks and initialization by sample repetition at startup ($k = 0$). The FIFO queue generates a batch of dimensions $[1, 20, 9]$ at each inference. Source: own elaboration. 37
3.11 General flow of the experimental procedure structured into 10 reproducible phases. The progressive generation of the training datasets, the classic optimization flow, neural training, and validation and OOD evaluation stages are illustrated. Source: own elaboration. 41
3.12 Classification of the evaluation levels and experimental generalization. Four levels are structured with incremental novelty and difficulty: nominal ID interpolation, classic transfer, recombination of known capabilities, and OOD geometries. Source: own elaboration. 42
4.1 Cross-transfer of PD controllers on the `test` set. The rows indicate the family used for tuning and the columns the evaluated family. The diagonal marks the use of the specialized controller in its native family. Source: own evaluation. 45
4.2 Average RMSE under known conditions for the representative PD and the three neural architectures. Only tracking error is shown, as it is the primary metric in this comparison. 46
4.3 Average out-of-distribution RMSE by scenario family. 47

4.4 OOD breakdown by scenario and controller for the four main references. The number in each cell is the RMSE in meters; the asterisk indicates that the run did not complete, although the episode is kept in the aggregation. 48
4.5 Termination types in the OOD series for the main controllers. 48
4.6 OOD scenario `lemniscate_fast_center_yaw`: visual comparison between MLP and LSTM. The top panel shows the horizontal plane and the bottom panel shows the position error throughout the episode. . . . 49
4.7 Average activation of protections and actuation limits in OOD. The figure separates tracking error, control demand, and constraints shared by the control architecture. 50
5.1 Visual sample of known families. 73
5.2 Visual sample of out-of-distribution trajectories. 74
5.3 Three-dimensional view of a representative OOD helix. Source: own elaboration. 75
5.4 Geometric tracking of two specialized PDs from the classic dataset: perturbed hover case and multi-axis Lissajous trajectory. Source: own elaboration. 76
5.5 Geometric tracking in a perturbed circular reference and a waypoint trajectory from the test split. Source: own elaboration. 77
5.6 Visual relationship between tracking error and attitude in two classic dataset cases with disturbances. Source: own elaboration. 77
5.7 Temporal evolution of position against reference in two families with different geometric behaviors. Source: own elaboration. 78
5.8 Actuation signals inspected after closed-loop simulation with native expert PD. Source: own elaboration. 79
5.9 Vehicle body angular velocities for two classic dataset tracking scenarios. Source: own elaboration. 79
5.10 Visual inspection of a cross-transfer in the circular scenario. 80

List of Tables

4.1 Coverage of the comparative execution. 44
4.2 Supervised force fidelity and associated inclination clipping. Clipping percentages correspond to the activation of the inclination limit before applying the force in the inner loop. 45
5.1 Cases from the classic dataset used for the telemetry plots. 76

Abbreviations and Acronyms

6DOF Six degrees of freedom.

ALVINN Autonomous Land Vehicle in a Neural Network.

BPTT Backpropagation Through Time.

CCW Counter-Clockwise.

CM Center of Mass.

CPU Central Processing Unit.

CSV Comma-Separated Values.

CW Clockwise.

DAgger Dataset Aggregation.

ENU East–North–Up local reference frame.

FIFO First In, First Out.

FRD Forward–Right–Down body reference frame.

GNSS Global Navigation Satellite System.

GPU Graphics Processing Unit.

GRU Gated Recurrent Unit.

ID In-distribution.

IMU Inertial Measurement Unit.

JSON JavaScript Object Notation.

LSTM Long Short-Term Memory.

MAE Mean Absolute Error.

MLP Multilayer Perceptron.

MSE Mean Squared Error.

NaN Not a Number.

OOD Out-of-distribution.

PD Proportional–derivative controller.

PID Proportional–integral–derivative controller.

ReLU Rectified Linear Unit.

RK4 Fourth-order Runge–Kutta integration method.

RMS Root Mean Square.

RMSE Root Mean Square Error.

RNN Recurrent Neural Network.

TOML Tom's Obvious Minimal Language.

UAV Unmanned Aerial Vehicle.

YAML YAML Ain't Markup Language.

ZOH Zero-Order Hold.

Symbols and Units

Unless otherwise stated, vector quantities of position, velocity, acceleration, and force are expressed in the ENU world reference frame, and attitude, angular velocity, moment, and inertia quantities are expressed in the FRD body reference frame.

Reference Frames and State

Symbol	Meaning	Unit
$\mathcal{W}$	World reference frame, with ENU convention.	–
$\mathcal{B}$	Body reference frame, with FRD convention.	–
x_W, y_W, z_W	World frame axes: East, North, and Up.	–
x_B, y_B, z_B	Body frame axes: Forward, Right, and Down.	–
$\boldsymbol{x}$	Vehicle state in the simulation.	–
$\boldsymbol{p}_{\mathcal{W}}$, $\boldsymbol{p}$	Position of the center of mass in the world frame.	m
$\boldsymbol{v}_{\mathcal{W}}$, $\boldsymbol{v}$	Linear velocity in the world frame.	m/s
$\boldsymbol{p}_r$, $\boldsymbol{v}_r$	Reference position and velocity.	m; m/s
$\boldsymbol{a}$, $\boldsymbol{a}_r$	Acceleration and reference acceleration.	$\mathrm{m/s^2}$
$\boldsymbol{a}_{cmd}$	Commanded acceleration from the outer loop.	$\mathrm{m/s^2}$
$\boldsymbol{v}_{w,W}$	Constant wind velocity in the world frame.	m/s
$\boldsymbol{v}_{r,B}$	Relative velocity with respect to the wind expressed in body frame.	m/s

Attitude, Orientation and Rotation

Symbol	Meaning	Unit
$\boldsymbol{q}_{\mathcal{WB}}$	Attitude quaternion relating body and world.	–
$\boldsymbol{q}_{WB,d}$	Desired attitude quaternion used by the inner loop.	–
q_0, q_1, q_2, q_3	Components of the attitude quaternion.	–
$\boldsymbol{q}_e$, $\boldsymbol{q}_{e,v}$	Error quaternion and vector part of the attitude error.	–
$\boldsymbol{\omega}_{\mathcal{B}}$	Angular velocity of the vehicle in the body frame.	rad/s
$\boldsymbol{R}_{WB}$	Rotation matrix from FRD body to ENU world.	–
$\boldsymbol{R}_{WB,d}$	Desired rotation matrix from FRD body to ENU world.	–
$\boldsymbol{x}_C$	Auxiliary horizontal vector constructed from the reference yaw.	–
$\boldsymbol{x}_{B,d}$, $\boldsymbol{y}_{B,d}$, $\boldsymbol{z}_{B,d}$	Desired body frame axes used to build the reference attitude.	–
ψ_r	Reference yaw.	rad
$\otimes$	Quaternion multiplication.	–

Vehicle and Physical Parameters

Symbol	Meaning	Unit
m	Vehicle mass.	kg
g	Acceleration due to gravity.	$\mathrm{m/s^2}$
$\boldsymbol{I}_B$	Inertia matrix with respect to the center of mass.	$\mathrm{kg\,m^2}$
$\boldsymbol{r}_{i,B}$	Position of rotor i with respect to the center of mass.	m
x_i, y_i	Coordinates of rotor i in the body frame.	m
s_i	Sign of the yaw reaction torque of rotor i.	–
τ	Time constant of the actuator first-order lag.	s
$\boldsymbol{M}$	Allocation matrix between rotor thrusts and collective command/moments.	–
$\boldsymbol{D}_B$	Linear drag matrix expressed in the body frame.	N/(m/s)

Forces, Moments and Actuation

Symbol	Meaning	Unit
$\boldsymbol{F}_B$	Applied force expressed in body frame.	N
$\boldsymbol{F}_{i,B}$	Force generated by rotor i expressed in body frame.	N
$\boldsymbol{F}_{d,W}$	Force with subscript d in world frame: desired force in the control loop or drag force in the disturbance model, depending on context.	N
$\widehat{\boldsymbol{F}}_{d,W}$	Desired force predicted by the neural controller.	N
T	Total collective thrust.	N
T_i	Thrust generated by rotor i.	N
$T_{\min}$, $T_{\max}$	Lower and upper bounds of the individual rotor thrust.	N
$T_{\mathrm{req},i}$	Required thrust for rotor i after the mixer.	N
T_{target}	Target average thrust used in the mixer saturation route.	N
$T_{\mathrm{thrust\_part}}$	Collective thrust contribution before applying moments.	N
$T_{\mathrm{moment\_part}}$	Moments contribution before applying rotor limits.	N
ω_i	Angular velocity of rotor i.	rad/s
$\omega_{\max}$	Maximum rotor angular velocity used in scenarios.	rad/s
rpm	Revolutions per minute, auxiliary reading unit for rotor speeds in telemetry.	rev/min
k_f	Rotor thrust quadratic coefficient.	$\mathrm{N\,s^2\,rad^{-2}}$
k_m	Rotor reaction torque quadratic coefficient.	$\mathrm{N\,m\,s^2/rad^2}$
Q_i	Reaction torque of rotor i.	N m
$\boldsymbol{\tau}_B$	Applied moment in the body frame.	N m
τ_x, τ_y, τ_z	Roll, pitch, and yaw moment components.	N m
$\delta_{\min}$, $\delta_{\max}$, δ	Collective thrust slack limits and shift in the mixer.	N
$\times$	Vector cross product.	–

Classic Control and Metrics

Symbol	Meaning	Unit
$\boldsymbol{e}_p$, $\boldsymbol{e}_v$	Position and velocity errors.	m; m/s
$\boldsymbol{e}_q$	Attitude error used by the inner loop.	–
$\boldsymbol{K}_{p,p}$, $\boldsymbol{K}_{d,p}$	Proportional and derivative gains of the position loop.	s^{-2}; s^{-1}
$\boldsymbol{K}_{p,a}$, $\boldsymbol{K}_{d,a}$	Proportional and derivative gains of the attitude loop.	N m; N m s
J	Score used to select PD controllers.	–
e_k	Scalar tracking error at sample k.	m
e_{RMSE}, e_{rms}	Root mean square error of position.	m
e_{MAE}	Mean absolute error of position.	m
$e_{\max}$	Maximum position error.	m
e_{att}	RMS of attitude angles in the tuning score.	rad
u	Normalized control effort used in the score.	–
s, d	Fractions of samples with command saturation and degradation.	–
$\overline{T}$	Average thrust used in the control effort term.	N
N	Number of samples used in a metric.	–

Time, Sampling and Trajectories

Symbol	Meaning	Unit
t	Continuous time used in trajectories and references.	s
k	Discrete time index of a sample.	–
Δt	Integration, control, or sampling period in each context.	s
f_{phys}, f_{ctrl}, f_{tel}	Physics, control, and telemetry frequencies.	Hz
T_p, T_c, T_t	Periods associated with physics, control, and telemetry.	s
L	Time window length used by recurrent models; in waypoint profiles, it can indicate the scalar segment length when specified in a figure.	samples; m
c_x, c_y, c_z	Coordinates of the center of a Lissajous trajectory.	m
A_x, A_y, A_z	Spatial amplitudes of a Lissajous trajectory.	m
$\omega_x, \omega_y, \omega_z$	Angular frequencies per axis in a Lissajous trajectory.	rad/s
$s(t)$	Scalar progress coordinate used to generate profiles between waypoints.	m

Neural Networks and Optimization

Symbol	Meaning	Unit
B	Batch size for training or inference.	samples
$\boldsymbol{x}_k$	Input vector to the network at time k.	norm.
$\boldsymbol{y}_k$	Output of a recurrent network at time k.	norm.
θ	Set of trainable parameters of a neural network.	–
$\mathcal{L}(\theta)$	Loss function dependent on trainable parameters.	–
n	Iteration index in gradient descent.	–
η	Learning rate in gradient descent.	–
∇_θ	Gradient with respect to the network parameters.	–
$\boldsymbol{W}$, $\boldsymbol{b}$	Weights and biases of a neural layer, including specific matrices and biases for recurrent layers or gates.	norm.
$\boldsymbol{z}^{(\ell)}$, $\boldsymbol{a}^{(\ell)}$	Pre-activation and activation of layer ℓ.	norm.
ℓ	Neural layer index.	–
$\boldsymbol{h}_k$	Hidden state of a recurrent network at time k.	–
$\boldsymbol{c}_k$	Cell state of an LSTM at time k.	–
$\boldsymbol{f}_k$, $\boldsymbol{i}_k$, $\boldsymbol{o}_k$	Forget, input, and output gates of an LSTM.	–
$\tilde{\boldsymbol{c}}_k$	Candidate cell state of an LSTM.	–
$\boldsymbol{z}_k$, $\boldsymbol{r}_k$	Update and reset gates of a GRU.	–
$\tilde{\boldsymbol{h}}_k$	Candidate hidden state of a GRU.	–
$\phi, \sigma, \tanh$	Generic non-linear activation, sigmoid, and hyperbolic tangent; ϕ can also denote the roll angle when context is attitude.	–
$\odot$	Hadamard (component-wise) product.	–
$\mathbb{R}^n$	Real vector space of dimension n.	–
$\|\cdot\|$	Vector norm.	–

Declaration of Generative AI Usage

I explicitly declare that during this Bachelor's Thesis, I have used generative artificial intelligence tools. AI has been part of my way of working on this project, always as an auxiliary tool and under my responsibility, but in a much deeper way than copying text into a chat and accepting an answer, as is the case for the vast majority of classmates I know and the common usage observed. This is why I wish to expand on this section.

My relationship with these tools began long before this work. In 2021 and 2022, I was already following the first advances in generative text and image models with interest. I remember generating images in Google Colab, seeing results that were still imperfect but good enough to sense that this was the beginning of something much larger and that it would have a breakthrough impact. I was fascinated to see dunes, something resembling an abstract car, or an otherwise strange scene, but generated by a computer. I also came across text models such as GPT-2, which, viewed from today's perspective, was limited and failed often, but already hinted at the significant developments to follow.

Then came more accessible text and image tools, such as ChatGPT in late 2022. I experienced it as just another development, having been involved in this field for some time, but it was not just another step; it marked the first major turning point. From then on, the way many people would write, search, code, learn, and organize work changed. A similar trend occurred with images, with noticeable improvements appearing every few months, to the point where an image created with AI, requested with proper criteria, cannot easily be distinguished from one produced by other means. This is also why I consider it important to have digital identification mechanisms, such as Google's SynthID, also implemented by other companies like OpenAI, to maintain the traceability of generated content and separate real content from that generated by AI on the internet.

In 2023, 2024, and 2025, I continued to watch generative AI transition from a curiosity into a feasible computing tool. I used ChatGPT, Google Bard, and later Gemini, Mixtral, and other emerging models, including some Chinese models that began to demonstrate that serious capabilities could be achieved with optimization. Even so, for quite a while I still viewed it as just another tool within my general interest in technology, similar to how I might be interested in new processors, graphics cards, computers, or any other technical advance.

The shift in perspective came at the end of the 2024–2025 academic year, following a talk at the School of Engineering that made me stop seeing it merely as something

I followed for leisure. I understood then that I needed to truly train myself, because it was not a passing fad, but a competency that would become part of engineering work. In June 2025, I decided to learn how to use these tools with sound criteria and also to learn about them. While starting to conceptualize this Bachelor's Thesis, I took introductory courses on artificial intelligence and networks, several of them from Google, which were basic but very useful for establishing fundamentals. This training influenced the topic of this thesis and the concept of imitation control.

At that time, programming tools using AI were still in a very early stage. Claude Code had appeared in February 2025 alongside Claude 3.7 Sonnet, but it was still a niche tool, used by few and closer to completing or assisting with code than driving a full workflow. Models in the Claude Sonnet 3 family, and later Sonnet 3.5 and 3.7, were starting to become interesting for programming, but they did not yet have the capability to become a complete workspace companion. Codex also began to gain ground in 2025, first as a code agent and later with improvements for agentic programming and real-world tasks. Meanwhile, I was learning how to speak to these tools, how to prompt them, how to correct them, and how to translate my ideas into verifiable work.

I also began to learn about the entire ecosystem that was forming around agents. *Agent skills* emerged, which are like small, reusable prompts or instructions to reproduce workflows. I began to understand MCPs (*Model Context Protocol*), which allow connecting an AI with external tools, documentation, searches, repositories, or local systems. More integrations with editors also appeared, along with *hooks* systems and ways to place boundaries on agent behavior. With all these tools, you can provide context, rules, and control gates to an AI acting freely within a folder on your computer.

During the 2025–2026 academic year, I used every assignment and lab work as an opportunity to learn how to work better with AI. I improved the way I wrote instructions, explained intent, requested reviews, and distinguished when a response was useful versus when it merely sounded convincing. I also saw the capabilities of these tools improve significantly in web design, SVG, diagrams, and figures. This was highly relevant for the thesis, as all figures are LaTeX TikZ or Python code, which can be modified, compiled, or executed and reviewed.

The second major turning point for me arrived between October and December 2025. Advanced usage was no longer about chatbots, but agents capable of planning, editing files, running commands, testing, failing, correcting, and trying again. Google Antigravity appeared in November 2025 as a development environment designed from the ground up to work with agents; OpenAI continued to push Codex as an agentic workflow tool; Claude Code and the Opus models became clear benchmarks in coding; and tools were appearing to connect agents more deeply with computer programs. In

2026, tools from the Grok Build ecosystem were also added, which have also been used in this thesis.

With all that background, I approached the development of this thesis convinced that I was going to use these tools. Not as a way to avoid work, but because it was the way of working to achieve results of much higher quality and scope. Just as it would not make sense to write a thesis in the 2010s by looking up everything in a paper encyclopedia and avoiding the internet, it did not seem reasonable to ignore tools that allow programming, documenting, reviewing, and experimenting at a much faster pace. The key was to build the project in such a way that the AI had context, documentation, rules, and constant traceability.

That is why this work was developed on my computer, using editors and tools prepared for agents. I used Google Antigravity as a code editor with integrated Gemini models, OpenAI Codex and Grok Build, NotebookLM to organize and consult documentation, standard chatbots as search tools on steroids, and the agent and code ecosystem tools mentioned above. I also used MCPs to extend the AI's capabilities, the most relevant being Context7 to have up-to-date documentation on code libraries.

What I want to make clear is the workflow: it is not 'asking a chatbot a question' and pasting it, but rather 'giving an agent a goal, context, rules, tests, and a way to verify if what it did makes sense.' When the AI only receives a snippet of code copied into a chat, it does not understand the project well, does not know past decisions, and cannot maintain traceability. In this thesis, I worked differently, with a repository set up so that the AI had context from day one and could act as my work companion.

Additionally, I used speech-recognition tools to verbalize ideas and convert them into text, as the most natural way to explain an intention is not to write it from scratch, but rather to speak it first and organize it later.

In this work, AI has been used primarily to translate intent into code and documentation. This includes Python code, configuration files, Markdown documentation, tests, commands, helper scripts, LaTeX, TikZ, diagrams, and figures. The code is not just Python; there is code in a figure made with TikZ, in a LaTeX template, a YAML file, or a JSON telemetry log. In all these cases, the AI served to accelerate the execution of decisions that I made, reviewed, and corrected.

The methodology followed was not asking 'Hello, write my thesis.' The repository was designed from the very first document to work with the AI as a companion, with specifications, plans, reviews, up-to-date documentation, requirements traceability, tests, and decision logs. The technique I followed is known as *Spec-Driven Development*: before making an implementation, a specification of what is to be achieved is defined, discussed, edge cases are reviewed, and the initial intent is converted into a set of concrete requirements.

The Q&A part with the model was central. The process started from an idea of mine, extensive but incomplete. The agent asked me questions, I answered, assumptions were corrected, my intentions were refined, thereby squaring the intent into a clear specification and plan. This recorded which idea was to be executed, what the limits were, and what decisions were made. After implementation, I reviewed the result, compared outputs, requested additional reviews, and corrected what did not fit. Thanks to the AI, we iterated faster, but the decisions regarding the scope of the work, physical extension and criteria, interpretation of results, and general direction were entirely mine.

In the public repository, all the supporting documentation generated to understand what was being done can be observed. There are plans, specifications, reviews, simulator documentation, traceability tables, requirements, maintenance instructions, working principles, and AGENTS.md files that explain to agents how they should behave within the project. People always say that AI must be given context, but I never read about anyone forcing the AI to maintain full experimental traceability.

The first iteration of the simulator was the clearest example of this methodology. I did not start with 'build a simulator for the thesis,' but with a specific intent: how I wanted the modules to be organized, what options the program should have, what type of scenarios I wanted, what data should be generated, and how I wanted to structure the execution. The AI helped convert that intent, expressed in natural language, into Python code and configuration files such as JSON, YAML, or TOML.

After that first version, the generated output was not simply accepted. There was a lot of *refinement* to adjust cases, change parameters, detect bugs, improve scenarios, review outputs, run tests, rewrite parts, and verify again. The agent was left to work on the computer: editing files, launching commands, running tests, reading errors, and proposing the next fix. The AI was not answering in a chat; it was controlling the development environment to meet a verifiable objective.

Updated documentation of the simulator was created regarding architecture, usage, scenarios, telemetry, validation, requirements, testing, and the status of each part. This documentation served me, anyone who might consult the project on GitHub, and also the AI itself, because it provided context about the project. If there was a question about a module, a scenario, a metric, or a design decision, the answer depended on the documentation previously written in the repository. Thus, progress was made in an orderly manner, through multiple iterations among several agents, differentiating between implementation and review, which would have been impossible without that context.

The resulting code is structured in a modular fashion and with software best practices, but without obfuscating the important part of the work: physics, aerospace, and control.

Functions related to dynamics, reference frames, integration, control, or network training have been documented with clear names, comments, and explanations.

The AI was also used to create auxiliary tools that are not the core of the simulator but facilitate working with it: scripts, commands, analysis utilities, results generation, documentation review, and support for reproducibility.

I have also used AI in the thesis manuscript to support essentially programming-like tasks. The writingthat is, the intent of what is to be said and defendedis entirely mine. But LaTeX is code, and it is written in the same editor as Python. That is why the AI worked with the preambles, table and figure parameters, references, layout, and general figure handling.

The latter are a particularly important case. None of them are images generated by image-generative AI; they are diagrams made with TikZ (LaTeX) and Python, which is code that is modified and executed like any other. I used Codex and Gemini to generate, adjust, and refine these figures, **always** starting from my own intent: what the diagram should show, what elements it should contain, and a proposed layout and color palette. All figures have a Markdown sheet detailing their intent, conventions, and traceability.

Without these tools, I would not have been able to make figures with this level of quality and flexibility. The AI understands TikZ and LaTeX well because, at their core, they are code. Thus, I have been able to include custom-made figures in the work instead of generic images. In my opinion, this shows the real potential: allowing anyone, including all researchers and scientists, to use them for artifacts that they would otherwise not make. The expert defines the what and the why; the AI fills in the how.

Another relevant aspect of the work has been verbalization. I have often used speech as the primary interface to explain ideas, as it is the most natural way to communicate intent. This is not exactly generative AI, but rather speech recognition and workflow integration, yet it was important for this thesis. I used my own local tools, such as Voice Stall, to transcribe voice to text and express long ideas before converting them into plans or instructions for agents.

In conclusion, yes, I have used generative AI for programming, documentation, information retrieval, linguistic coherence, figure generation and tuning, code review, repository analysis, script writing, LaTeX work, refinement of specifications with plans, and execution of agentic workflows. I assume full responsibility for the originality, veracity, rigor, and authorship of this document. In my view, this usage has been part of my learning as a future Aerospace Engineer and represents a professional competency that will be in high demand. Thanks to using it this way during the academic year, I can explain which tools I use, the procedures, the limits, the review process, and how AI can be used without sacrificing human originality. I can present this repository and

not just claim that I used AI, but show how, thereby building trust with others to share knowledge.

Just as it would make no sense in 2015 to write a paper ignoring the internet and searching in paper encyclopedias, it does not seem reasonable to ignore tools that are already a real part of engineering work in multiple companies, most prominently in the United States. The difference lies in the usage: a substitute or a capability multiplier. In this thesis, I have used it as the latter. For me, this is the path forward: not to ban or demonize generative AI, but to use it responsibly to increase the complexity of what can be achieved, program better with less experience, and produce high-quality material.

1. Introduction

1.1. MOTIVATION

The autonomous trajectory tracking of a quadcopter requires having a sufficient estimate of its state with respect to the environment and the reference, and transforming the tracking error into an actuation that stabilizes the vehicle while guiding it along the designated path. The difficulty does not lie in minimizing an instantaneous error with respect to a fixed position; rather, the reference evolves over time, and the controller acts on a rigid body with coupled dynamics and exposure to internal and external disturbances. Therefore, it must coordinate translation and attitude without losing stability or failing in the displacement.

This coupled nature motivates the use of a two-level control architecture [1–3]. In this case, a cascaded PD first transforms position and velocity errors into a desired force in the local East–North–Up (ENU) reference frame. Subsequently, the inner loop converts this force, along with the yaw reference, into a desired attitude, thrust, and moments in the Forward–Right–Down (FRD) body reference frame. Finally, the mixer distributes these commands among the four rotors. Tuning a single controller for maneuvers of a diverse nature can impose a compromise between, for example, maintaining a position and tracking a geometry with continuously changing motion. Specialization avoids this compromise but scales the number of controllers to be tuned and validated, as well as the constant switching of gains during operation. For this reason, in this work, it is decided to build a set of controllers with a PD for each trajectory family, and it is studied whether a neural implementation can condense the joint operation of the experts. This idea of having different control parameters depending on the operating regime is closely related to gain scheduling [4].

The comparison between the two approaches requires an environment where the model, references, disturbances, random seeds, actuation limits, and metrics remain under control. The developed 6DOF simulator provides this testbed where all controllers share the vehicle, state observation, and actuation chain. Furthermore, each run preserves traceability of the configuration to reproduce the experiment, as well as the telemetry necessary for subsequent review. This platform does not intend to replace a future in-flight validation, but rather to isolate the differences in a controlled and reproducible environment.

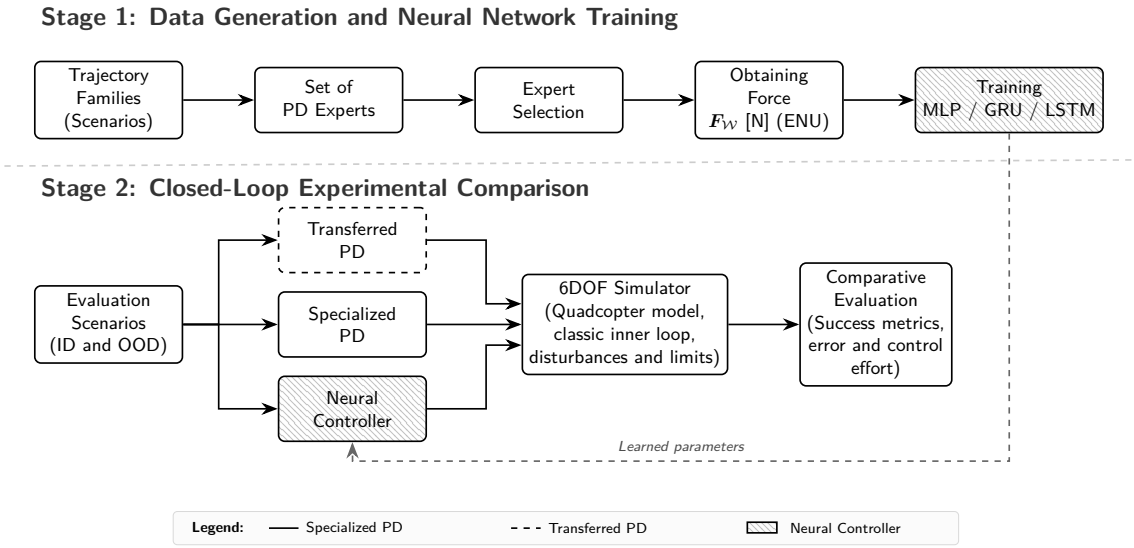


Figure 1.1. Conceptual diagram of the experimental workflow of the work. It consists of trajectory generation, specialization of classic controllers, training of neural networks, and comparative evaluation. Source: own elaboration.

In Figure 1.1, the experimental logic structured in two stages is detailed visually. The first stage illustrates obtaining the expert controllers and generating data to train the neural networks. The second stage shows the comparative evaluation of the three control references in the simulator under the same conditions: conditions within the training distribution or *in-distribution* (ID), and conditions outside said distribution or *out-of-distribution* (OOD).

Problem Addressed

The main problem addressed in this work consists of determining whether a single controller using neural networks, trained on data generated by PD controllers optimized for different trajectory families, can replace this set of controllers when evaluated under the same references and disturbances. The comparison should indicate whether the neural implementation maintains competitive behavior under known conditions and whether it possesses better transfer to composite, new, and varied trajectories.

Three references are compared. The *specialized PD* is the controller whose parameters have been fixed for the family to which the trajectory belongs. A *transferred PD* is one of those same controllers applied, without retuning, to another family. The neural system replaces the outer loop and performs a *desired force prediction* from the observation and the reference, producing a three-dimensional vector in the ENU reference frame and preserving the classic inner loop. In this way, the network does not transmit commands directly to the rotors nor does it generate moments in the FRD reference frame.

On one hand, *competitive performance* is defined as the ability to preserve, in closed-loop, the completion of the trajectory while obtaining tracking errors, control effort, and protection usage comparable to those of the relevant reference. The metrics and evaluation criteria associated with these concepts are defined in the epigraph on metrics, success criteria, and analysis. On the other hand, *transfer* is evaluated by applying the controller outside the operating conditions for which it was originally tuned. To structure this analysis, the evaluation distinguishes between in-distribution (ID, *in-distribution*) and out-of-distribution (OOD, *out-of-distribution*) scenarios, whose attributes are defined later in section 3.5. In the ID case, scenarios of the same type as those used to tune the PDs and train the networks are evaluated, albeit with parameters reserved for testing. In the OOD case, unseen dynamic variations, external disturbances, or new geometries are imposed. Within the latter geometric references, a distinction is made between compositions of already known trajectories and completely unseen geometric families.

It is important to point out that this study does not intend to demonstrate the complete superiority of any particular architecture, nor to guarantee the robustness of the system against any physical disturbance or its suitability for real flight. Likewise, the neural controller is not credited with the attitude stabilization of the inner loop nor the management of protections imposed through saturations and force limits. The scope of this work is limited to a traceable experimental comparison, carried out on the same physical model and environment, between gain specialization, classic controller transfer, and neural imitation.

1.2. HYPOTHESES AND OBJECTIVES

The central hypothesis holds that a single neural system, trained by imitation to predict the desired force of the outer loop, can achieve competitive tracking capacity compared to the specialized PD of each family under known conditions and transfer better than the PDs applied outside their native family, while preserving the classic inner loop in both.

The general objective is to develop a reproducible simulation environment that allows determining whether the neural imitation controller can replace a set of specialized PD controllers and offer better performance in trajectories not used during gain tuning and training.

This objective is broken down into the following specific objectives:

1. Formulate and implement a 6DOF rigid-body model for a quadcopter, with appropriate reference frames, actuators, simplified disturbances, and differentiated update frequencies;

2. Define trajectory families and parameters that allow repeating the experimental conditions;

3. Implement a cascaded PD and obtain, through a deterministic tuning process over the training scenarios, a specialized controller for each family;

4. Assess the specialization of the PDs and their transfer capability by applying each controller to families other than its native one;

5. Train MLP, GRU, and LSTM to predict the desired force and compare the absence or presence of temporal memory under the same conditions;

6. Evaluate the quality of the prediction in supervised execution and, separately, the closed-loop tracking in ID and OOD conditions; and

7. Preserve sufficient results to reconstruct the experiments.

1.3. STRUCTURE OF THE DOCUMENT

The contribution of the work is the integration of four elements: a 6DOF simulator, a set of trajectories and parameters through scenarios, a cascaded PD selection process specialized by family, and a closed-loop comparison of MLP, GRU, and LSTM. The interest lies in building an environment that allows distinguishing which properties originate from the specialized PD, what the network learns, what the classic inner loop contributes, and what the limits of this approach to the problem are.

After this introduction, the document is structured in four blocks: the state of the art on modeling, classic control, and imitation control; the methodology, which details the simulator and the evaluated control architectures; the analysis of experimental results; and, finally, the conclusions and future lines of development.

2. State of the Art

2.1. QUADCOPTER SIMULATION AND MODELING

The quadcopter is a six-degree-of-freedom aerial vehicle whose position and attitude evolve in a coupled manner, and whose actuation originates from a limited number of rotors. This combination makes trajectory tracking a particularly notable problem within unmanned systems, since the vehicle must modify its orientation to achieve the required translation while maintaining stability and respecting actuation limits. For this reason, literature on small UAVs tends to describe the problem using rigid-body models, well-defined reference frames, and cascaded control architectures [1–3].

In a 6DOF rigid-body model, the vehicle state combines position, velocity, attitude, and angular velocity. Translation is achieved by summing gravity, aerodynamic forces, and the force generated by the rotors, transformed from the body reference frame to the local frame. Rotation is described by Euler's equations, where the moments applied by the rotors modify the angular velocity as a function of the inertia matrix. This type of formulation allows studying the interaction between position tracking, tilt, actuator limitations, and stability, although it does not reproduce all the aerodynamic phenomena of flight.

The term *6DOF* indicates that the vehicle has three degrees of freedom in translation and three in rotation. However, it cannot independently impose accelerations on each axis and arbitrary attitude at the same time. In a traditional quadcopter, the rotors generate thrust along the vertical axis of the vehicle. Lateral force appears by tilting the quadcopter and, thereby, orienting a portion of the thrust toward the desired horizontal direction. This is the physical reason why position tracking and attitude control are said to be coupled.

Reference frames are an essential part of this formulation. In aircraft and aerial vehicles, a distinction is made between a reference frame associated with the environment and another associated with the vehicle. The former allows expressing position, velocity, and trajectory references in an absolute manner, while the latter facilitates determining the moments and forces generated by the rotors. The transformation between the two is performed via a rotation matrix associated with attitude. In this context, quaternions are a common alternative to Euler angles as they represent orientation without introducing kinematic singularities at specific attitudes [5]. Euler angles remain useful for visualization and for setting operational limits, but they are not the most robust method for attitude computation and propagation during numerical integration.

Force generation in a quadcopter is modeled by a quadratic relationship between the rotor angular velocity and thrust. Each rotor also contributes to the total moment

through its position relative to the center of mass and the reaction torque associated with its rotational direction [1,6]. The actuator mixer solves the inverse problem: given a total thrust and three moments, it calculates the individual thrusts that the rotors should produce. When the solution requires negative thrusts or thrusts exceeding the maximum available, command saturations occur. In this case, the controller may have calculated a coherent yet unrealizable command. Therefore, the comparison of controllers is not limited to the position error, but also records when an actuation could not be carried out.

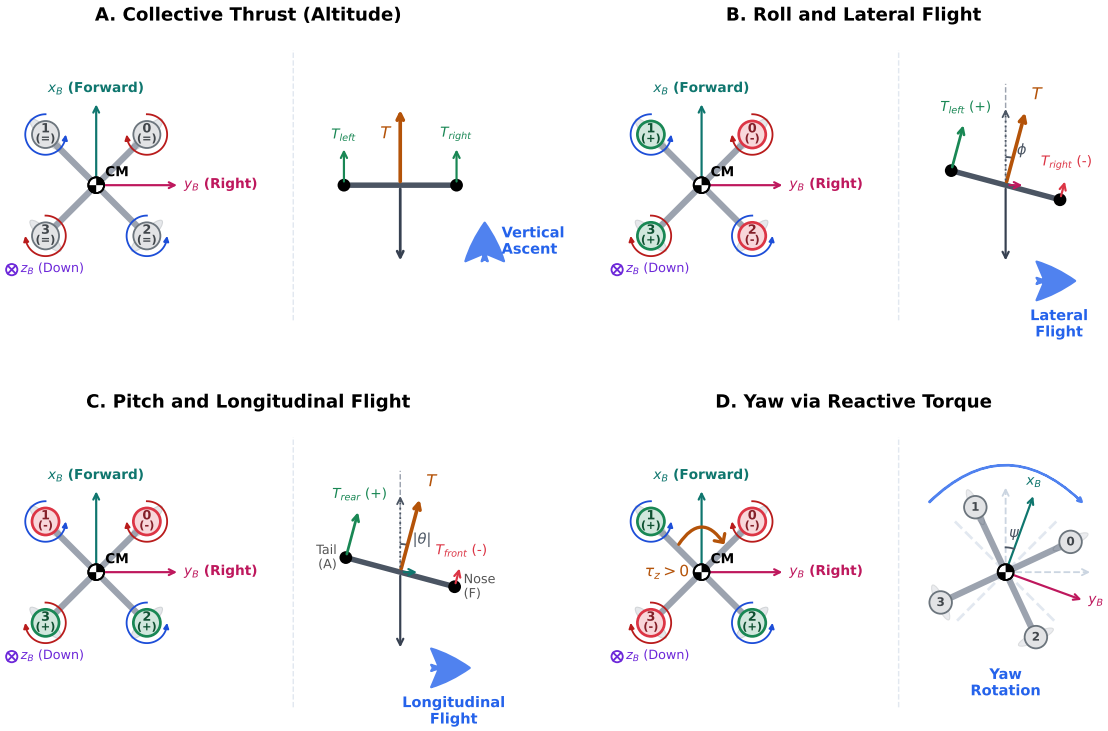


Figure 2.1. Actuation and motion mechanisms in a quadcopter. For each maneuver, the top view of the vehicle in layout (left) is shown with the rotational direction of each rotor and indications of thrust imbalance (+, −, =), and the resulting physical effect in side view (right). Tilting the vehicle (ϕ, θ) redirects the total thrust vector (T), generating the lateral (F_y) or longitudinal (F_x) force components that enable horizontal translation. Source: own elaboration.

Actuation management can be understood from four basic combinations (see Figure 2.1). If all four rotors increase or decrease their thrust in a similar manner, the total thrust changes, and therefore the vertical acceleration in a level attitude changes. If thrust increases on one side relative to the opposite side, a roll moment appears, and if the difference occurs between the front and rear parts, a pitch moment appears. Yaw is obtained by leveraging the reaction torques of rotors spinning in opposite directions. In summary, each rotor contributes a force T_i, and the assembly produces an output of the type

$$\boldsymbol{u} = [T\ \tau_x\ \tau_y\ \tau_z]^{\mathsf{T}},$$

where T is the collective thrust and τ_x, τ_y, and τ_z are the moments in the FRD body frame.

The time integration of the model requires discretizing the differential equations. Runge–Kutta methods, and in particular RK4, are a classical option when seeking a favorable trade-off between precision and computational cost [7]. In digital control environments, it is common to separate the physics integration frequency, the controller update frequency, and the telemetry logging frequency. Between two control cycles, the last calculated command is kept constant using a zero-order hold (ZOH), thus approximating the behavior of a digital system acting on continuous dynamics [8].

Proven platforms exist for simulating aerial vehicles. MATLAB, Simulink, and the UAV Toolbox provide tools to design, simulate, test, and deploy UAV applications, including scenarios, sensors, autopilots, and test workflows both on a computer and in hardware-in-the-loop configurations [9]. In the open-source domain, RotorPy offers a Python multirotor simulator oriented toward education and research, featuring 6DOF dynamics, aerodynamics, actuators, sensors, wind, and validation against real data [10].

The present work does not need to reproduce the full breadth of these platforms; instead, complete modularity and traceability are the most important aspects. The previously discussed parts are implemented: 6DOF dynamics, specific reference frames, attitude using quaternions, quadratic control law, mixer with saturation, simplified disturbances, integration, and sufficient telemetry to reproduce and review each run.

2.2. PROGRAMMING AND MACHINE LEARNING ENVIRONMENTS

The choice of an implementation environment conditions the available tools, reproducibility, and ease of inspection of the work. Python has established itself as an environment for scientific programming and simulations, with an open license and an extensive ecosystem of libraries. Its interest in this context does not lie in being the most efficient option for a future onboard flight, but rather in allowing the model to be expressed clearly, generating data, running comparisons, and connecting the simulator with machine learning tools.

PyTorch provides tensors, loss functions, and optimizers that allow training supervised and recurrent models on CPU or GPU [11]. Furthermore, it includes standard implementations of linear layers, GRU, LSTM, MSE loss, and Adam, allowing work at a higher level and avoiding the risk of introducing errors in widely established components.

Reproducibility also depends on environment management and the files related to experiments. Tools such as `uv` allow declaring, installing, and running Python environments in a repeatable and simple manner [12]. The use of human-readable

configuration files, such as YAML (*YAML Ain't Markup Language*, a readable text format to describe structured data using keys, lists, and values), makes it easy for scenarios, seeds, trajectories, controllers, and termination conditions to be reviewed without modifying code in a visual and simple way. Compared to JSON, which is widespread but more rigid for reading and writing by a researcher, and TOML, which is clear for relatively flat configurations, YAML offers a convenient syntax for describing hierarchical structures with lists, dictionaries, and nested blocks [13–15]. Finally, Git and GitHub provide common support for change history, code distribution, and traceability of experiments [16]. Thanks to this toolset, auditing and reproducing experiments by anyone is facilitated.

2.3. CLASSIC CASCADED CONTROL AND GAIN SCHEDULING

Classic quadcopter control is typically organized in cascade. The outer loop works on position and velocity, and calculates a desired force or acceleration in the ENU world reference frame. The inner loop transforms this translational intent into a desired attitude and stabilizes the vehicle using moments in the FRD body reference frame. This separation is also observed in speed: translational dynamics are slower and require fewer corrections than angular dynamics [1–3].

In this architecture, the outer loop does not directly decide the rotor speeds; rather, its output is a desired force in the local frame. If the vehicle is below the reference, the force will have a larger vertical component; and if it needs to move horizontally, it will have a component in this direction. The inner loop takes the force, calculates what vehicle orientation would allow generating it with the available thrust, and modifies the attitude toward that orientation. The reference yaw completes the orientation, since the thrust direction only fixes the direction of the vertical axis.

PD and PID controllers remain the benchmark due to their simplicity, interpretability, and low computational cost. A proportional term corrects the position or attitude error, a derivative term dampens the response based on velocity or angular velocity, and an integral term can reduce steady-state errors. However, the integral action can accumulate command during saturations, thus requiring anti-windup protection mechanisms. In this case, a PD structure combined with trajectory acceleration feedforward and gravity force compensation results in a more feasible implementation.

Gain tuning can be approached in several ways. Manual tuning provides intuition but hinders traceability if all iterations are not preserved. Classic Ziegler–Nichols rules offer systematic procedures for certain classes of systems, although they do not directly represent a 6DOF quadcopter with saturations and spatial trajectories [17]. It is also

possible to use controllers on models linearized around operating points, or to search for gains using numerical optimization evaluated in closed-loop simulation. This latter approach is of special interest when the objective is to compare various candidates under the same trajectories, limits, and metrics.

When a single set of gains does not yield a satisfactory result across the entire operating range, a classic response is gain scheduling. This technique selects or interpolates control parameters according to certain variables describing the operating point, so that different flight regions can use different tunings [4]. Conceptually, a set of specialized controllers by trajectory family is related to this idea, as there are several experts and the most appropriate one is applied for each condition. However, in this work, the neural implementation does not perform an explicit gain interpolation according to fixed criteria; instead, it learns to predict the force of the outer loop from the same data used by the PD.

2.4. IMITATION LEARNING AND HYBRID CONTROL

Imitation learning aims to train a system using data generated by an expert system. In its most direct form, this behavior cloning poses a supervised problem: the system's observations are received, and it learns to reproduce its actions. This concept has antecedents in autonomous driving, such as ALVINN, where a network learned to produce steering commands from human examples [18]. In robotics and control, imitation avoids the design of a complete reward function and allows behavior transfer from an already available controller.

However, behavior cloning has a significant limitation: the data comes from states visited by the expert, but during the execution of the learned policy, small errors can accumulate, leading to states not represented in the training data. This discrepancy between the training distribution and the one induced by the learned system is known as distribution shift (covariate shift). Methods such as DAgger [19] reduce this problem by aggregating data from states visited by the neural system and querying the original expert system again. Although this work does not implement DAgger, it is important to remember that a low supervised loss during training does not guarantee good behavior in closed loop.

In physical control applications, a machine learning policy can replace the entire controller or only a part of the architecture. Hybrid approaches preserve classic elements when these contribute stability, limits, or interpretability. In this case, a network learns the behavior of the outer loop, while the inner loop, mixer, and protections remain classic. However, this introduces the risk of erroneously attributing effects to the network, meaning the contributions of each element must be identified and separated.

2.5. NEURAL NETWORKS FOR CONTROL

The neural networks used in this work are formulated as supervised regression models. The input is a vector or sequence of vectors constructed from the observation and the trajectory reference, while the output is a three-dimensional force in the ENU world reference frame. During training, the parameters are adjusted to reduce the difference between the predicted force and the force commanded by the specialized PD. This formulation requires representing the data as tensors, grouping samples into batches, and optimizing a differentiable loss function via backpropagation [11, 20].

The training of a network is not determined solely by the architecture and the data. It also depends on hyperparameters—decisions set before the tuning of weights and biases that govern how the optimization is performed. Among these are the learning rate, batch size, maximum number of epochs, and stopping criteria. Unlike the trainable parameters of the network, these values are not learned but are selected as part of the design.

A batch is the set of samples processed together before calculating a loss and updating the model parameters. Its size is usually denoted by B; larger values provide a more stable estimate of the gradient, while smaller values introduce more variation between updates and require more iterations. An epoch corresponds to a complete pass through the training samples. Early stopping halts training when the validation loss stops improving, and patience indicates how many consecutive epochs without improvement are tolerated before interrupting the process.

A neural network does not store explicit rules of the *if–then* type; instead, it learns weights and biases that transform inputs into outputs. In supervised regression, each sample provides an input–target pair, and training modifies the parameters to minimize a loss. If θ groups all the parameters of the network and $\mathcal{L}(\theta)$ measures the average error, gradient descent updates the parameters in the direction opposite to the gradient:

$$\theta_{n+1} = \theta_n - \eta \nabla_\theta \mathcal{L}(\theta_n),$$

where η is the learning rate. A value that is too large can cause the training to oscillate or diverge; one that is too small can make it excessively slow. Optimizers like Adam adapt the effective update step size using accumulated gradient information, but they still depend on a base learning rate [21].

2.5.1. Feedforward Networks

A feedforward network computes the output through a succession of layers applied in a single direction, from the input to the output prediction. In a linear layer, the output is obtained by multiplying the input by a weight matrix and adding a bias vector.

If only linear transformations were chained, the model would remain equivalent to a global linear transformation. This is why non-linear activation functions are introduced between the layers. The Rectified Linear Unit (ReLU) is a common activation due to its simplicity and because it facilitates training compared to the saturations inherent in other functions [22].

For a general layer, the operation can be written as

$$\boldsymbol{z}^{(\ell)} = \boldsymbol{W}^{(\ell)}\boldsymbol{a}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}, \qquad \boldsymbol{a}^{(\ell)} = \phi(\boldsymbol{z}^{(\ell)}),$$

where $\boldsymbol{W}^{(\ell)}$ and $\boldsymbol{b}^{(\ell)}$ are weights and biases, and ϕ is a non-linear activation. Depth is given by the number of layers, while width is associated with the number of units per layer.

The multilayer perceptron, or MLP, combines several linear layers with non-linear activations. Theoretically, feedforward networks with enough units can approximate a wide range of continuous functions [23]. Thus, the MLP is used as the memoryless neural baseline, so that two identical inputs produce the same force regardless of what occurred in the previous step. This feature makes it simpler and computationally cheaper than recurrent architectures, but it also limits its ability to use temporal information when lags or transient states exist.

2.5.2. Recurrent Networks

Recurrent neural networks (RNNs) extend the feedforward formulation through a hidden state that is updated over time. At each step, the model processes the current input and a compact representation of the prior history. Thus, the output depends not only on the error at that instant, but also on how that state was reached.

A basic RNN can be summarized by

$$\boldsymbol{h}_k = \phi(\boldsymbol{W}_x\boldsymbol{x}_k + \boldsymbol{W}_h\boldsymbol{h}_{k-1} + \boldsymbol{b}_h), \qquad \boldsymbol{y}_k = \boldsymbol{W}_y\boldsymbol{h}_k + \boldsymbol{b}_y.$$

The vector $\boldsymbol{x}_k$ is the input at the given step k, $\boldsymbol{h}_k$ is the hidden state summarizing the processed history up to that step, and $\boldsymbol{y}_k$ is the output. During training, the network is unrolled in time, and the gradient is backpropagated through the temporal sequence. This procedure, known as backpropagation through time (BPTT), allows tuning the same parameters from the errors observed in the temporal sequence.

However, the basic RNN presents difficulties in learning long temporal dependencies. During backpropagation, gradients can vanish or explode, hindering the preservation of relevant information over a large number of steps. This is why modern recurrent architectures feature internal gates to regulate which information is kept, updated, or

discarded [24]. Figure 2.2 illustrates the fundamental difference between the static and temporal approaches of these two types of architectures.

Long short-term memory (LSTM). The LSTM architecture introduces a cell state and several gates controlling the input, forgetting, and output of information. Its goal is to facilitate the network's retention of relevant information over more time steps than a simple RNN, mitigating the problem of vanishing gradients [25]. In the context of trajectory tracking, this memory is beneficial if the recent history contains information about delays or error evolution. However, it also causes an increase in network parameters and can introduce sensitivity to temporal patterns that are poorly represented in the training dataset.

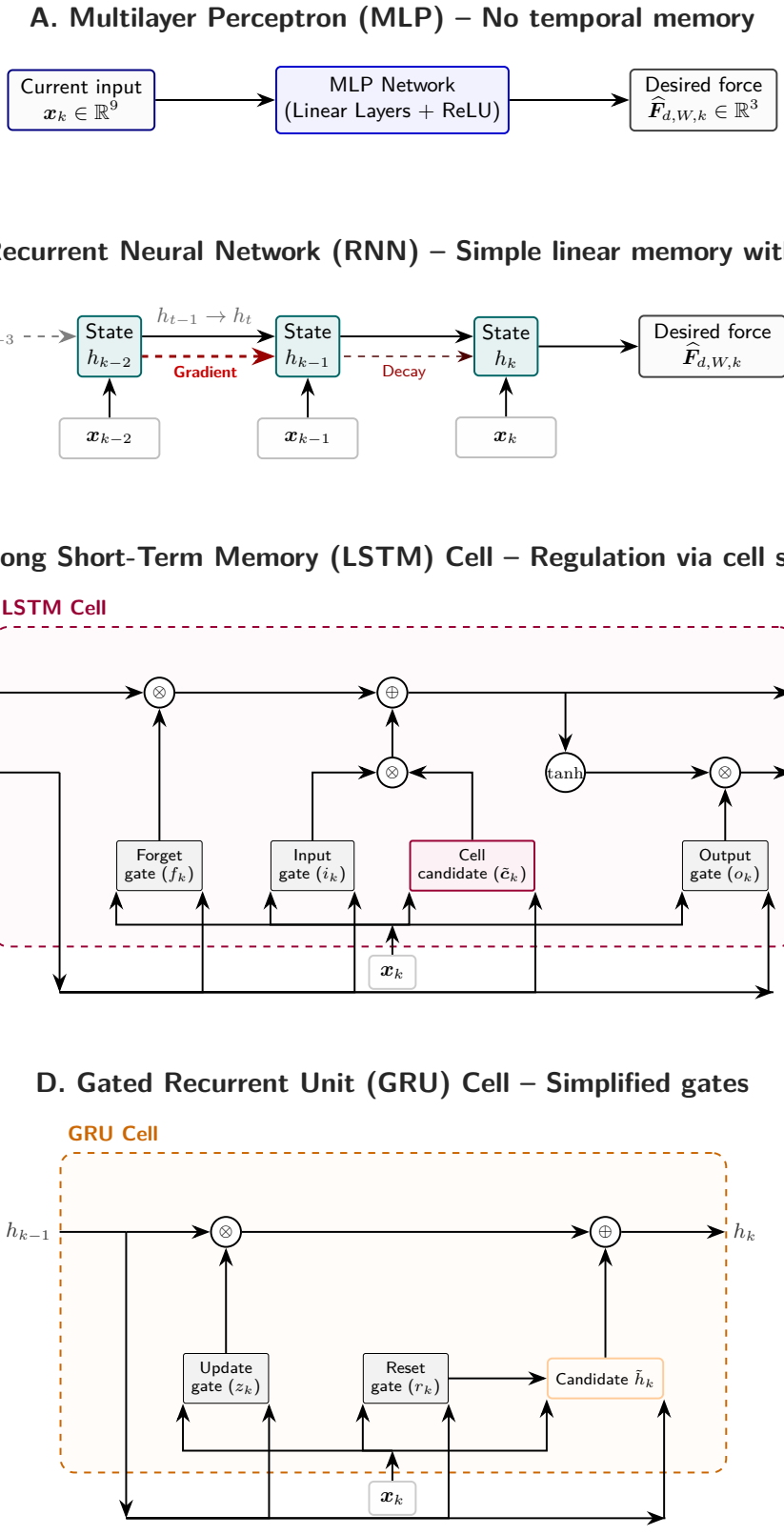


Figure 2.2. Conceptual comparison of neural architectures for control. In (A), the purely static nature of the MLP is shown. In (B), the unrolled flow of the basic RNN and its vulnerability to gradient vanishing are detailed. In (C) and (D), the selective gated structure of the LSTM cell and the GRU cell are contrasted. Source: own elaboration.

The LSTM separates two internal quantities. The hidden state $\boldsymbol{h}_k$ is the representation output from the cell, used to calculate the prediction or serve as input for the next step. The cell state $\boldsymbol{c}_k$ acts as a more direct internal memory, designed to carry information along the sequence. The three main gates regulate this memory: the forget gate decides what portion of $\boldsymbol{c}_{k-1}$ is kept, the input gate decides what new information is incorporated, and the output gate decides what portion of the updated memory is exposed as $\boldsymbol{h}_k$. A common formulation of the LSTM is

$$\boldsymbol{f}_k = \sigma(\boldsymbol{W}_f[\boldsymbol{h}_{k-1}, \boldsymbol{x}_k] + \boldsymbol{b}_f), \tag{2.1}$$
$$\boldsymbol{i}_k = \sigma(\boldsymbol{W}_i[\boldsymbol{h}_{k-1}, \boldsymbol{x}_k] + \boldsymbol{b}_i), \tag{2.2}$$
$$\tilde{\boldsymbol{c}}_k = \tanh(\boldsymbol{W}_c[\boldsymbol{h}_{k-1}, \boldsymbol{x}_k] + \boldsymbol{b}_c), \tag{2.3}$$
$$\boldsymbol{c}_k = \boldsymbol{f}_k \odot \boldsymbol{c}_{k-1} + \boldsymbol{i}_k \odot \tilde{\boldsymbol{c}}_k, \tag{2.4}$$
$$\boldsymbol{o}_k = \sigma(\boldsymbol{W}_o[\boldsymbol{h}_{k-1}, \boldsymbol{x}_k] + \boldsymbol{b}_o), \tag{2.5}$$
$$\boldsymbol{h}_k = \boldsymbol{o}_k \odot \tanh(\boldsymbol{c}_k). \tag{2.6}$$

Here, σ is the sigmoid activation, tanh is the hyperbolic tangent activation, $[\boldsymbol{h}_{k-1}, \boldsymbol{x}_k]$ denotes concatenation, and $\odot$ is the Hadamard (element-wise) product.

Gated recurrent unit (GRU). The GRU also uses gates, but with a more compact structure than the LSTM. Instead of explicitly separating a cell state and a hidden state, it combines update and reset mechanisms to decide how much prior information to retain and how to incorporate the current input [26]. This positions it as a recurrent network alternative with less complexity than the LSTM while maintaining sequence-processing capabilities. Thus, comparing GRU and LSTM under the same conditions will allow assessing whether a more complex memory provides an improvement for outer-loop control.

In a GRU, the update gate $\boldsymbol{z}_k$ regulates how much of the previous state is kept versus how much is replaced by a new proposal. The reset gate $\boldsymbol{r}_k$ controls how much the previous state influences the candidate calculation $\tilde{\boldsymbol{h}}_k$. A common formulation is

$$\boldsymbol{z}_k = \sigma(\boldsymbol{W}_z[\boldsymbol{h}_{k-1}, \boldsymbol{x}_k] + \boldsymbol{b}_z), \tag{2.7}$$
$$\boldsymbol{r}_k = \sigma(\boldsymbol{W}_r[\boldsymbol{h}_{k-1}, \boldsymbol{x}_k] + \boldsymbol{b}_r), \tag{2.8}$$
$$\tilde{\boldsymbol{h}}_k = \tanh(\boldsymbol{W}_h[\boldsymbol{r}_k \odot \boldsymbol{h}_{k-1}, \boldsymbol{x}_k] + \boldsymbol{b}_h), \tag{2.9}$$
$$\boldsymbol{h}_k = (1 - \boldsymbol{z}_k) \odot \boldsymbol{h}_{k-1} + \boldsymbol{z}_k \odot \tilde{\boldsymbol{h}}_k. \tag{2.10}$$

By not maintaining a separate cell state, the GRU typically requires fewer parameters than an equivalent LSTM. However, there may be cases where this lower complexity is

sufficient to capture the temporal dependency and even reduce sensitivity to sequences with low representation.

In inference, there are two practical ways to use a recurrent network. In a *stateful* execution, the hidden state computed in one step is preserved and supplied as the initial state of the next step. In a window-based execution, by contrast, for each prediction a sequence of fixed length is reconstructed, fully processed, and the output is provided. The first option avoids repeatedly recalculating past information but causes a dependence on the network's internal propagation state throughout the episode. The second is computationally more expensive but maintains a local impact restricted to the window. This distinction will be relevant later, as the implementation carried out uses fixed-length temporal windows instead of a fully stateful recurrent inference.

The training of these architectures retains elements of supervised learning. Normalization calculated solely using the training split prevents variables of different scales from dominating the loss function and reduces information leakage from validation or testing. The mean squared error (MSE) is a useful function in this case, penalizing large deviations between prediction and ground truth with greater intensity. Validation is also used to select a checkpoint without tuning weights on the test set, and early stopping is applied to halt training if no improvement is observed after a set number of epochs.

2.6. EVALUATION, GENERALIZATION AND METRICS

In dynamic systems of this type, evaluation must be conducted both in a supervised manner and on the behavior of the system in closed loop. It can happen that a network approximates the specialized controller's force very well on test set samples and yet generates a worse or even divergent trajectory when its predictions are fed back into the dynamics. This discrepancy stems from the fact that closed-loop control modifies the future states seen by the network itself, turning deviations in forces into different input data. For this reason, an evaluation of imitation learning must distinguish between supervised fidelity, tracking error, and the utilization of system limitations.

The separation between training, validation, and testing is necessary to estimate performance on data not used for parameter tuning or checkpoint selection. Furthermore, when studying transfer, one must also distinguish between in-distribution (ID) and out-of-distribution (OOD) conditions. Thus, the former verify interpolation within families and disturbances already represented in training, while the latter evaluate whether a reasonable behavior is preserved under recombinations or new geometries.

Tracking metrics must capture complementary aspects of the error. In this way, RMSE penalizes large errors more heavily, MAE describes the average error, and

maximum error detects localized extremes. These quantities must be interpreted alongside saturations, control effort, and simulation termination causes, as a low average error can be achieved with a continuous demand for unrealizable actions.

2.7. POSITIONING OF THIS WORK

This work positions itself at an intermediate point between classic control and purely learned control. On one hand, cascaded control allows the separation between translation and attitude, observability of its functioning, force, moment, and saturations, but faces a limitation due to the tuning process depending on the trajectory family, transient motions, and disturbances considered. On the other hand, neural networks are a flexible way to approximate relationships from data, but require a distinction between supervised fidelity and closed-loop behavior.

This work, therefore, employs a hybrid scheme where the classic inner loop, the mixer, and the actuation protections are maintained, and only the outer loop is replaced by an imitation-trained neural network. The network generates a desired force in the ENU world frame, thus preserving the separation between the stochastic component and the deterministic elements. In this context, a simulator with traceability is constructed, specialized PD controllers are tuned, and MLP, GRU, and LSTM architectures are trained to condense the behavior of classic controllers. The comparison supplements the supervised loss with tracking metrics, terminations, and closed-loop protection activation.

3. Methodology

3.1. PHYSICAL MODEL AND 6DOF SIMULATOR

Requirements, Conventions and Hypotheses

The simulator has been designed so that the classic controller and the neural controller face the same physical problem and the same actuation flow; that is, they share the same interface, meaning the simulator does not distinguish the nature of the controller. Thus, the observed differences will be due exclusively to the controller and not to external factors.

The state is written as $\boldsymbol{x} = (\boldsymbol{p}_{\mathcal{W}}, \boldsymbol{v}_{\mathcal{W}}, \boldsymbol{q}_{\mathcal{WB}}, \boldsymbol{\omega}_{\mathcal{B}})$: position and velocity in the world reference frame $\mathcal{W}$, orientation of the body relative to the world, and angular velocity in the body reference frame $\mathcal{B}$. The input applied to the simulator consists of the total rotor force and total moment, both expressed in the body reference frame. The recorded output includes the state, the observation provided to the controller, the trajectory reference, the commanded control input, and the applied actuation. This allows checking when and why a command could not be executed.

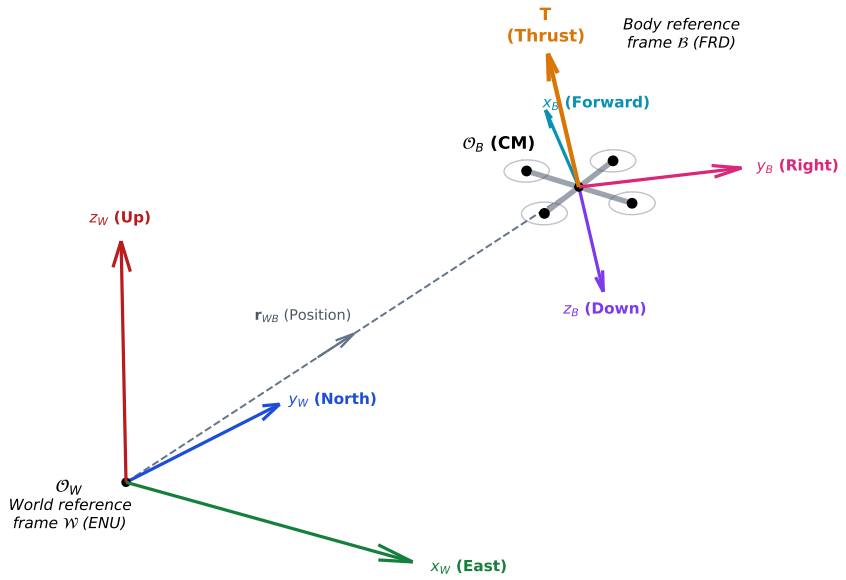


Figure 3.1. Reference frames used by the simulator. The world frame follows the ENU convention and the body frame follows the FRD convention; total thrust is applied in the opposite direction of the vertical body axis (z_B). Source: own elaboration.

The reference frame $\mathcal{W}$ follows the ENU convention, where x points East, y North, and z upward. The reference frame $\mathcal{B}$ is FRD, where x_B points forward, y_B to the right, and z_B downward. The rotors must generate thrust in the $-z_B$ direction (see Figure 3.1). The rotation matrix $\boldsymbol{R}_{WB}(\boldsymbol{q}_{\mathcal{WB}})$ transforms vectors expressed in FRD to vectors expressed

in ENU. The attitude is represented by the quaternion $\boldsymbol{q}_{\mathcal{WB}} = [q_0, q_1, q_2, q_3]^\mathsf{T}$. All quantities are expressed in the International System of Units (SI), and angles are only converted to degrees for visualization purposes.

A rigid body of constant mass and inertia is assumed, with a fixed center of mass and uniform gravity. Rotors are modeled with thrust proportional to the square of their angular velocity, subject to a first-order lag and delay. Advanced aerodynamics, rotor flapping, structural flexibility, and detailed physical sensors are not modeled.

Reference Vehicle and Physical Parameters

A hypothetical quadcopter of 1 kg mass is defined, with gravity $g = 9.81\,\mathrm{m\,s^{-2}}$ and principal moments of inertia $\boldsymbol{I}_B = \mathrm{diag}(0.05, 0.05, 0.10)\,\mathrm{kg\,m^2}$. The four rotors are positioned in the FRD body reference frame at

$$\boldsymbol{r}_{i,B} \in \{(0.17, 0.17, 0), (0.17, -0.17, 0), (-0.17, 0.17, 0), (-0.17, -0.17, 0)\}\,\mathrm{m},$$

corresponding to an X configuration relative to the body axes. Rotational directions are encoded by $s_i = \{-1, 1, 1, -1\}$, where the sign determines the yaw reaction torque. Figure Figure 3.2 shows a schematic of the vehicle's layout.

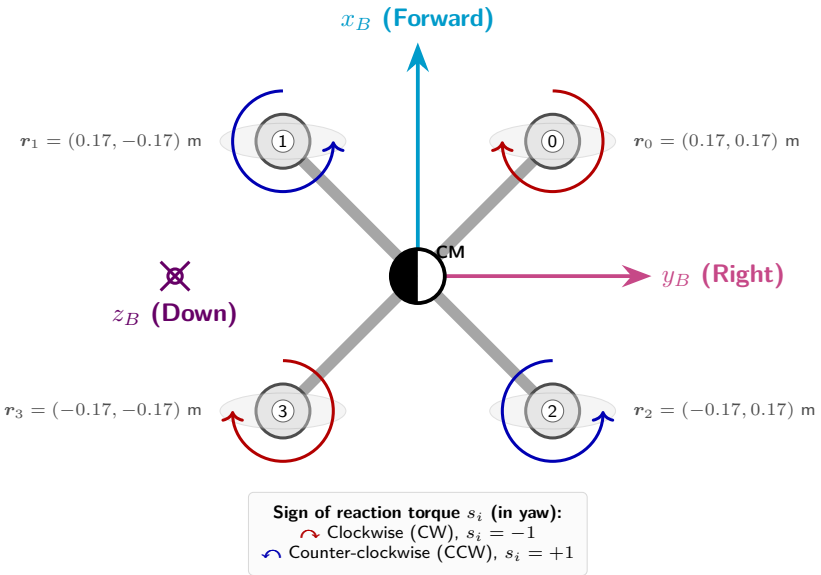


Figure 3.2. X-configuration of the motors and rotor rotational directions in the body reference frame. The physical direction of the rotors is associated with the sign of the reaction yaw moment. Source: own elaboration.

Each rotor uses $k_f = 1 \times 10^{-4}\,\mathrm{N\,s^2\,rad^{-2}}$ for thrust and $k_m = 1 \times 10^{-6}\,\mathrm{N\,m\,s^2\,rad^{-2}}$ for reaction torque. In the scenarios, $\omega_{\max} = 500\,\mathrm{rad\,s^{-1}}$ is set, so that the maximum thrust per rotor is

$$T_{\max} = k_f \omega_{\max}^2 = 25\,\mathrm{N}. \tag{3.1}$$

This limit is arbitrary and much higher than the hover thrust per rotor ($mg/4 \simeq 2.45\,\mathrm{N}$). The controller also imposes a total force limit of $2.5mg$ before conversion to attitude, so that a rotor saturation is interpreted as a limit in the allocation.

The methodology followed maintains the same vehicle in all comparison scenarios, although the simulator allows changing mass, inertia, or rotor parameters in the individual configuration of each scenario.

Rigid-Body Kinematics and Dynamics

Let $\boldsymbol{F}_B$ and $\boldsymbol{\tau}_B$ be the force and moment applied in the FRD body reference frame. The translational dynamics are

$$\dot{\boldsymbol{p}_{\mathcal{W}}} = \boldsymbol{v}_{\mathcal{W}}, \tag{3.2}$$

$$m\dot{\boldsymbol{v}_{\mathcal{W}}} = \boldsymbol{R}_{WB}\boldsymbol{F}_B + \boldsymbol{F}_{d,W} + [0\;0\;-mg]^{\mathsf{T}}, \tag{3.3}$$

where $\boldsymbol{F}_{d,W}$ is the linear drag defined in the epigraph on disturbances, observation, and safety. The rotational dynamics are expressed as

$$\dot{\boldsymbol{q}_{\mathcal{WB}}} = \tfrac{1}{2}\boldsymbol{q}_{\mathcal{WB}} \otimes [0\;\boldsymbol{\omega}_{\mathcal{B}}^{\mathsf{T}}]^{\mathsf{T}}, \tag{3.4}$$

$$\dot{\boldsymbol{\omega}_{\mathcal{B}}} = \boldsymbol{I}_B^{-1}\left(\boldsymbol{\tau}_B - \boldsymbol{\omega}_{\mathcal{B}} \times \boldsymbol{I}_B\boldsymbol{\omega}_{\mathcal{B}}\right). \tag{3.5}$$

During intermediate evaluations of the Runge–Kutta method and at the end of each step, the quaternion is normalized, thereby preventing accumulated numerical errors from causing its norm to deviate from unity.

The state is propagated using fourth-order Runge–Kutta (RK4) with a fixed step of 0.01 s. This choice is supported by lower computational cost and sufficient discretization for trajectories without abrupt maneuvers. The applied command remains constant during each integration step [7].

Actuators, Mixer and Actuation Limits

For rotor i, the applied speed determines thrust and reaction torque through

$$T_i = k_f \omega_i^2, \qquad Q_i = s_i k_m \omega_i^2, \tag{3.6}$$

where $s_i \in \{-1, 1\}$ represents the rotational direction. The rotor force in the FRD body frame is $\boldsymbol{F}_{i,B} = [0\ 0\ -T_i]^\mathsf{T}$, and its contribution to the moment includes the moment arm term $\boldsymbol{r}_{i,B} \times \boldsymbol{F}_{i,B}$ and the reaction torque $[0\ 0\ Q_i]^\mathsf{T}$. The quadratic model of thrust and reaction torque is a standard approximation in low-order quadcopter models [1, 2].

The mixer transforms a total thrust and moment command into individual rotor commands. To do so, it constructs an allocation matrix $\boldsymbol{M}$ whose columns collect the contribution of each thrust T_i to the total thrust and roll, pitch, and yaw moments:

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & 1 & 1 \\ -y_1 & -y_2 & -y_3 & -y_4 \\ x_1 & x_2 & x_3 & x_4 \\ s_1 k_m/k_f & s_2 k_m/k_f & s_3 k_m/k_f & s_4 k_m/k_f \end{bmatrix}}_{\boldsymbol{M}} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix}. \tag{3.7}$$

Therefore, a rotor located on the right side of the vehicle ($y_i > 0$) contributes a negative roll moment when its thrust increases, while a rotor located in front ($x_i > 0$) contributes a positive pitch moment, as shown in Figure 2.1.

The solution of the system is obtained by inverting the allocation matrix ($\boldsymbol{M}$) for the defined geometry. As illustrated in the schematic of Figure Figure 3.3, the mixer separates the contribution of the commanded collective thrust (T) from the part due to attitude moments ($\boldsymbol{\tau}_B$):

$$T_{\text{thrust\_part}} = \boldsymbol{M}^{-1}[T, 0, 0, 0]^\mathsf{T}, \tag{3.8}$$

$$T_{\text{moment\_part}} = \boldsymbol{M}^{-1}[0, \tau_x, \tau_y, \tau_z]^\mathsf{T}. \tag{3.9}$$

To prevent final thrusts from exceeding the individual physical limits of the actuators, set in the range $[T_{\min}, T_{\max}]$, the existence of a margin (slack) in the collective thrust is evaluated. The lower ($\delta_{\min}$) and upper ($\delta_{\max}$) bounds for the collective shift are calculated by determining the most restrictive value among the four rotors:

$$\delta_{\min} = \max_i (T_{\min} - T_{\text{thrust\_part},i} - T_{\text{moment\_part},i}), \tag{3.10}$$

$$\delta_{\max} = \min_i (T_{\max} - T_{\text{thrust\_part},i} - T_{\text{moment\_part},i}). \tag{3.11}$$

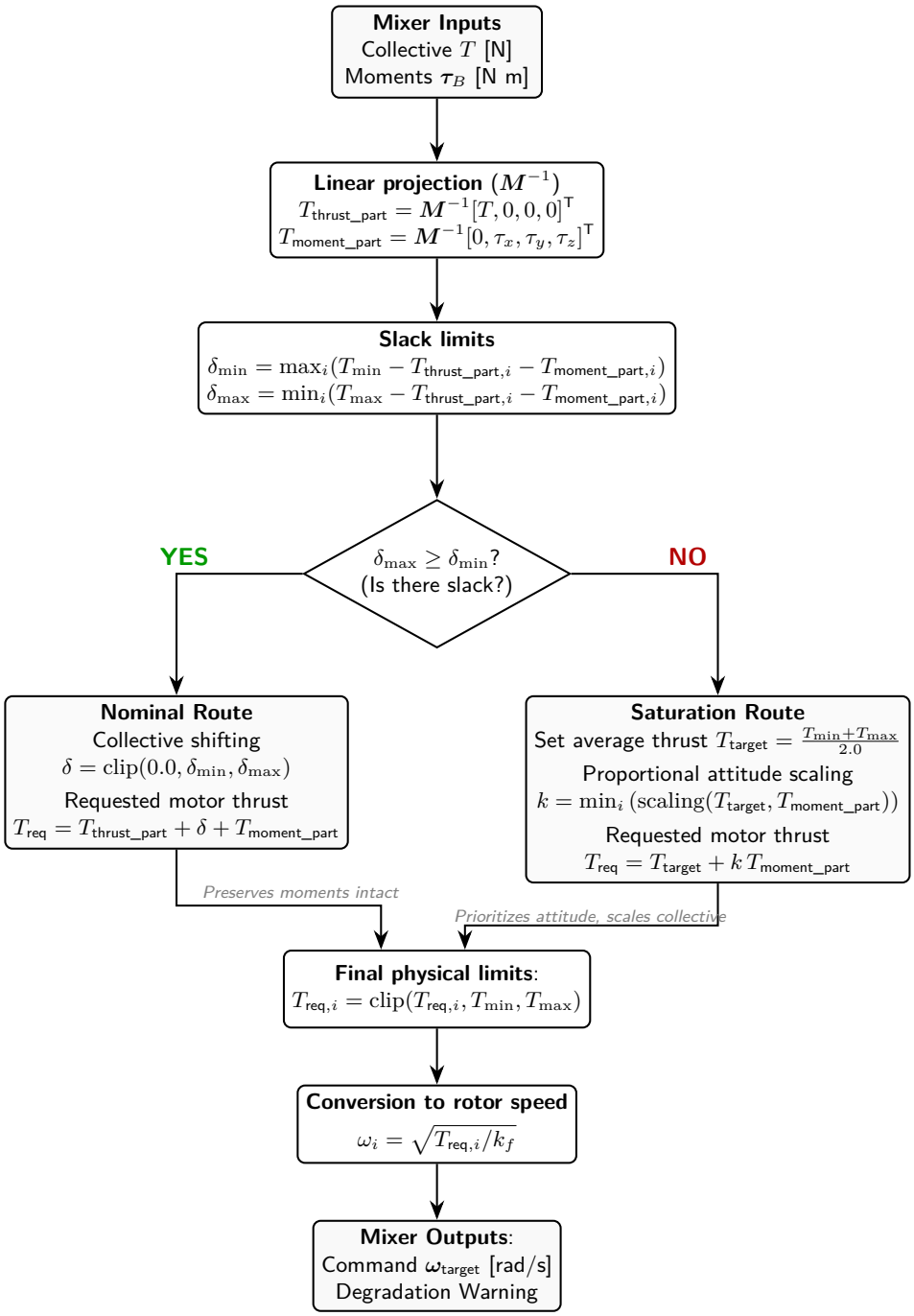


Figure 3.3. Logical schematic and decision-making of the rotor mixer under nominal and saturated conditions. Attitude control is prioritized by proportional scaling of moments (k) over deviation of the requested collective thrust. Source: own elaboration.

Based on these slack margins, the mixer algorithm branches into two possible paths:

- **Nominal Path** ($\delta_{\max} \geq \delta_{\min}$): There is a range to adjust the collective without perturbing the moments. The shift is found via $\delta = \text{clip}(0.0, \delta_{\min}, \delta_{\max})$, yielding

the requested thrust per rotor:

$$T_{\text{req},i} = T_{\text{thrust\_part},i} + \delta + T_{\text{moment\_part},i}. \tag{3.12}$$

- **Saturation Path** ($\delta_{\max} < \delta_{\min}$): No feasible solution exists, and the motor limits prevent satisfying both collective thrust and attitude simultaneously. In this condition, the mixer prioritizes stability over collective thrust. A target average thrust is set to $T_{\text{target}} = (T_{\min} + T_{\max})/2.0$, and a proportional scaling factor $k \in [0, 1]$ is applied to the moment contribution to ensure no rotor saturates:

$$k = \min_i \left(\text{recorte}(T_{\text{target}}, T_{\text{moment\_part},i})\right), \tag{3.13}$$

where the scaling function evaluates the available margin to the thrust limits based on rotor direction and actuation:

$$\text{recorte}(T_{\text{target}}, T_{\text{moment\_part},i}) = \begin{cases} \dfrac{T_{\max} - T_{\text{target}}}{T_{\text{moment\_part},i}} & \text{si } T_{\text{moment\_part},i} > 0, \\ \dfrac{T_{\min} - T_{\text{target}}}{T_{\text{moment\_part},i}} & \text{si } T_{\text{moment\_part},i} < 0. \end{cases} \tag{3.14}$$

The resulting thrust for each motor is:

$$T_{\text{req},i} = T_{\text{target}} + k\, T_{\text{moment\_part},i}. \tag{3.15}$$

In either of the two cases where the original command must be modified to meet the physical limits of the actuators, the simulator records a command degradation. Finally, thrusts are strictly clipped using $T_{\text{req},i} = \text{clip}(T_{\text{req},i}, T_{\min}, T_{\max})$ and transformed into commanded rotor angular velocities $\omega_i = \sqrt{T_{\text{req},i}/k_f}$.

The rotor dynamics combine a discretized pure delay and a first-order lag. The pure delay shifts the command by an integer number of physics loop steps, while the lag approximates that the rotor speed does not reach the command instantaneously but rather approaches it with a time constant τ. In the runs, depending on the trajectory profile, values of $\tau = 0.03\,\text{s}$ with a delay of $0.01\,\text{s}$ and $\tau = 0.05\,\text{s}$ with a delay of $0.02\,\text{s}$ are used.

Disturbances, Observation and Safety

Linear drag is calculated in the FRD body reference frame from the relative velocity with respect to the wind:

$$\boldsymbol{v}_{r,B} = \boldsymbol{R}_{WB}^{\mathsf{T}}(\boldsymbol{v}_{\mathcal{W}} - \boldsymbol{v}_{w,W}), \tag{3.16}$$

and expressed back in the ENU world frame as

$$\boldsymbol{F}_{d,W} = \boldsymbol{R}_{WB}\left(-\boldsymbol{D}_B \boldsymbol{v}_{r,B}\right). \tag{3.17}$$

Two levels of trajectory dissipation have been implemented in the simulator. A constant wind defined by a fixed magnitude and direction throughout the execution can also be applied. The observation received by the controller can include a small Gaussian noise component, representing in a simplified manner that the observed signal is not perfect, while the core dynamics, integration, and evaluation continue to use the true state. The scenario seed determines the pseudo-random initialization, allowing the same YAML file to be reproduced identically.

The simulation terminates if the vehicle crashes into the ground, if the roll/pitch inclination exceeds the configured limit, if a non-finite value appears, or if actuator saturation persists longer than the allowed duration. In the runs, a threshold of 2 s for persistent saturation and an inclination limit of 1.256 rad are used. The simulator execution also maintains internal position and velocity bounds to halt numerically divergent trajectories.

Trajectories and Scenarios

The trajectory families were chosen to introduce sufficient variability without turning this module into an excessively complex problem. Figure Figure 3.4 shows a representative case of each family. `hold` requires maintaining a constant position and yaw. `circle` imposes a horizontal trajectory with centripetal acceleration and continuous advance, which introduces a non-periodic effect when combined with wind. `lissajous` introduces a coupled periodic motion across axes:

$$x(t) = c_x + A_x \sin(\omega_x t), \tag{3.18}$$
$$y(t) = c_y + A_y \sin(\omega_y t), \tag{3.19}$$
$$z(t) = c_z + A_z \sin(\omega_z t). \tag{3.20}$$

By using different frequencies per axis, the reference covers a spatial region richer than a circle and forces the controller to follow simultaneous changes in position, velocity, and acceleration.

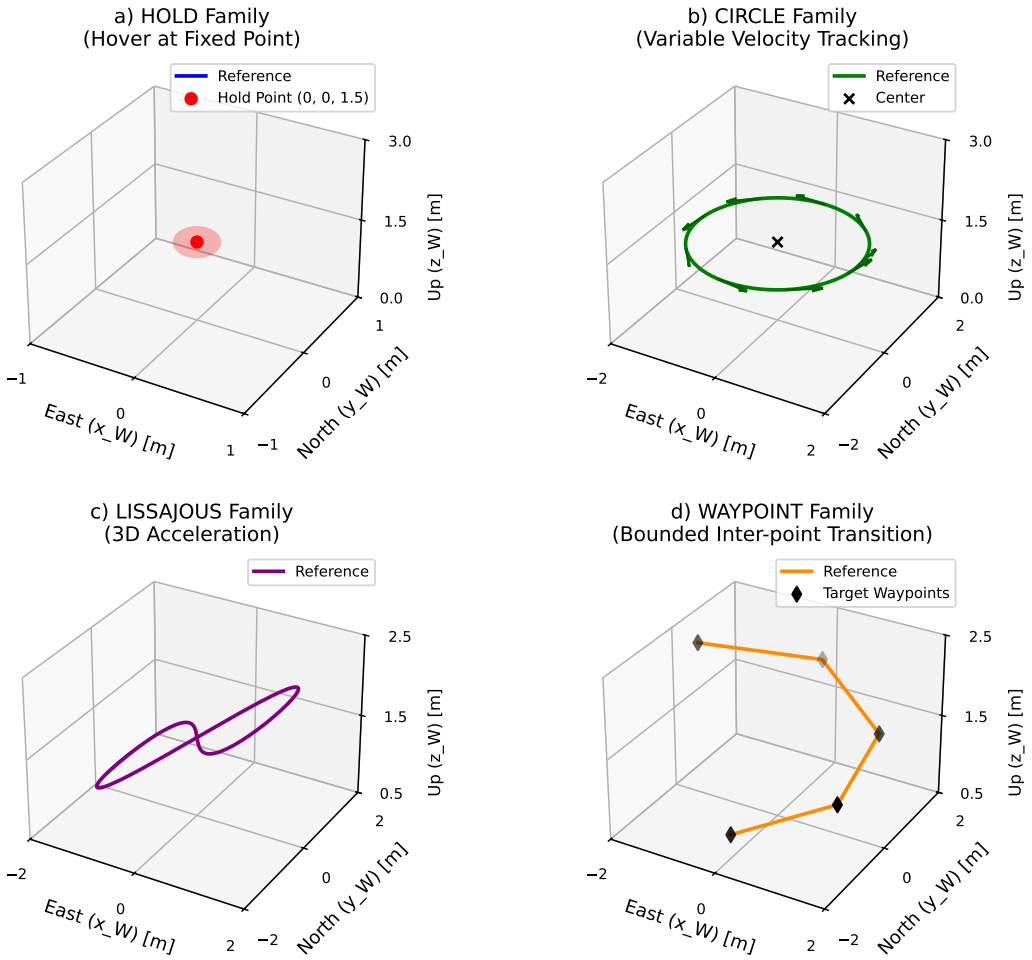


Figure 3.4. Analytical trajectory families of the simulator. Each geometry introduces different stimuli (stationary, variable speed, coupled acceleration in three axes, or discrete segments). Source: own elaboration.

The `waypoint` family defines a point-to-point trajectory. The vehicle must reach a series of positions, slow down, settle within a position and velocity tolerance, and remain there for a configurable settling time (defaulting to 0.40 s) before advancing to the next point. For each segment, a scalar progress coordinate $s(t)$ is generated along the straight line connecting two waypoints. If the distance allows reaching the maximum velocity, the profile is trapezoidal, with constant acceleration, a constant velocity segment, and deceleration. If the segment is short, the profile is triangular and the maximum velocity is not reached. In both cases, position and velocity are continuous, avoiding step changes in the reference. Figure Figure 3.5 shows the kinematic profiles of both cases.

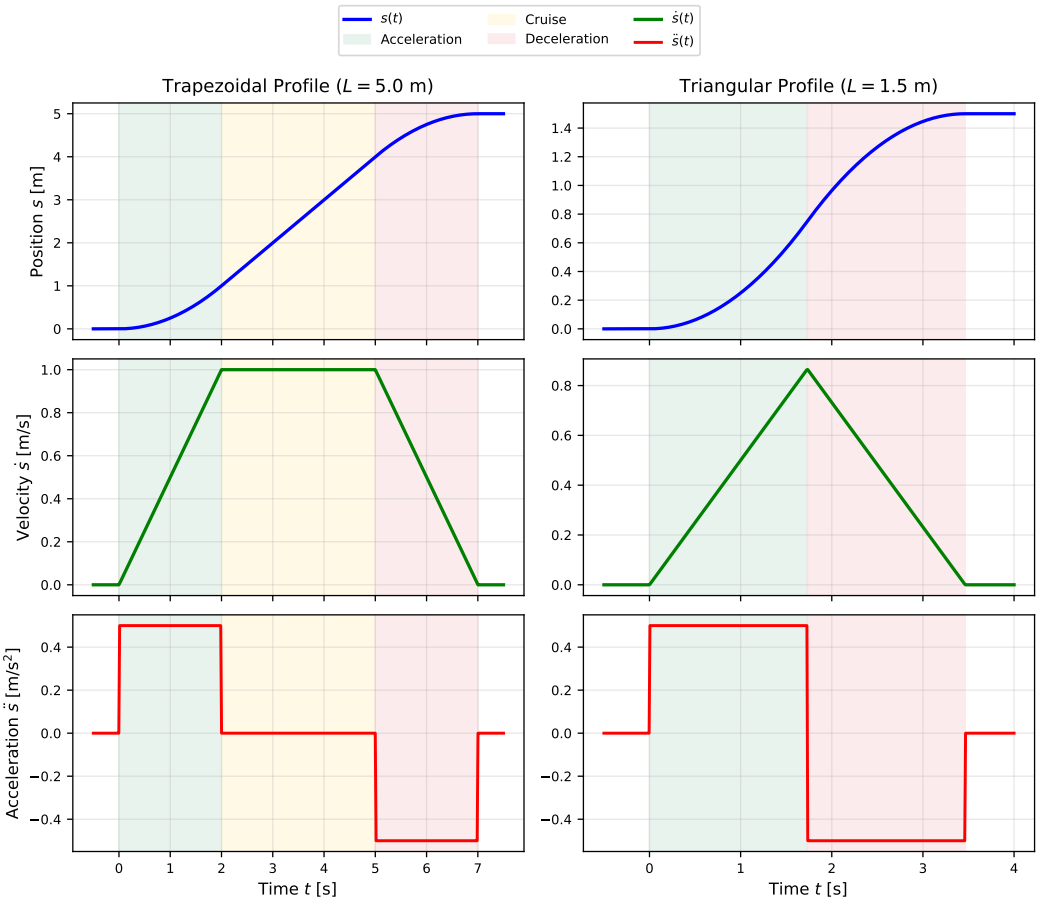


Figure 3.5. Kinematic profiles generated during transitions between waypoints. On the left, the trapezoidal profile (for long segments) is illustrated, and on the right, the triangular profile (for short segments, where the commanded speed is not reached). All represented quantities are scalars along the progress axis of the segment. Source: own elaboration.

```yaml
name: "Waypoint Clean"
seed: 42
vehicle: {mass_kg: 1.0, rotors: [...]}
initial_state: {position_W_m: [0, 0, 0], velocity_W_m_s: [0, 0, 0]}
trajectory:
  type: "waypoint"
  waypoints: [[0, 0, 0], [0, 0, 2], [2, 0, 2]]
  max_speed_m_s: 0.6
  max_acceleration_m_s2: 0.5
controller: {type: "classic"}
perturbations: {constant_wind_W_m_s: [0, 0, 0], pos_std_m: 0.0}
timing: {physics_dt_s: 0.01, control_dt_s: 0.02, telemetry_dt_s: 0.1}
termination: {max_duration_s: 40.0, z_min_m: -0.1}
```

Listing 1: Simplified structure of a YAML scenario.

In each YAML file, as shown in Listing 1, the vehicle, initial state, trajectory, controller, disturbances, seed, execution periods, and termination conditions are defined. By convention, the initial state is set equal to the trajectory reference at $t = 0$, with a level attitude and initial thrust corresponding to hover. This evaluates trajectory tracking rather than recovery from an initial fall caused by starting with the rotors turned off.

The specification of the quantity and characteristics of the trajectories for each family used for training, validation, and testing (train, validation, test) is developed in the section on classic dataset design. Additionally, the simulator allows sequential compositions of trajectories from the described families, as well as lemniscates. These variants are developed later within the context of OOD evaluation, distinguishing between variations and compositions of known families and completely new trajectories.

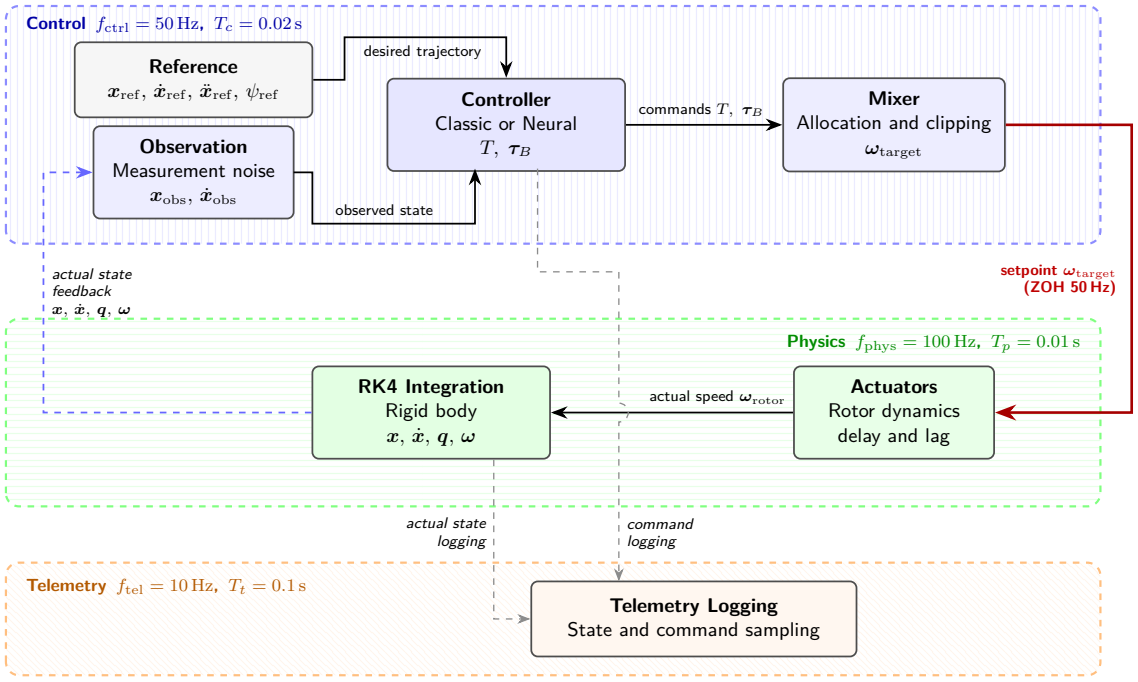


Figure 3.6. Multirate time execution sequence and data dependencies. The three frequency bands associated with physics integration (100 Hz), the digital control cycle (50 Hz), and periodic variable logging (10 Hz) are distinguished. Source: own elaboration.

Multirate Simulation Flow

The simulator operates with a multirate flow; that is, with multiple update periods within a single execution, as can be observed in Figure 3.6. Physics are integrated every 0.01 s, the controller calculates a new command every 0.02 s, and telemetry is saved every 0.1 s. Notably, like all other parameters, these frequencies are configurable in the episode's YAML file. At each step, the reference is retrieved, the observation is formed,

and, when applicable, a new command is calculated. The mixer and actuators produce the applied force and moment based on the last available command, RK4 advances the state, and telemetry records the variables.

Between two control cycles, a zero-order hold (ZOH) is applied, meaning the last command requested by the controller remains constant until the next update. This attempts to approximate a real system, where physics are continuous and sensors, controllers, and actuators operate at finite frequencies.

Telemetry

Each telemetry record contains the simulation time, the state, the observation received by the controller, the reference, the control command, the command for each rotor, and the applied actuation. For the rotors, both target and applied thrust and speed are recorded, as well as reaction torque and revolutions per minute. The reason for simulation termination is also included in the final record.

This structure allows subsequent study of the tracking error, the difference between state and observation, mixer state, actuator saturation, and the effect of rotor lag and delay. Additionally, a summary of statistical values and the simulation configuration data are provided.

Choice of Implementation Environment

The implementation has been carried out in Python, as it offers a suitable balance between capabilities, ease of use, and availability. It is an open-source programming language with no licensing costs, a large user base, and numerous scientific libraries. PyTorch is used, a highly capable and flexible library for training and working with neural networks. Likewise, Python allows seamless interaction with external files used in development, such as CSV, YAML, JSON, Markdown, and LaTeX.

Environment management is performed with `uv`, so package dependencies and reproduction commands are defined clearly and simply for other users to use. The work as a whole has been developed openly in a GitHub repository, facilitating code inspection, change traceability, and reuse of the material. [1] [11, 12, 16].

However, Python is not claimed to be the best choice for all phases of the work. An implementation in C or C++ would have lower performance overhead and would be ideal for an onboard platform or real-time execution, though it would increase development complexity, especially regarding neural networks. Since the central goal is to compare control strategies in an asynchronous simulation environment, Python is sufficient and appropriate for this development.

[1] https://github.com/ChristianGonper/modelado-control-dron-v2

3.2. CLASSIC CASCADED CONTROL

Cascaded PD Architecture

The classic controller serves three functions. First, it provides the benchmark against which the neural implementations are compared. Second, it generates the data for behavior cloning itself. Third, it provides the inner attitude loop that accompanies the neural-type outer loop.

The structure is separated into two loops. An outer loop for translation, which receives the position and velocity observation and the trajectory reference in the ENU frame, and calculates a desired force $\boldsymbol{F}_{d,W}$. The inner loop receives this force and the reference yaw, constructs an attitude compatible with the thrust direction, and calculates the moments in the FRD body frame. Finally, the mixer previously described in the section on actuators, mixer, and actuation limits transforms the collective thrust and moments into rotor commands. This flow can be seen in Figure 3.7.

Through this separation, it is clear which part will be replaced by learning and why they are said to use the same interface. Both the classic outer loop and the neural network receive position and velocity errors and output the desired force. It is worth mentioning that the network will also receive the acceleration required by the reference as input, while gravity compensation will be added in the same way as with the classic outer loop. From the force output onward, the rest of the flow remains identical.

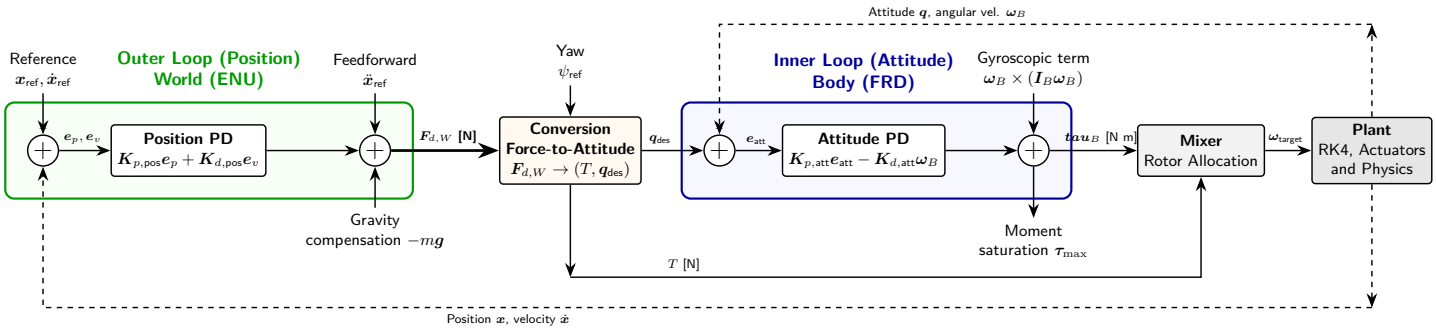


Figure 3.7. Architecture of the implemented classic cascaded controller. The outer loop (position in ENU world frame), the geometric conversion of force to attitude, and the inner loop (attitude stabilization in FRD body frame) are highlighted. The neural substitution boundary is located at the desired force signal $\boldsymbol{F}_{d,W}$. Source: own elaboration.

Equations, Gains and Limits

The trajectory reference provides position, velocity, acceleration, and yaw at each instant. In continuous families, acceleration comes from the analytical derivative of the reference, whereas in the `waypoint` family, it is derived from the profile described and illustrated in Figure 3.5.

Letting $\boldsymbol{e}_p = \boldsymbol{p}_r - \boldsymbol{p}$ and $\boldsymbol{e}_v = \boldsymbol{v}_r - \boldsymbol{v}$ represent the position and velocity errors, respectively, where $\boldsymbol{p}, \boldsymbol{v} \in \mathbb{R}^3$ correspond to the observed position and velocity of the vehicle in the ENU world frame, and $\boldsymbol{p}_r, \boldsymbol{v}_r \in \mathbb{R}^3$ are the reference position and velocity dictated by the trajectory, the outer loop calculates the commanded acceleration $\boldsymbol{a}_{cmd}$ and the desired force in ENU $\boldsymbol{F}_{d,W}$ via:

$$\boldsymbol{a}_{cmd} = \boldsymbol{K}_{p,p} \odot \boldsymbol{e}_p + \boldsymbol{K}_{d,p} \odot \boldsymbol{e}_v + \boldsymbol{a}_r, \tag{3.21}$$

$$\boldsymbol{F}_{d,W} = m \left(\boldsymbol{a}_{cmd} - [0\ 0\ -g]^\mathsf{T} \right). \tag{3.22}$$

Here, $\boldsymbol{a}_r \in \mathbb{R}^3$ represents the reference acceleration, m is the quadcopter mass, g is the acceleration of gravity, and the symbol $\odot$ indicates the Hadamard (element-wise) product. The reference acceleration acts as a trajectory feedforward term, and the gravitational term converts the desired kinematic acceleration into physical force. In a hover, for example, $\boldsymbol{a}_r = \boldsymbol{0}$ and zero errors lead to $\boldsymbol{F}_{d,W} = [0\ 0\ mg]^\mathsf{T}$, i.e., the upward force required to balance the weight.

The norm of $\boldsymbol{F}_{d,W}$ defines the requested collective thrust, limited in the controller to $[0, 2.5mg]$. Since an FRD frame is used, the thrust acts in the $-z_B$ direction, so for the vehicle to generate a force $\boldsymbol{F}_{d,W}$, the desired z_B axis is taken opposite to that force:

$$\boldsymbol{z}_{B,d} = -\frac{\boldsymbol{F}_{d,W}}{\|\boldsymbol{F}_{d,W}\|}.$$

In case the norm is virtually zero, the attitude is set to level.

The reference yaw completes the desired orientation. First, an auxiliary horizontal axis is formed: $\boldsymbol{x}_C = [-\sin\psi_r\ \cos\psi_r\ 0]^\mathsf{T}$, obtaining $\boldsymbol{y}_{B,d}$ as the vector cross product between $\boldsymbol{z}_{B,d}$ and $\boldsymbol{x}_C$, normalizing it when possible. Next, $\boldsymbol{x}_{B,d} = \boldsymbol{y}_{B,d} \times \boldsymbol{z}_{B,d}$ is calculated, and the matrix $\boldsymbol{R}_{WB,d}$ is constructed with these three axes as columns. This matrix is then transformed into a quaternion to compare the desired attitude with the observed attitude.

The cascaded structure and the force-to-attitude conversion are consistent with standard quadcopter control formulations [1, 2]. The attitude error is computed as $\boldsymbol{q}_e = \boldsymbol{q}_{WB}^{-1} \otimes \boldsymbol{q}_{WB,d}$, where $\boldsymbol{q}_{WB}$ is the quaternion describing the observed attitude of the body $\mathcal{B}$ relative to the world $\mathcal{W}$, $\boldsymbol{q}_{WB,d}$ is the desired attitude calculated in the previous step, and the operator $\otimes$ represents quaternion multiplication. If the scalar part of the error quaternion is negative, its sign is flipped to choose the equivalent shortest-rotation representation. For small errors, the angular deviation is approximated by the vector $\boldsymbol{e}_q = 2\boldsymbol{q}_{e,v}$, where $\boldsymbol{q}_{e,v} \in \mathbb{R}^3$ is the vector part of the quaternion $\boldsymbol{q}_e$. Assuming a zero reference angular velocity, the commanded moment in the FRD frame $\boldsymbol{\tau}_B \in \mathbb{R}^3$ is

defined as:

$$\boldsymbol{\tau}_B = \boldsymbol{K}_{p,a} \odot \boldsymbol{e}_q + \boldsymbol{K}_{d,a} \odot (-\boldsymbol{\omega}_{\mathcal{B}}) + \boldsymbol{\omega}_{\mathcal{B}} \times \boldsymbol{I}_B \boldsymbol{\omega}_{\mathcal{B}}. \tag{3.23}$$

Here, $\boldsymbol{K}_{p,a}$ and $\boldsymbol{K}_{d,a}$ are the proportional and derivative gain vectors of the attitude loop, $\boldsymbol{\omega}_{\mathcal{B}} \in \mathbb{R}^3$ is the vehicle angular velocity in FRD, and $\boldsymbol{I}_B$ is its inertia matrix. The final term compensates for the gyroscopic coupling appearing in the rotational dynamics described in the section on rigid-body kinematics and dynamics. The result is component-wise saturated using the limits declared in the controller.

All controllers start with default gains, which are subsequently tuned and fixed in a specialized PD per family in files named `pid_<family>_v1.yaml`. As can be observed, the controller does not include an integral term. This is due to the risk of integrator windup during saturated phases or continuous reference changes, which would require additional mechanisms such as anti-windup.

Choice of Tuning Method

The goal of tuning is not to find the analytical optimum for an isolated trajectory, but to obtain a controller suitable for an entire family of trajectories, subject to variations in parameters, disturbances, and actuator dynamics. For this reason, tuning is evaluated directly in simulation over a set of training trajectories, using metrics for tracking, attitude, control effort, saturation, degradation, and termination.

The manual tuning alternative would produce a valid controller if all iterations were recorded, but it would provide less traceability than a reproducible search. Classic rules such as Ziegler–Nichols are designed for systems and response criteria that do not directly represent a quadcopter with coupled loops, saturations, and variable spatial trajectories. A design based on a linearized model would also be possible, but it would force setting operating points and local criteria different from the evaluation intended to be run.

Progressive Search Algorithm

The search begins with a diagnostic run per family on the training set defined in classic dataset design. A diagram of this algorithm is illustrated in Figure Figure 3.8. For each family, two representative geometries are selected—one slow and one demanding—and combined with three profiles: nominal, East wind, and combined disturbance. If the initial controller passes the filters in all cases and its average RMSE remains below the threshold selected for the family, it is preserved without retuning. The thresholds, established heuristically, are 0.25 m, 0.35 m, 0.45 m, and 0.40 m for `hold`, `circle`, `lissajous`, and `waypoint`, respectively.

When the diagnosis requires tuning, the four gain blocks—$\boldsymbol{K}_{p,p}$, $\boldsymbol{K}_{d,p}$, $\boldsymbol{K}_{p,a}$, and $\boldsymbol{K}_{d,a}$—are modified jointly. Each candidate is defined by four multipliers scaling these

full blocks. The first round evaluates 32 candidates with log-uniform multipliers between 0.5 and 2, in addition to the initial candidate. Next, the best candidates from that round are selected, and 16 local perturbations are generated around them, resulting in a total of 49 evaluations per family to tune its controller. This second phase is designed as a local simplex-like refinement, inspired by the Nelder–Mead logic [27] to explore the neighborhood of the best candidates without aiming for full convergence.

Before comparing the scores of the controllers, filters are applied. Candidates with missing or non-finite metrics, invalid termination, more than 2% saturation, more than 2% command degradation, or a maximum error exceeding the limit are discarded.

The remaining candidates receive the score

$$J = e_{\text{rms}} + 0.5e_{\text{max}} + 0.2e_{\text{att}} + 0.1u + 2s + 2d, \tag{3.24}$$

where e_{rms} and e_{max} are position metrics, e_{att} is the RMS of attitude angles, s and d are the fractions of samples with saturation and degradation, and

$$u = \frac{\overline{T}}{mg} + \frac{\overline{\|\boldsymbol{\tau}_B\|}}{0.1}.$$

The first term of u normalizaes the average thrust by the vehicle's weight, and the second uses 0.1 N m as the scale for moments. This metric aims to favor controllers that require less control effort.

The final selection is based on the lowest score in the set. Within 5% of the best score, the controller with the lowest average effort is chosen, and if a tie remains, the one closest to the initial controller in terms of multipliers is preferred. In this way, moderate changes are favored when performance is equal.

The weights in Equation 3.24, the thresholds, the multiplier range, the number of candidates, and the 5% margin are heuristic execution choices. They are not constants derived from control theory, nor do they prove global optimization. They should be interpreted as a local optimum under a reasonable computational budget and criteria declared prior to evaluating test or OOD scenarios.

Fixed-Parameter Controllers and Cross-Transfer

Tuning only queries scenarios from the training set. Once a family's candidate is chosen, its gains, source, diagnostic trajectories set, and metrics are saved to a YAML file. All subsequent runs—validation, test, and OOD scenarios—use these files as a fixed-parameter controller set, without modification.

A specialized PD refers to the controller whose parameters were tuned for the same family as the trajectory being evaluated. Cross-transfer consists of running a scenario

with the PDs of the other families without retuning them, keeping the rest of the configuration fixed.

The transfer matrix allows quantifying the importance of having an optimized PD per family. It also defines a baseline for the neural controller, which must be compared against the transferred PDs when evaluating the ability to use the network under different conditions.

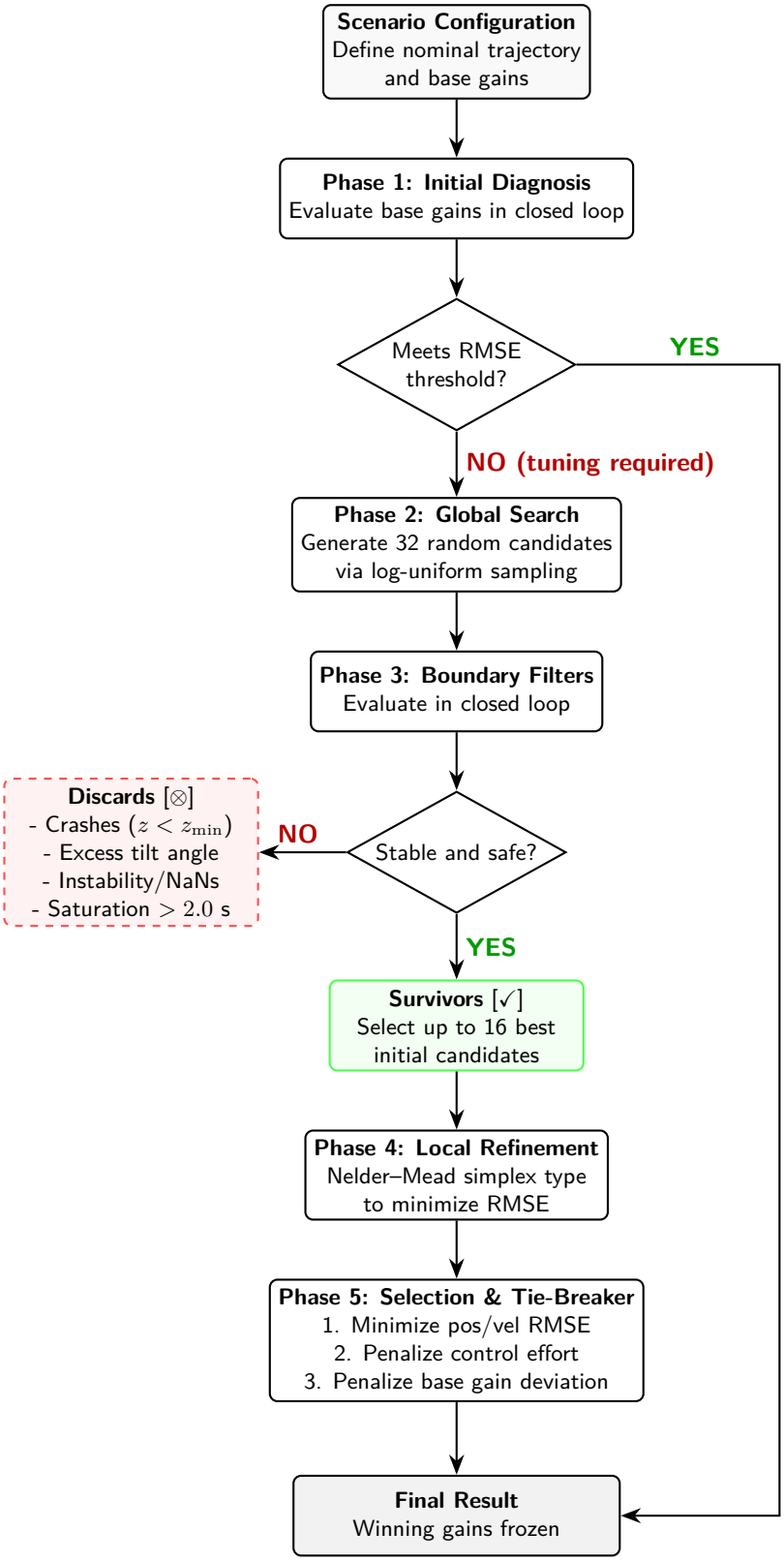


Figure 3.8. Logical flow of optimization and progressive tuning of classic gains. Global random candidates are filtered by stability and safety conditions before local simplex/Nelder–Mead-like refinement begins. Source: own elaboration.

3.3. NEURAL CONTROL BY IMITATION

Hybrid Formulation of Desired Force Prediction

The adopted neural implementation learns the translational decision defined by the outer loop in the classic controller. In each control cycle, it receives variables constructed from the observation and the reference, and predicts a desired force $\widehat{\boldsymbol{F}}_{d,W} \in \mathbb{R}^3$ in the ENU world reference frame. Subsequently, this output follows the same limits and classic components as in the previous implementation. Figure 3.9 represents the operational flow.

The shared interface allows a direct comparison between the learned policy and the classic decision. On the other hand, an end-to-end architecture where the neural network acts directly on the rotors would have a broader scope and mix the learning of tracking capability with stabilization.

The hybrid architecture forces a separation of the classic and neural contributions in the results. For this reason, the evaluation quantifies not only the tracking error but also attitude saturations and limits. In this way, an approximation of the actions observed during training is obtained, as well as the limitations arising from the dependence on the training distribution itself [19].

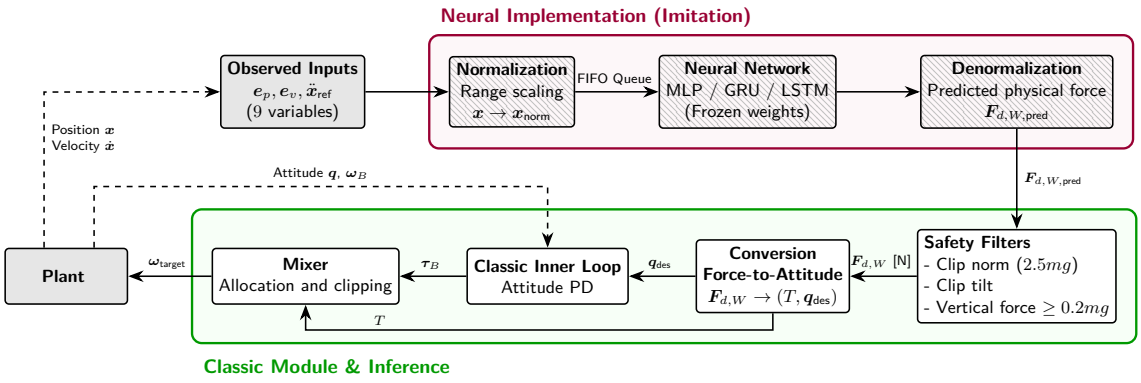


Figure 3.9. Structure of the hybrid neural controller. The neural network replaces only the translational decision of the outer loop to generate $\boldsymbol{F}_{d,W}$, delegating safety filtering, force-to-attitude conversion, and stabilization to the classic inner loop. Source: own elaboration.

Construction and Selection of Imitation Data

The neural dataset is constructed in a phase subsequent to tuning the classic PDs. First, the classic workflow fixes the scenarios, trajectories, disturbances, seeds, and training–validation–testing splits, as well as the reference controllers per family. Then, each scenario is taken, and a set of outer-loop variants is constructed, modifying exclusively the gains $\boldsymbol{K}_{p,p}$ and $\boldsymbol{K}_{d,p}$ while preserving all other parameters.

The implemented gain variants scale the gains using five profiles: conservative, base, aggressive, and two more damped profiles. In relative terms, these profiles apply the factors $(0.7, 0.9)$, $(1.0, 1.0)$, $(1.3, 1.2)$, $(1.6, 2.0)$, and $(1.4, 1.8)$ respectively to $(\boldsymbol{K}_{p,p}, \boldsymbol{K}_{d,p})$. Each variant is executed in the same scenario and evaluated with the simulation metrics, considering only those that pass the filters.

Among the candidates for each scenario, the lowest position RMSE is identified first. Candidates situated up to 5% above that value are then admitted, and within this subset, the one with the lowest control effort is chosen. If a tie remains, the most conservative profile is selected. This second round of tuning allows generating imitation targets without modifying the inner loop of the classic reference, whose parameters were previously fixed.

This procedure should be interpreted as behavior cloning. The network observes the actions of the external PDs and learns to approximate their forces; it does not directly optimize the trajectory error nor does it possess a tracking reward function. Consequently, it can also inherit biases or errors from the PDs in specific regions of the trajectory. In turn, if the training set contains large errors or sufficiently represented transient states, the network can learn responses within that more aggressive coverage away from the reference. Distinguishing the nature of what the network learns to execute justifies the double evaluation of supervised fidelity and closed-loop behavior.

Tensors, Input Variables, Targets, and Sequences

The input data represents each step using nine variables:

$$\boldsymbol{x}_k = [\boldsymbol{e}_{p,k}^{\mathsf{T}} \; \boldsymbol{e}_{v,k}^{\mathsf{T}} \; \boldsymbol{a}_{r,k}^{\mathsf{T}}]^{\mathsf{T}}. \tag{3.25}$$

The position errors $\boldsymbol{e}_{p,k}$ and velocity errors $\boldsymbol{e}_{v,k}$ are calculated with the observation supplied to the controller, and the third part of the vector is the reference acceleration in ENU. The supervised target is the three-component desired force that the external PD would calculate with those same variables, equivalent to Equation 3.22. Therefore, there is a direct correspondence, with the nuance of how the reference acceleration is treated, between the PD and the neural network.

Attitude variables, actuation variables, yaw, or the complete observed state could be added. However, the objective of this comparison is to imitate the outer loop, not to provide the network with information that the loop does not need to compute the force. This choice reduces dimensionality and maintains the interpretation as a strict replacement of the external PD.

The MLP processes tensors of shape $[B, 9]$, where B is the training batch size. The GRU and LSTM receive tensors of shape $[B, L, 9]$, with $L = 20$, and produce the force corresponding to the last step of the window. Since the control period is 0.02 s, the

recurrent window covers 0.40 s of samples. During training, windows are constructed within each simulation using a sliding window and are kept bounded within the training set, without crossing into other simulations. Each time a new sample is added, the oldest in the queue is removed, maintaining the size at 20. In inference, at the start of a simulation, since no prior samples exist yet, the queue is filled with the first input until the length is satisfied.

The recurrent inference used in this work is not *stateful* in the strict sense. The GRU and LSTM do not preserve the hidden state calculated in the previous call to use it directly as the initial state of the next cycle. In each cycle, a window with the last twenty normalized inputs is reconstructed and processed fully, obtaining the force applied at the last step of that window. Thus, two executions with the same input queue produce the same output, without depending on an accumulated internal state outside the window. Figure 3.10 illustrates this operation.

The temporal window allows studying whether recent history provides useful information in the presence of actuator delays and dynamics. The value $L = 20$ establishes a short and common window for both architectures. As a check, a sensitivity study was performed with $L = 10$ and $L = 40$ for both networks, and the observed differences were not sufficient to justify a change in configuration.

Architectures and Parameters of MLP, GRU, and LSTM

The MLP employs two hidden layers of 64 units with ReLU activation and a final linear layer with three outputs. With nine inputs, the number of parameters is obtained by summing the weights and biases of each layer: $(9{\cdot}64{+}64){+}(64{\cdot}64{+}64){+}(64{\cdot}3{+}3) = 4995$. GRU and LSTM use a single recurrent layer of 64 units and a final linear projection to three components. Under the PyTorch formulas for a recurrent layer, their sizes are 14,595 and 19,395 trainable parameters, respectively. As indicated, the output originates from the last step of the window.

The MLP acts as the baseline without explicit memory, such that two identical inputs produce the same force; meanwhile, GRU and LSTM allow testing whether recent history improves prediction. The width of 64 units, the two hidden layers of the MLP, and the single recurrent layer are kept fixed throughout the evaluation. The parameter choice was contrasted with a 128-unit variant; however, the supervised improvement did not yield a systematic advantage, so it was decided not to modify the parameters.

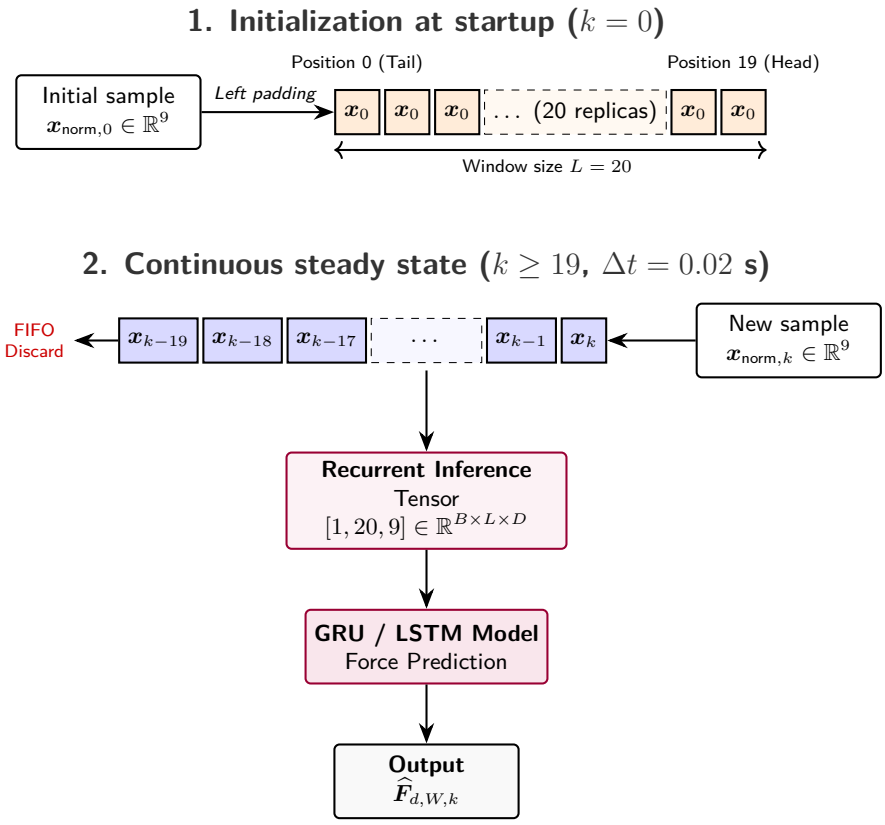


Figure 3.10. Management of the temporal sliding window in recurrent networks and initialization by sample repetition at startup ($k = 0$). The FIFO queue generates a batch of dimensions $[1, 20, 9]$ at each inference. Source: own elaboration.

Normalization, Training, and Model Selection

Before training, the mean and standard deviation of inputs and targets are calculated using only the training set. This normalization prevents variables with larger absolute magnitudes from dominating the loss due to scale differences, expressing each component as a deviation relative to the statistics calculated during training. These same statistics are subsequently applied to validation, test, and inference, and the network output is denormalized before being sent to the controller.

Learning minimizes the mean squared error (MSE) between the normalized force predicted by the network and the normalized force computed by the specialized PD. Optimization is performed using Adam [21] with a learning rate of 10^{-3}. A limit of 100 epochs, batches of 64 samples, and seed 42 are set for training reproducibility. Robustness to the seed was verified with 2 additional values, and the resulting variability did not cause a significant change in the interpretation of results. This random ordering is only applied during training, to reduce gradient dependency on the episode read sequence.

After each epoch, the validation loss is calculated, and the checkpoint with the lowest value is saved. Training stops if the validation performance does not improve for ten consecutive epochs, and the best model is preserved, thus reducing the number of runs and limiting overfitting.

A low supervised loss indicates that the network accurately replicates the forces commanded by the specialized controllers on the observed samples. However, it does not necessarily imply that it minimizes position error in closed loop, since small force deviations can accumulate and lead the network to visit states poorly represented in training. Therefore, supervised evaluation solely verifies fidelity to the classic controller.

Closed-Loop Inference

During inference, the same input variables are constructed in each cycle, normalized using the statistics learned during training, and passed to the network. The predicted force is denormalized and interpreted again as $\widehat{\boldsymbol{F}}_{d,W}$. In the MLP, this operation depends only on the current step, whereas in the GRU and LSTM, a queue of the last 20 normalized inputs is maintained, proceeding as detailed in tensors of inputs and targets.

Before converting the force to a desired attitude, just as in the classic implementation, the force norm is limited to $2.5mg$, tilt is restricted by the maximum angle, and negative thrust requests are avoided by imposing a minimum upward component. Subsequently, the classic inner loop calculates moments and rotor saturations are applied.

Telemetry records how many times thrust clipping and tilt clipping are activated, calculating percentages relative to the number of control steps executed in the simulation. These metrics are interpreted alongside the other output parameters, as a trajectory might be completed due to frequent intervention of safety protections, indicating suboptimal operation of the network.

Alternatives Implemented Outside the Main Comparison

During development, alternatives were implemented that are not part of the main comparison of this work. One of them learns an instantaneous, real-time gain scheduling: the network predicts multipliers for the outer-loop gains $\boldsymbol{K}_{p,p}$ and $\boldsymbol{K}_{d,p}$, bounded within a preset interval, and the cascaded PD then calculates the force, attitude, and commands.

A force-prediction variant with an expanded observation was also implemented. Instead of using only the nine variables necessary to reproduce the external PD, it incorporates 31 variables including observed state, reference, errors, and a continuous representation of yaw. This can serve to study whether the additional context improves prediction, but the network ceases to behave as a strict replacement of the external PD loop.

3.4. DATASETS AND EVALUATION METRICS

Comparison Criteria

The objective is not to demonstrate the superiority of an architecture in all cases, but to determine under what conditions a neural controller can replace several specialized outer-loop PDs, what is lost when transferring a PD outside its family, and what role temporal memory plays in GRU and LSTM networks.

The first benchmark is the specialized PD of each family. Its role is to define the expected performance when the classic controller is tuned specifically for the type of trajectory being evaluated. The second benchmark is cross-transfer: applying those same PDs, with their parameters already fixed, to other families without retuning. In this way, the quality of the controller in its specialized conditions can be separated from the performance loss in other situations.

On this basis, the imitation-trained neural networks are evaluated. First, supervised fidelity is measured using the desired force from the PDs, although this value does not replace closed-loop execution. The evaluation of MLP, GRU, and LSTM does not assume that recurrence must necessarily improve tracking. Temporal memory can be useful in the presence of delays or dependence on previous states, but it can also introduce a different sensitivity when the trajectory deviates from the training distribution.

Results are judged using tracking error, actuation, saturation, and the use of limitations independently. Likewise, a distinction is made between known conditions, classic transfer, and OOD scenarios to identify at which stage of the procedure each difference appears and its source.

Classic Dataset Design

The dataset contains 150 episodes: 18 of `hold`, 48 of circle, 48 of Lissajous, and 36 of `waypoint`. Their splits by family are 12/3/3, 34/7/7, 34/7/7, and 25/5/6 for training, validation, and testing. In total, this results in 105 training episodes, 22 validation episodes, and 23 test episodes. The allocation of scenarios to each split is generated deterministically using seed 1042.

The families are chosen to cover hover equilibrium, periodic planar tracking, periodic motion in multiple axes, and finite runs with acceleration, braking, and settling. Each combines geometries of varying difficulty with profiles of drag, wind, noise, and actuators. `hold` uses 18 combinations and fewer profiles because it does not introduce the geometric diversity of the other families, thereby avoiding an overrepresentation of samples in the training set.

Metrics and Success Criteria

For the samples $e_k = \|\boldsymbol{p}_{r,k} - \boldsymbol{p}_k\|_2$, the following are calculated for each scenario:

$$e_{\text{RMSE}} = \sqrt{\frac{1}{N}\sum_{k=1}^{N} e_k^2}, \qquad e_{\text{MAE}} = \frac{1}{N}\sum_{k=1}^{N} e_k, \qquad e_{\max} = \max e_k. \tag{3.26}$$

RMSE penalizes large errors, MAE describes the average error, and maximum error detects localized extremes.

Actuation is described separately using collective thrust in N, moments in N m, and rotor speed in rad/s. Saturation and degradation are percentages of samples, and the percentages of norm clipping and inclination clipping of the force predicted by the neural networks are also added.

A trajectory is considered successful if the termination is of a temporal nature, or trajectory/composition completion. Infinite references must reach the time limit, while `waypoint` must complete its path.

3.5. EXPERIMENTAL WORKFLOW AND REPRODUCIBILITY

Global Procedure Design

The experimental procedure follows ten phases. First, initial checks of the simulator and representative scenarios are run. Then, the initial classic dataset is generated, the PDs are diagnosed per family, and, if necessary, a specialized PD is searched for each family. With these fixed-parameter controllers, the classic scenarios are regenerated and both nominal and cross-transfer simulations are executed.

From that baseline, imitation data is generated. Subsequent phases train MLP, GRU, and LSTM, evaluate their supervised fidelity, execute the closed-loop comparison on the test set, evaluate performance on the OOD scenarios series, and finally compile the results into comparative tables. The detailed commands for these phases are deferred to Appendix 1. Figure Figure 3.11 synthesizes this sequence and separates distinct groups of steps.

The training set is involved in tuning controllers, normalization, and weight optimization; validation is used to select the network checkpoints; and testing and OOD scenarios are employed for the final performance verification.

Training Configuration and Comparison

The three architectures share the dataset, nine inputs, three targets, 64 hidden units, training normalization, MSE loss, Adam optimizer, a learning rate of 10^{-3}, batch size

of 64, a maximum of 100 epochs, a patience of 10, and seed 42. They also share the criterion of saving the lowest validation MSE.

The comparison is performed on the common set of parameters to evaluate solely the effect of replacing the architecture. Additionally, a sensitivity study on the configuration parameters was conducted to verify its robustness.

Evaluation on Known Families and Classic Transfer

The ID evaluation uses the 23 test episodes: 3 `hold`, 7 circle, 7 Lissajous, and 6 `waypoint`. In each scenario, the native PD and the three networks are run under identical physical configurations. Thus, a first comparison is made against the specialized PD of each family, which represents the most demanding reference.

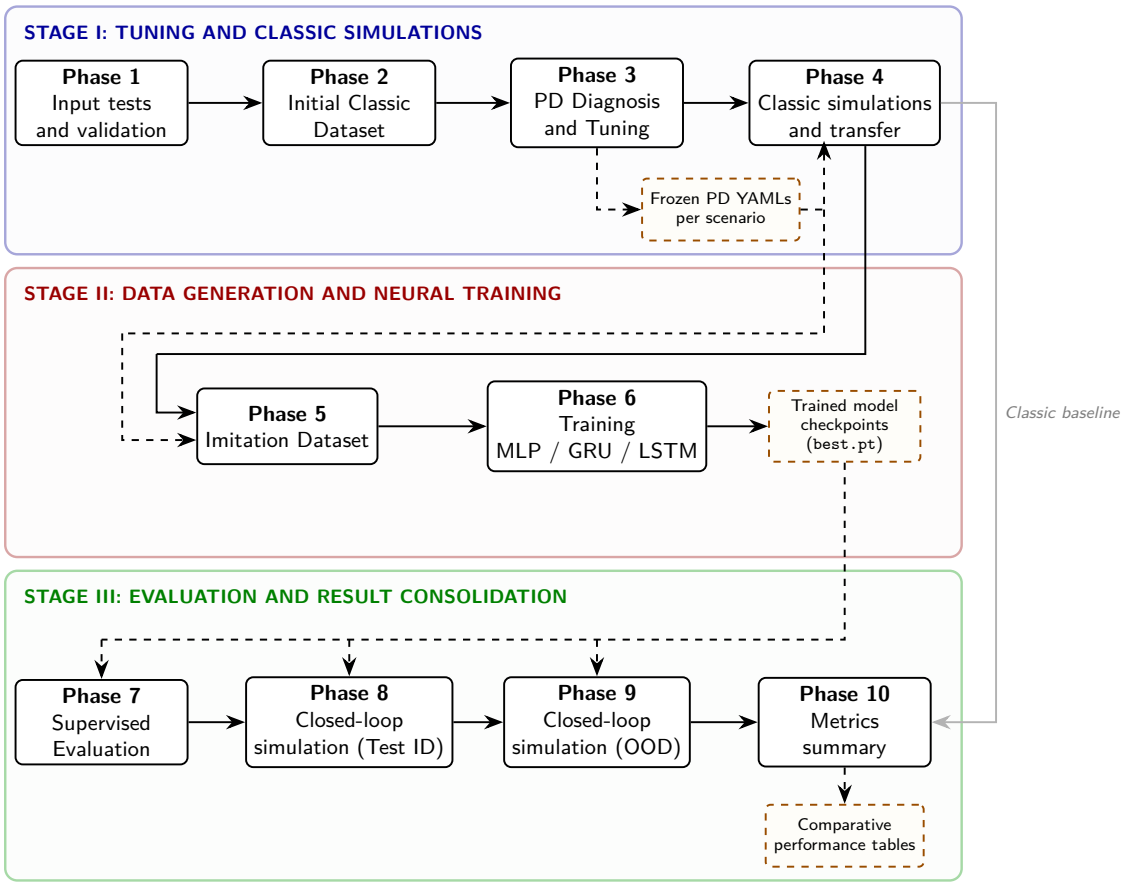


Figure 3.11. General flow of the experimental procedure structured into 10 reproducible phases. The progressive generation of the training datasets, the classic optimization flow, neural training, and validation and OOD evaluation stages are illustrated. Source: own elaboration.

Classic transfer applies the four fixed-parameter PDs to each of the 23 scenarios. This produces $23 \times 4 = 92$ runs and a controller–family matrix. Differences are attributed

solely to the controller change, since the remaining parameters, including the seed, are kept fixed.

Evaluation Out of the Training Distribution

OOD evaluation groups scenarios whose episodes are not involved in any other phase of the implementation. Its function is to check what happens when controllers are subjected to conditions never seen during tuning or network training. The OOD series is organized by level of novelty. In all cases, the networks and the transferred PDs are run under the same conditions, and the available PDs are used as references for comparison.

Figure Figure 3.12 summarizes the trajectories used at each evaluation level. It starts with variations on known families, then classic transfer is evaluated, and finally OOD scenarios are presented to study the recombination of known capabilities on new geometries. With this separation, OOD results can be interpreted according to the degree of novelty introduced.

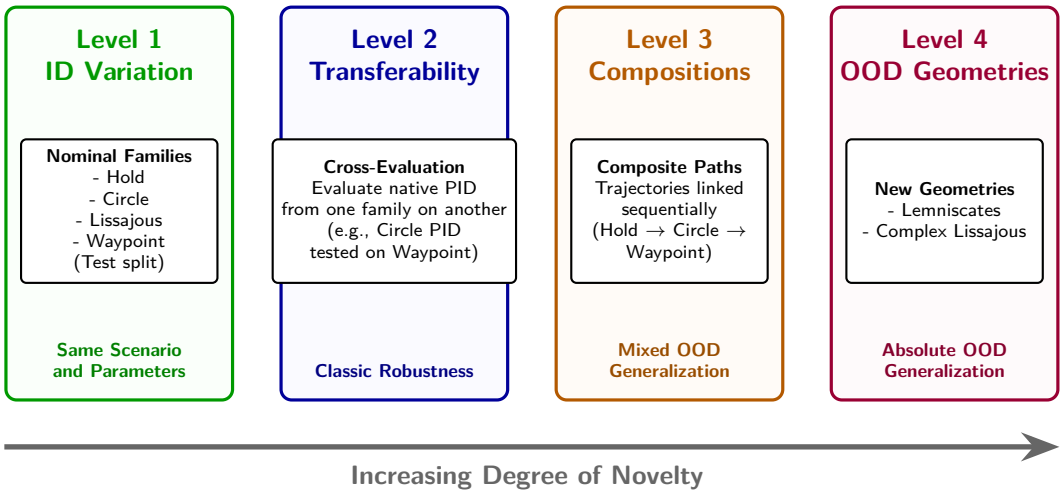


Figure 3.12. Classification of the evaluation levels and experimental generalization. Four levels are structured with incremental novelty and difficulty: nominal ID interpolation, classic transfer, recombination of known capabilities, and OOD geometries. Source: own elaboration.

Variations and Compositions of Known Families

The first OOD group preserves paths present in the training set but modifies their difficulty or combines them. It includes three-dimensional, faster, or high-noise Lissajous trajectories, sequential compositions of `hold`, `circle`, `waypoint`, and higher-aggressiveness segments, as well as helical trajectories approximated by waypoints.

This separation allows evaluating two distinct questions. Variations of a known family measure whether the controller tolerates parameter changes in a structure already

present. Compositions evaluate whether a single policy can navigate heterogeneous segments without explicitly switching controllers. A specialized PD may perform well on the part for which it was tuned and degrade when the geometry changes, whereas the MLP, GRU, and LSTM networks are evaluated as single controllers applied to the entire trajectory.

Completely New Trajectories

The second OOD group introduces a more intense geometric novelty. Lemniscates do not belong to the four training families and force the vehicle to follow a closed curve with curvature changes and vertical components. The implementation uses a Gerono lemniscate in the horizontal plane,

$$x(t) = c_x + a\sin(\omega t), \qquad y(t) = c_y + b\sin(2\omega t), \tag{3.27}$$

with an optional vertical oscillation $z(t) = c_z + A_z \sin(\omega_z t)$. In the OOD series, three-dimensional variants at various speeds and with combined disturbances are used. This category is a more demanding test since it changes the entire structure of the reference and requires curvature changes. Keeping it separate from the other OOD trajectories allows studying its behavior independently.

4. Experimental Results

4.1. EXECUTION COVERAGE AND VALIDITY

The comparison used to test the hypotheses is based on a closed-loop, reproducible, and traceable execution. Before presenting interpretations, the size of the evaluation series is detailed; therefore, Table 4.1 summarizes scenarios, controllers, runs, and the percentage of cases that complete the simulation without exceeding the established constraints. Finally, generation and visualization commands are compiled in Appendix 1, and the code structure producing these artifacts is synthesized in Appendix 2.

Table 4.1. Coverage of the comparative execution.

Split	Scenarios	Controllers	Runs	Completed [%]	Limits [%]
`test`	23	8	184	99.5	99.5
`ood`	10	8	80	66.3	76.3

The eight controllers in the table correspond to four PDs of the families (`hold`, `circle`, `lissajous`, and `waypoint`), three neural networks (MLP, GRU, and LSTM), and a representative PD benchmark, for which the native PD is taken under nominal conditions and the assigned PD is used for each OOD scenario.

In the table, a clear difference between ID and OOD conditions is observed. In the latter, a higher number of trajectory failures and violations of actuation limits occur.

4.2. CLASSIC REFERENCE AND CROSS-FAMILY TRANSFER

Before assessing the networks, the transfer of specialized PDs to other families must be measured. This phase is highly important since the goal of the neural networks is to compete against this transfer.

Figure 4.1 shows a partial specialization. The `hold` PD transfers the worst to the other families, and the `hold` family itself achieves a better RMSE with other PDs than with its own specialized controller. This result suggests that the tuning of `hold`, at least with the criteria applied in this run, does not achieve the best possible result for this family.

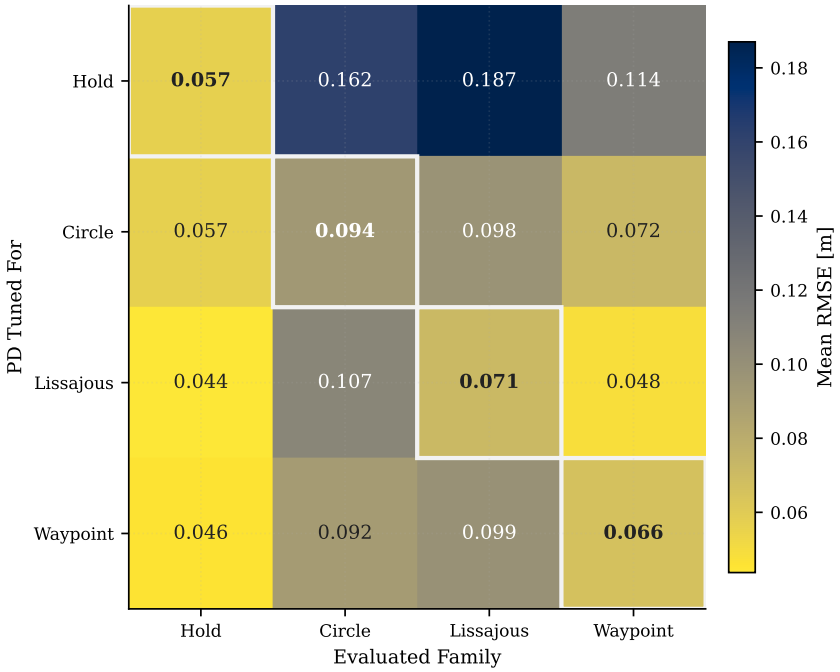


Figure 4.1. Cross-transfer of PD controllers on the `test` set. The rows indicate the family used for tuning and the columns the evaluated family. The diagonal marks the use of the specialized controller in its native family. Source: own evaluation.

In `circle`, the specialized PD offers a low error and also transfers reasonably to `waypoint`, consistent with both references requiring continuous and smooth tracking, albeit with different geometries. The `lissajous` PD presents the best RMSE in its family and also performs very well on `hold` and `waypoint`, but does not replicate the same margin on `circle`. Finally, the `waypoint` PD does not transfer with the same quality to `circle` or `lissajous`, where sustained curved tracking requires a different response compared to essentially straight segments.

4.3. NEURAL LEARNING UNDER KNOWN CONDITIONS

Supervised evaluation does not measure trajectory tracking, but rather fidelity to the desired force calculated by the specialized PDs. Thus, Table 4.2 is introduced first to address two questions: on one hand, how well each network mimics the training signal, and on the other, what happens subsequently when it is fed back into the dynamics.

Table 4.2. Supervised force fidelity and associated inclination clipping. Clipping percentages correspond to the activation of the inclination limit before applying the force in the inner loop.

Architecture	Norm. MSE	Force RMSE [N]	Supervised Clip [%]	Closed-loop Clip [%]
MLP	0.0185	0.083	0.057	0.062
GRU	0.0102	0.053	0.000	0.000
LSTM	0.0111	0.055	0.000	0.000

GRU and LSTM mimic the specialized PD's force better. However, this advantage does not directly translate to position tracking. Figure 4.2 shows that the MLP obtains a lower RMSE on `circle` and `lissajous`, while the LSTM lags behind despite its superior force fidelity. On `hold`, all networks maintain low errors, and on `waypoint`, none clearly outperform the representative PD.

The contrast between Table 4.2 and Figure 4.2 shows that a network can approximate the specialized PD's force better on the samples and yet not be the best once the loop is closed.

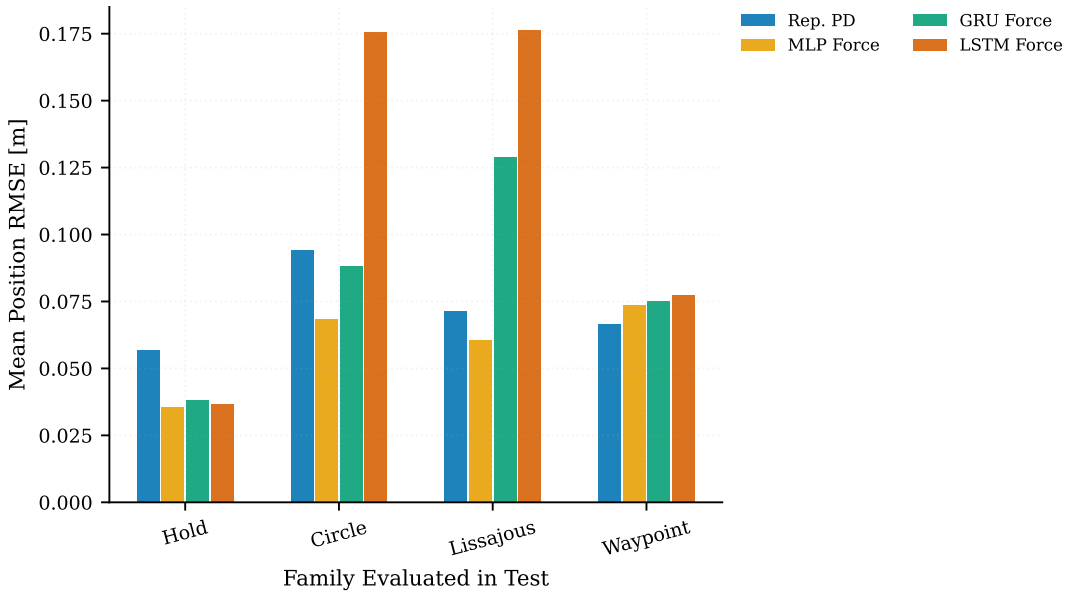


Figure 4.2. Average RMSE under known conditions for the representative PD and the three neural architectures. Only tracking error is shown, as it is the primary metric in this comparison.

4.4. OUT-OF-DISTRIBUTION GENERALIZATION

OOD evaluation is the most demanding part of the comparison, as it subjects the controllers to geometries, speeds, and combinations of disturbances not directly present in the training set. In this series, the representative PD benchmark is taken from the specialized PD closest to the scenario: `lissajous` for `lemniscate`, `composite`, and `lissajous`, and `waypoint` for helices.

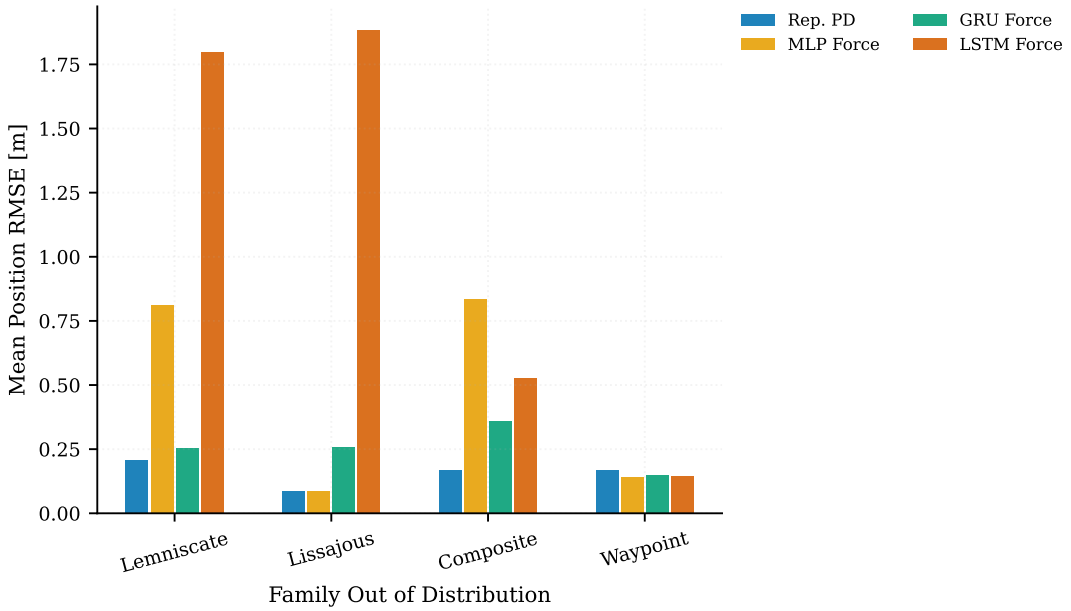


Figure 4.3. Average out-of-distribution RMSE by scenario family.

Figure 4.3 first aggregates by OOD family to identify general trends. It shows that the representative PD maintains a competitive behavior in several OOD groups. It also confirms that the LSTM is once again the architecture with the largest average error in the most demanding families. The GRU achieves a better result than the MLP in the completely new `lemniscate` and `composite` families, while the MLP remains particularly competitive in the OOD variants closest to known families, such as `lissajous` and `waypoint`.

This high-level aggregation can mask significant variability. Therefore, Figure 4.4 descends to the scenario level, showing which specific combinations of controller and trajectory concentrate the error.

The breakdown by scenario in Figure 4.4 qualifies the averages in Figure 4.3. Most scenarios maintain low errors, but cases combining lemniscatas, composite transitions, or high dynamic demands concentrate the failures. In particular, the MLP exhibits a sharp increase in error in composite scenarios with lemniscatas, while the LSTM degrades significantly in fast trajectories or those with pronounced geometric changes. The pattern suggests that temporal memory does not provide a uniform advantage and can introduce additional sensitivity when the visited state deviates from the training distribution.

The next step is to distinguish between error and failure in the simulation. Figure 4.5 groups the termination causes to analyze whether degradation stems from attitude, saturation, maximum duration, or other conditions.

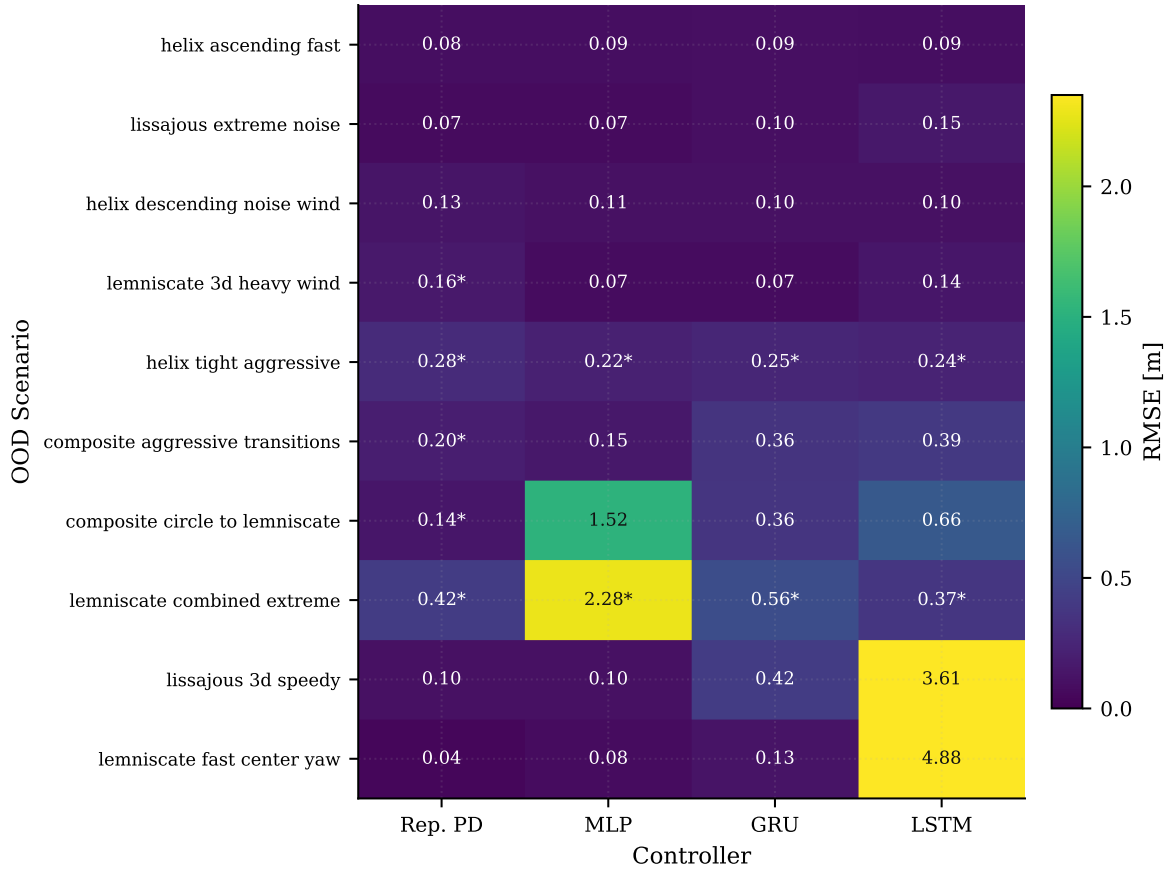


Figure 4.4. OOD breakdown by scenario and controller for the four main references. The number in each cell is the RMSE in meters; the asterisk indicates that the run did not complete, although the episode is kept in the aggregation.

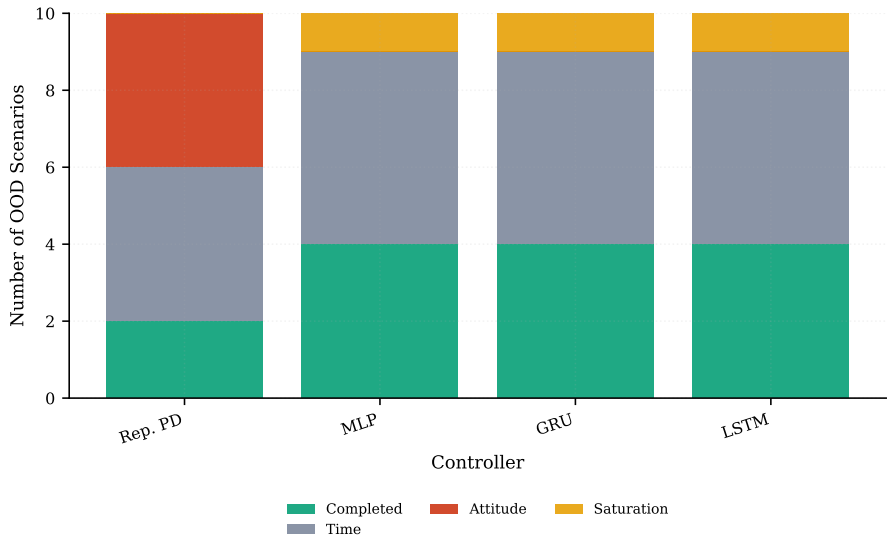


Figure 4.5. Termination types in the OOD series for the main controllers.

The representative PD accumulates attitude failures in several OOD scenarios, while the networks concentrate their failures on saturation. Furthermore, the number of errors is larger in the classic implementation, which is relevant for drawing a conclusion.

4.5. REPRESENTATIVE ARCHITECTURE CASE

Figure 4.6 illustrates a specific case where two architectures with the same interface produce very different behaviors. The MLP exhibits a larger initial error but stabilizes tracking and completes the geometry reasonably. The LSTM, by contrast, drifts from the reference throughout the simulation. This result reinforces the need to evaluate behavior on a trajectory-by-trajectory basis, which is why a set of geometry plots and telemetry information is presented in Appendix 3.

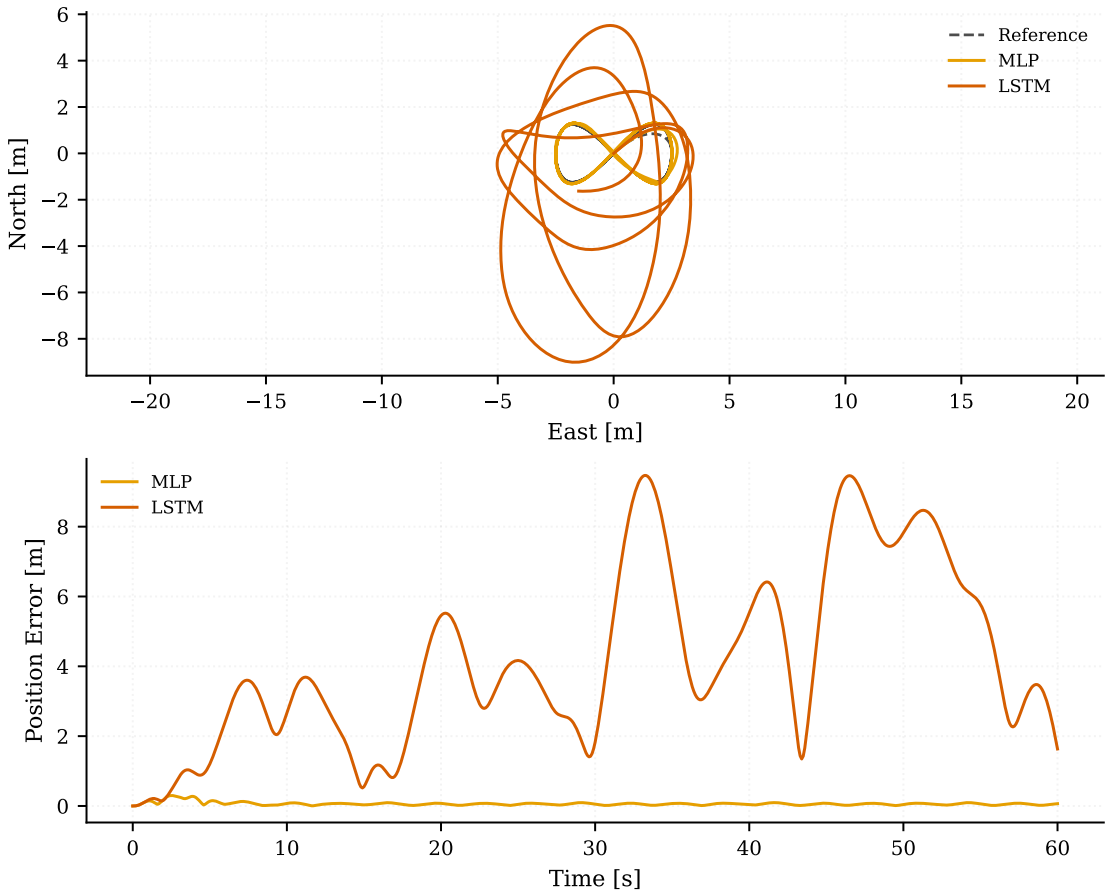


Figure 4.6. OOD scenario `lemniscate_fast_center_yaw`: visual comparison between MLP and LSTM. The top panel shows the horizontal plane and the bottom panel shows the position error throughout the episode.

4.6. ACTUATION, PROTECTIONS, AND SHARED LIMITS

The observed results do not depend solely on the network. The classic inner loop, mixer, actuator saturation, and force clipping limit the commands that ultimately reach the

vehicle. In Figure 4.7, mixer degradation measures the percentage of time collective command scaling is required to respect actuation limits, and the clipping percentages indicate how much forces are clipped before passing to the inner loop.

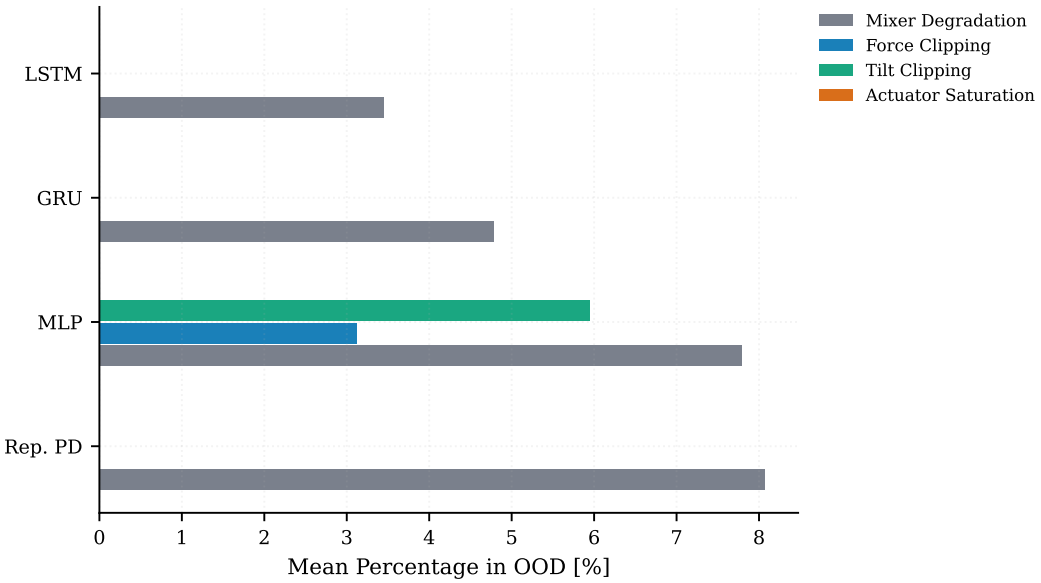


Figure 4.7. Average activation of protections and actuation limits in OOD. The figure separates tracking error, control demand, and constraints shared by the control architecture.

The MLP is the only one of the three networks that activates force and inclination clipping appreciably in OOD. This behavior is consistent with a stateless architecture that responds to instantaneous errors with more aggressive demands. The GRU and LSTM, by incorporating temporal history, do not activate these clippings to the same extent, as they observe the recent trend.

4.7. DISCUSSION OF RESULTS AND LIMITATIONS

Synthesis of Results

The results show that the comparison cannot be reduced to a single metric. The transfer of specialized PDs depends on the trajectory family: some combinations maintain low errors, while others clearly degrade with changes in geometry or dynamic requirements. In the networks, supervised fidelity alone does not predict closed-loop behavior either. Although GRU and LSTM approximate the force of the specialized PD better during training, the MLP obtains more favorable tracking errors in several in-distribution families, confirming that mimicking the PD's action does not necessarily equate to improving the resulting trajectory.

Out of distribution, better performance is observed in moderate cases, whereas demanding scenarios show limitations in the controllers. The neural architectures are

competitive in several recombinations close to the trained families, but they do not globally outperform the representative PD. Furthermore, failures do not stem solely from position error; saturations, clipping, and inner-loop protections condition the interpretation of the results. Thus, performance depends on the interaction between trajectory family, neural architecture, the imitated PD, and system limits.

Interpretation with Respect to Hypotheses and Objectives

The presented evidence supports the hypothesis that replacing several specialized PDs with a common neural implementation is viable for known families, but it does not lead to homogeneous behavior across architectures. On the `test` set, the MLP, GRU, and LSTM complete the trajectories and remain within an RMSE range comparable to the specialized PD. In OOD, the networks are competitive in moderate-difficulty recombinations, while newer, faster, or more demanding geometries show that generalization depends on both geometric novelty and dynamic requirements.

No global neural superiority can be claimed. The simulation success rate in OOD (66.3%) falls clearly below that obtained on the `test` set (99.5%), and failures are not explained solely by an increase in tracking error but also involve attitude limits, persistent saturation, and timeout in finite trajectories.

Dependency on the Specialized PD and the Dataset

The networks mimic the desired force calculated by specialized PDs; they are not trained by reinforcement learning, nor do they directly optimize the position error. Therefore, their performance is limited by the quality of the specialized PDs, the filters applied to the input simulations, and the geometric and dynamic coverage of the dataset. If the specialized PD produces suboptimal actions for a specific region, the network is likely to reproduce that limitation.

The comparison between supervised fidelity and closed-loop performance illustrates precisely this limitation. GRU and LSTM present lower force error during training, but this does not translate uniformly to RMSE. A plausible interpretation is that the MLP, by not depending on an explicit time window, can respond more locally to states that resemble samples seen during training, whereas recurrent networks can be more sensitive when recent history deviates from learned patterns.

Value and Limits of the Classic Inner-Loop Approach

Maintaining the classic inner loop allows isolating the network's contribution, so that the network is only responsible for the desired outer force, while attitude tracking, the mixer, and actuation limits follow an interpretable classic design.

Figure 4.7 shows that the ultimate safety depends on this combination. Saturating terminations do not indicate instability, but rather that forces or kinematic changes exceeding the set limits are requested, which, although necessary, condition the interpretation of the results.

Role of the Neural Architecture

The results indicate that the architecture, and in particular the presence or absence of temporal memory, substantially affects behavior. The MLP obtains low errors in several ID families and remains competitive in mixed OOD, but it is also the architecture that activates force and inclination clipping most clearly. This suggests a more aggressive response to instantaneous errors.

The GRU offers a solid compromise between supervised fidelity and performance in several OOD scenarios, especially when evaluated on families not directly seen. The LSTM, by contrast, does not systematically outperform the GRU and exhibits very marked failures in some fast or geometrically demanding scenarios, such as the representative case in Figure 4.6. Recurrent memory should not be interpreted as a direct improvement; it provides temporal context while introducing sensitivity to poorly represented data and the accumulation of deviations.

Validity, Limitations, and Reproducibility

The simulator provides a valid platform for studying the comparison, although it does not substitute validation with real hardware. The OOD series includes ten scenarios and contains highly demanding cases.

The execution is deterministic per scenario and seed, meaning statistical uncertainty over multiple runs has not been estimated. Furthermore, other neural formulations were explored during the work but are not incorporated into the final comparison. The networks are not retrained after defining the OOD series, and the checkpoints remain fixed, meaning direct transfer is evaluated rather than a recurrent adaptation process.

Another limitation comes from the choice of the representative PD in OOD. For lemniscatas, Lissajous, and composite trajectories, the `lissajous` PD is used, and for helices, the `waypoint` PD is used. This decision is reasonable due to the geometric similarity between the trajectories, but remains an approximation. The curves share some features, and composite trajectories depend on the segment that poses the greatest difficulty. Finally, although various configurations of parameters and hyperparameters were tested to support the final decisions, no exhaustive hyperparameter search or evaluation over thousands of seeds was performed.

The regeneration of scenarios, controller execution, and figure generation are contained within the GitHub repository. The main commands are documented in Appendix 1, so that the chapter's tables and figures can be reconstructed.

5. Conclusions and Future Work

5.1. CONCLUSIONS

This work has been developed in a reproducible and traceable environment to compare classic and neural imitation control for quadcopter trajectory tracking. The contribution consists of a complete pipeline: from a fully modular 6DOF simulator, to the creation of test scenarios, specialized PD tuning, MLP, GRU, and LSTM network training, and their closed-loop execution, up to the generation of comparative metrics.

The initial hypothesis is partially fulfilled, with nuances. Neural architectures trained to predict the desired force of the outer loop are viable on trajectory families known during training, and they can successfully replace the set of PD controllers in several cases and complete their objective. However, no global superiority of the neural networks is observed. Under out-of-distribution conditions, failures emerge, alongside notable differences between architectures and a clear dependency on the geometry and dynamic demand level of the reference trajectory.

The first conclusion addresses the specialization of classic control. Some transferred PDs perform better than the specialized PD for the specific family, while others lose practically all performance when departing from their tuning trajectory. This confirms that classic transfer is uneven, and therefore this set of specialized PDs represents a necessary baseline for evaluating neural imitation. It also shows that the PD tuning process could be refined in future versions, particularly in families where the specialized controller is not the best on its own trajectories or behaves better in transfer than in native mode.

The second conclusion concerns the comparison between supervised fidelity and closed-loop tracking. GRU and LSTM approximate the specialized PD's force better in supervised evaluation, but they do not systematically dominate position RMSE. The MLP, despite not using explicit temporal memory, obtains very competitive results in several ID families and in mixed OOD. Therefore, while supervised fidelity is necessary, it is not sufficient to predict the quality of dynamic tracking.

The third conclusion centers on recurrent memory, as it does not yield automatic improvement. The GRU offers a solid result in several simulations, whereas the LSTM exhibits severe failures on geometrically demanding trajectories that are faster than those encountered during training. This suggests that history can provide value in certain states, but can also introduce sensitivity to poorly represented states, larger delays, and accumulated deviations. Consequently, the choice of architecture should be analyzed in conjunction with the trajectory type and its specific characteristics.

The fourth conclusion concerns safety in simulations, which belongs to the control system as a whole, not just the neural network. The classic inner loop, mixer, actuator saturation, and force clipping condition the final behavior. This design decision was appropriate because it allowed studying the neural network's behavior while maintaining controllability over the rest of the system, although it required more thorough analysis before attributing observations to the neural component.

Overall, the work meets its objective of comparing the replacement of a set of PDs with a neural implementation of desired force, all within a traceable, reproducible, and coherently scoped experimental environment. The results show that neural imitation is promising under conditions known during training and in certain OOD variations, although generalization cannot be taken for granted.

5.2. SCOPE AND LIMITATIONS

The physical scope of the work is defined by the 6DOF simulator described in Section 3.1. The vehicle is represented as a rigid body; position and velocity are expressed in the East–North–Up (ENU) world frame, while angular velocity, moments, and rotor geometry use the Forward–Right–Down (FRD) body frame. Attitude is propagated using quaternions, and dynamics use Newton–Euler equations integrated with fixed-step RK4. The model includes gravity, simplified linear drag, constant wind, Gaussian noise for position and velocity observations, a quadratic rotor law, delay, first-order dynamics, and actuator saturation.

These choices represent modeling assumptions. Constant mass, geometry, inertia, and rotor parameters are assumed. Structural flexibility, ground effect, rotor-to-rotor interference, rotor flapping, non-linear aerodynamics, turbulence, battery discharge, and thrust variation with voltage are not modeled. Wind is treated as a constant disturbance, and linear drag as a simplified dissipative term. Observations incorporate synthetic noise in position and velocity; the expected variability of an IMU, GNSS receiver, or specific delays of real sensors is not included.

The scope of the experiments is also bounded. The in-distribution dataset contains 150 episodes distributed among `hold`, `circle`, `lissajous`, and `waypoint`, with 105 training episodes, 22 validation episodes, and 23 test episodes. Within this environment, ID families, PD transfer between families, and a separate OOD series of ten scenarios with compositions, variations, helices, and lemniscatas are evaluated. The summary in Table 4.1 collects 184 runs in `test` and 80 OOD runs, sufficient to test the hypothesis, though not to exhaustively cover all geometries, speed profiles, vertical actions, disturbances, or maneuvers that would occur in real flight.

The controller comparison is limited to fixed-parameter specialized PDs and three neural network architectures for external force prediction: MLP, GRU, and LSTM. The networks are trained by supervised behavior cloning of the desired force calculated by specialized PDs, preserving the classic inner attitude loop, mixer, saturations, and force and inclination limits. Therefore, a full neural policy or a controller trained to minimize position error in closed loop via reinforcement learning is not evaluated. The neural force prediction implementation can inherit biases from the PDs and from the limited coverage of the training set. Furthermore, the PD used as a reference in OOD scenarios is an approximation based on geometric similarity, not a controller optimized for those geometries.

The parametric study has remained limited. The PDs are tuned using a reproducible procedure and then their parameters are fixed, but no global search in the space of possible gains or exhaustive multi-objective optimization was run. Similarly, MLP, GRU, and LSTM share the input configuration, normalization method, number of hidden units, batch size, learning rate, early stopping criteria, and seed to isolate the effect of the architecture. Sensitivity tests were conducted on the number of hidden units, window size, and select seeds, but not a full sweep of hyperparameters, regularization, dataset size, or checkpoint selection criteria. No sweep over the physical parameters of the vehicle, delay and saturation levels, noise, or wind was performed either.

The statistical validity of the results is approached with caution. The runs are deterministic for a given scenario and seed, favoring reproducibility, but confidence intervals or estimates of variability across seeds are not provided.

Finally, attitude limits, persistent saturation, and force and inclination limits define the region of validity for the runs. These constraints are necessary for simulation analysis but condition interpretation: part of the behavior originates from the complete control architecture rather than the neural network alone.

5.3. FUTURE WORK

A first line of work consists of adapting the simulator to a specific vehicle. This requires identifying mass, inertia, rotor geometry, actuator limits, thrust curve, delays, saturations, and sensor parameters from tests or precise technical documentation. Once system identification is completed, the results observed in simulations would bear a more direct relationship to the capability of a real quadcopter.

The second line is to increase the physical fidelity of the model. The current simulator includes a bounded set of phenomena, sufficient to conduct this comparison, but more advanced models of aerodynamics, ground effect, turbulence, rotor-to-rotor interference, battery discharge effects, thrust variation with voltage, and thermal, electrical, and

flexible structural constraints can be incorporated. These extensions would allow studying whether controllers maintain stable behavior when actuation response varies unpredictably during a simulation or when effects currently estimated in a simplified manner appear.

It is also necessary to expand and refine the set of trajectories. In this work, few representative families have been used, but a flight-oriented implementation should increase the number of families, their geometric diversity, speeds, vertical profiles, and transitions between different trajectories. This refinement should be accompanied by a review of termination, thrust, tilt, and saturation limits to reflect the actual demands of a given vehicle type.

Another highly relevant line is the systematic study of neural architectures. In this work, MLP, GRU, and LSTM are compared under a series of common configurations to isolate the effect of the architecture. In a subsequent phase, the objective would be to find the best configuration for trajectory tracking, exploring width, depth, window length, normalization, input selection, regularization, seeds, training set size, and checkpoint selection criteria. It is feasible that each architecture requires a different size and window, as well as possessing different strengths, an aspect hinted at in the discussed results.

Within that line, the inference method for recurrent networks should also be studied. Currently, a window of 20 samples is reconstructed and processed in each control cycle without directly reusing the hidden states calculated in the previous cycle. An alternative would be to preserve these states between steps, both to reduce recalculation and to approach a more stateful inference. However, this modification would alter the relationship of the network with its operation, since the output would depend on a state propagated between internal cycles rather than just a window of samples calculated at each step.

Likewise, it is advisable to conduct a correlation study to identify which data subsets contribute to generalization capacity. It is not enough to add more trajectories; rather, one must analyze which families, parameters, and disturbances truly enrich the training distribution. This line of research would allow distinguishing whether OOD failures stem from geometric limitations, a lack of high-acceleration states, overly restrictive actuation limits, or an architecture incapable of representing the relationship between observation, reference, and desired force in those types of trajectories.

The natural subsequent step is to transfer the approach to a real vehicle. To do so, simply loading the network trained in simulation is insufficient; portability to a lower-level programming language is required, as well as guaranteeing that the real quadcopter provides the necessary state, reference, and actuation variables. It would also be necessary to align control, telemetry, and clock frequencies between the simulator

and the vehicle, define a progressive testing procedure, and establish safe operating limits.

Finally, a closing line of work consists of using the simulator as a pre-flight validation platform. This would allow testing new trajectories, detecting regimes where the controller demands excessive forces, adjusting protection limits, and bounding the risk for real trials. In this way, the developed work would serve as a baseline to transition from a simulation for comparisons to an experimental evaluation on hardware.

References

[1] R. W. Beard and T. W. McLain, *Small Unmanned Aircraft: Theory and Practice*. Princeton University Press, 2012.

[2] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2011, pp. 2520–2525.

[3] T. Lee, M. Leok, and N. H. McClamroch, "Geometric tracking control of a quadrotor UAV on SE(3)," in *Proceedings of the IEEE Conference on Decision and Control*, 2010, pp. 5420–5425.

[4] D. J. Leith and W. E. Leithead, "Survey of gain-scheduling analysis and design," *International Journal of Control*, vol. 73, no. 11, pp. 1001–1025, 2000.

[5] J. B. Kuipers, *Quaternions and Rotation Sequences: A Primer with Applications to Orbits, Aerospace, and Virtual Reality*. Princeton University Press, 1999.

[6] P. Pounds, R. Mahony, and P. Corke, "Modelling and control of a large quadrotor robot," *Control Engineering Practice*, vol. 18, no. 7, pp. 691–699, 2010.

[7] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 3rd ed. John Wiley & Sons, 2016.

[8] K. J. Åström and B. Wittenmark, *Computer-Controlled Systems: Theory and Design*, 3rd ed. Prentice Hall, 1997.

[9] The MathWorks, Inc., "UAV Toolbox Documentation," https://www.mathworks.com/help/uav/index.html, accessed: July 1, 2026.

[10] S. Folk, J. Paulos, and V. Kumar, "RotorPy: A python-based multirotor simulator with aerodynamics for education and research," arXiv:2306.04485, 2023. [Online]. Available: https://arxiv.org/abs/2306.04485

[11] PyTorch Contributors, "PyTorch Documentation," https://docs.pytorch.org/docs/stable/index.html, accessed: July 1, 2026.

[12] Astral Software Inc., "uv Documentation," https://docs.astral.sh/uv/, accessed: July 1, 2026.

[13] YAML Language Development Team, "YAML Ain't Markup Language version 1.2.2," https://yaml.org/spec/1.2.2/, accessed: July 2, 2026.

[14] T. Bray, "The JavaScript Object Notation (JSON) Data Interchange Format," RFC 8259, 2017.

[15] TOML Contributors, "TOML: Tom's obvious minimal language," https://toml.io/en/v1.0.0, accessed: July 2, 2026.

[16] GitHub, Inc., "GitHub Docs: Repositories," https://docs.github.com/en/repositories, accessed: July 1, 2026.

[17] J. G. Ziegler and N. B. Nichols, "Optimum settings for automatic controllers," *Transactions of the ASME*, vol. 64, no. 8, pp. 759–765, 1942.

[18] D. A. Pomerleau, "ALVINN: An autonomous land vehicle in a neural network," in *Advances in Neural Information Processing Systems*, 1989, pp. 305–313. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/1988/file/812b4ba287f5ee0bc9d43bbf5bbe87fb-Paper.pdf

[19] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 2011, pp. 627–635.

[20] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986.

[21] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[22] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th International Conference on Machine Learning*, 2010, pp. 807–814. [Online]. Available: https://www.cs.toronto.edu/~fritz/absps/reluICML.pdf

[23] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303–314, 1989.

[24] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.

[25] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[26] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 2014, pp. 1724–1734. [Online]. Available: https://aclanthology.org/D14-1179/

[27] J. A. Nelder and R. Mead, "A simplex method for function minimization," *The Computer Journal*, vol. 7, no. 4, pp. 308–313, 1965.

Appendix 1. Commands and Experimental Traceability

This appendix compiles the minimal commands required to reconstruct the experimental evidence used in this thesis. It does not replace the repository's `README.md`, but rather outlines a compact execution path.

Environment Setup

The project is managed using `uv`. The first command synchronizes the environment defined by `pyproject.toml` and `uv.lock`, while the second runs the available tests before launching long-running executions.

```
uv sync
uv run pytest
```

Minimal Scenario Execution

The basic execution takes a YAML scenario, initializes the vehicle, trajectory, controller, disturbances, and termination conditions, and outputs two files:

- `telemetry.json` with the time series and
- `metrics.json` with the aggregated metrics.

```
uv run simulador-quad run scenarios\hover_clean.yaml --no-visualization
```

```
set CASE=results\hover_clean
uv run simulador-quad plot ^
  %CASE%\telemetry.json ^
  --metrics %CASE%\metrics.json ^
  --out %CASE%\figures_report ^
  --profile report --formats png pdf --lang en
```

The same command allows executing individual scenarios during development. For the final figures of the visual appendix, however, the telemetry files from the classic dataset are used instead of isolated runs.

```
uv run simulador-quad run ^
  scenarios\lissajous_clean.yaml ^
  --no-visualization
```

Classic Dataset and Expert PDs

The classic dataset establishes reproducible scenarios and allows tuning or fixing PD controllers per family. After modifying gains or tuning criteria, the scenarios must be regenerated using `--overwrite` so that the incorporated gains and manifests remain consistent.

```
uv run python tools\generate_classic_dataset.py ^
  --version v1 --out data\classic_dataset\v1 --overwrite

uv run python tools\tune_classic_pid.py ^
  --dataset data\classic_dataset\v1 ^
  --out data\classic_dataset\v1\pids --workers 8
  %% Runs in parallel on 8 CPU cores

uv run python tools\run_classic_dataset.py ^
  --dataset data\classic_dataset\v1 ^
  --no-visualization --workers 4

uv run python tools\summarize_classic_dataset.py ^
  --dataset data\classic_dataset\v1
```

Neural Outer-Force Dataset

The neural controller learns the desired outer-loop force by behavior cloning. To do so, a bank of external PD candidates is generated first, the safe expert per scenario is selected, and the input/output samples used by the MLP, GRU, and LSTM are exported.

```
uv run python tools\generate_outer_force_pid_bank.py ^
  --dataset data\classic_dataset\v1 ^
  --out data\outer_force_pid_bank\v1 --workers 8

uv run python tools\generate_outer_force_dataset.py ^
  --source-dataset data\classic_dataset\v1 ^
  --pid-bank data\outer_force_pid_bank\v1 ^
  --out data\outer_force_dataset\v1

uv run python tools\train_neural_controller.py ^
  --dataset data\outer_force_dataset\v1 ^
  --architecture mlp ^
  --feature-version outer_force_min_v1 ^
  --out data\neural_control\outer_force_mlp_min_v1 ^
  --device auto

uv run python tools\evaluate_neural_controller.py ^
  --dataset data\outer_force_dataset\v1 ^
```

```
  --run data\neural_control\outer_force_mlp_min_v1 ^
  --device auto
```

Recurrent architectures are trained using the same command, replacing `mlp` with `gru` or `lstm`. During closed-loop evaluation, the corresponding checkpoint and normalization files must always be supplied to avoid mixing models and scalings.

OOD Series and Closed-Loop Comparison

The OOD series used in the results originates from `data/neural_ood/battery_v1`. The parameter `max_duration_s`=60 s was set in the generator so that fast trapezoidal trajectories have sufficient margin.

```
uv run python tools\generate_ood_battery.py ^
  --out data\neural_ood\battery_v1 ^
  --pid-source-dataset data\classic_dataset\v1 ^
  --overwrite

uv run python tools\run_classic_transfer_dataset.py ^
  --dataset data\neural_ood\battery_v1 ^
  --pid-source-dataset data\classic_dataset\v1 ^
  --pid-family all --include-native --workers 8 ^
  --no-visualization --rerun

uv run python tools\run_neural_outer_force_dataset.py ^
  --dataset data\neural_ood\battery_v1 --split ood ^
  --architecture mlp --device auto --workers 4 ^
  --no-visualization --rerun

uv run python tools\run_neural_outer_force_dataset.py ^
  --dataset data\neural_ood\battery_v1 --split ood ^
  --architecture gru --device auto --workers 4 ^
  --no-visualization --rerun

uv run python tools\run_neural_outer_force_dataset.py ^
  --dataset data\neural_ood\battery_v1 --split ood ^
  --architecture lstm --device auto --workers 4 ^
  --no-visualization --rerun

uv run python tools\summarize_comparison.py
```

Results Figures and Appendices

The comparative figures in the results chapter are regenerated from the CSV. The telemetry plots in the visual appendix are generated from the classic dataset, which contains the runs of the expert PDs with fixed parameters for training, validation, and testing. When illustrating cross-transfer, the transfer results folder is used instead.

```
uv run simulador-quad plot-comparison ^
  results\comparison_all_runs.csv ^
  --out TFG_Report\Figuras\resultados ^
  --formats pdf png --lang en

set ROOT=data\classic_dataset\v1\results\circle
set RUN=circle_g04_P5_combined_s1083
uv run simulador-quad plot ^
  %ROOT%\%RUN%\telemetry.json ^
  --metrics %ROOT%\%RUN%\metrics.json ^
  --out TFG_Report\Figuras\resultados\classic_circle_combined ^
  --profile report --formats pdf png --lang en

set ROOT=data\classic_dataset\v1\results_transfer
set RUN=circle_g04_P5_combined_s1083_with_pid_lissajous
uv run simulador-quad plot ^
  %ROOT%\%RUN%\telemetry.json ^
  --metrics %ROOT%\%RUN%\metrics.json ^
  --out TFG_Report\Figuras\resultados\transfer_circle_lissajous ^
  --profile report --formats pdf png --lang en
```

The same `plot` command was applied to five native cases: `hold_g05_P5`, `circle_g04_P5`, `circle_g07_P2`, `lissajous_g07_P5`, and `waypoint_g03_P5`. It was also used for a cross-transfer pair of the circular case. In all of them, the `report` profile removes redundant titles while preserving axes, units, and legends.

Evidence Files

- `results/comparison_all_runs.csv`: Comparable runs used by the main results figures.
- `results/comparison_summary.csv`: Aggregation by controller, split, and family.
- `results/evidence_manifest.csv`: Origin of datasets, checkpoints, and inclusion decisions.

- Time-series files of the native expert PDs in `data/classic_dataset/v1/results/*/*/telemetry.json`.
- Metrics and metadata files for those runs in `data/classic_dataset/v1/results/*/*/metrics.json`.
- Cross-transfer runs between PD families in `data/classic_dataset/v1/results_transfer/*/`.
- Thesis figures directory in `TFG_Report/Figuras/resultados/`.

Appendix 2. Code Architecture and Relevant Snippets

This appendix complements the methodological description with an engineering perspective on the code. It aims to highlight the elements supporting the reproducibility of the work, the simulation contract, the separation between dynamics, control, telemetry, and visualization, and the integration of the neural controller within the classic loop.

Functional Organization of the Package

The `simulador_quad` package is organized modularly according to responsibilities under the `src/simulador_quad/` directory:

- `dynamics`: Rigidbody dynamics and actuators.
- `core`: Reference frames, data contracts, and common utilities.
- `control`: Implementation of the different controllers.
- `trajectories`: Reference trajectory generation and management.
- `telemetry`: Structured logging of simulation telemetry.
- `metrics`: Performance and error metrics calculation.
- `visualization`: Rendering and graphical outputs.

The scripts in `tools/` manage dataset generation, expert PD series, neural training, batch execution, and results consolidation.

This separation is relevant because the work compares controllers without modifying the physical environment. An execution can replace the outer loop with a network, fix expert PDs, or run an OOD series, while the integrator, mixer, actuators, disturbances, and metrics remain common.

Multirate Simulation Loop

The following snippet summarizes the core of the temporal contract. In each cycle, safety limits are verified, the reference is computed, the control is updated using zero-order hold, the actuator system is applied, telemetry is logged at a lower frequency, and physics are integrated using RK4. This structure is what allows setting 100 Hz for physics, 50 Hz for control, and 10 Hz for telemetry without mixing responsibilities.

```
while True:
    is_saturated = any(current_applied.saturation_flags)
    term, reason = self._check_safety_termination(state, is_saturated)
    if term:
        break

```

```
    current_ref = trajectory.get_reference_for_state(time_s, state)

    if time_s - last_control_time >= self.control_dt_s:
        obs_pos, obs_vel = self.noise.apply_noise(
            state.position_W_m,
            state.velocity_W_m_s,
        )
        current_control = controller.compute_control(time_s, current_obs,
     current_ref)
        current_rotor_cmd = self.mixer.compute_rotor_commands(
            current_control.collective_thrust_N,
            current_control.body_moments_Nm,
        )

    current_applied, torque_B, thrust_B = \
        self.actuators.compute_applied_forces(current_rotor_cmd.
    target_omega_rad_s)

    if time_s - last_telemetry_time >= self.telemetry_dt_s:
        telemetry.append(sample_from_state_control_and_reference(...))

    state = rk4_step(..., thrust_B, torque_B, wind_W_m_s,
    drag_coefficients)
```

Listing 2: Simplified schematic of the simulation loop.

Neural Controller Integration

The neural controller does not replace the entire control loop. The network estimates the desired force of the outer loop in the world frame, while the classic controller retains the force-to-attitude conversion, the inner loop, saturations, and the mixer. Recurrent inference is performed using a window of fixed length; thus, the network is not run in a persistent *stateful* mode, but rather reconstructs the sequential input at each step from a FIFO queue of normalized samples.

```
x = build_outer_force_min_features_from_observation(obs_state, reference)
x_norm = self.normalizer.normalize_x(torch.tensor(x, dtype=torch.float32)
    )

if self.architecture == "mlp":
    model_input = x_norm.unsqueeze(0)          # [1, input_dim]
else:
```

```
    self.window.append(x_norm)
    while len(self.window) < self.sequence_length:
        self.window.appendleft(x_norm)
    model_input = torch.stack(list(self.window)).unsqueeze(0)  # [1, 20,
    input_dim]

with torch.no_grad():
    y_norm = self.model(model_input).squeeze(0)
```

Listing 3: Input preparation for MLP and recurrent networks.

This decision facilitates a common interface across MLP, GRU, and LSTM and prevents the hidden state from depending on previous runs. As a trade-off, it introduces recalculation of historical samples at each recurrent inference step.

Output Traceability

Each execution writes `telemetry.json` and `metrics.json`. Telemetry preserves state, observation, reference, control command, rotor commands, applied actuation, wind, and, where applicable, neural forces before and after clipping. Metrics aggregate RMSE, maximum error, duration, termination cause, saturation, mixer degradation, and provenance metadata.

Appendix 3. Visual Simulation Evidences

The figures in this appendix aim to display the visual evidence following a simulation and provide context for the aggregated metrics shown in the results chapter. The telemetry files used in this appendix originate from the classic dataset at `data/classic_dataset/v1/results`. Therefore, they correspond to the specialized PDs with fixed parameters that feed the training, validation, and testing phases. Finally, an example from the transfer results folder is added to illustrate cross-transfer using those same PDs.

Trajectory Atlas

Before discussing time signals, Figure 5.1, Figure 5.2, and Figure 5.3 visually outline the variety of reference trajectories. The first two figures summarize ID families and OOD trajectories, while the third singles out a helix to show its three-dimensional nature.

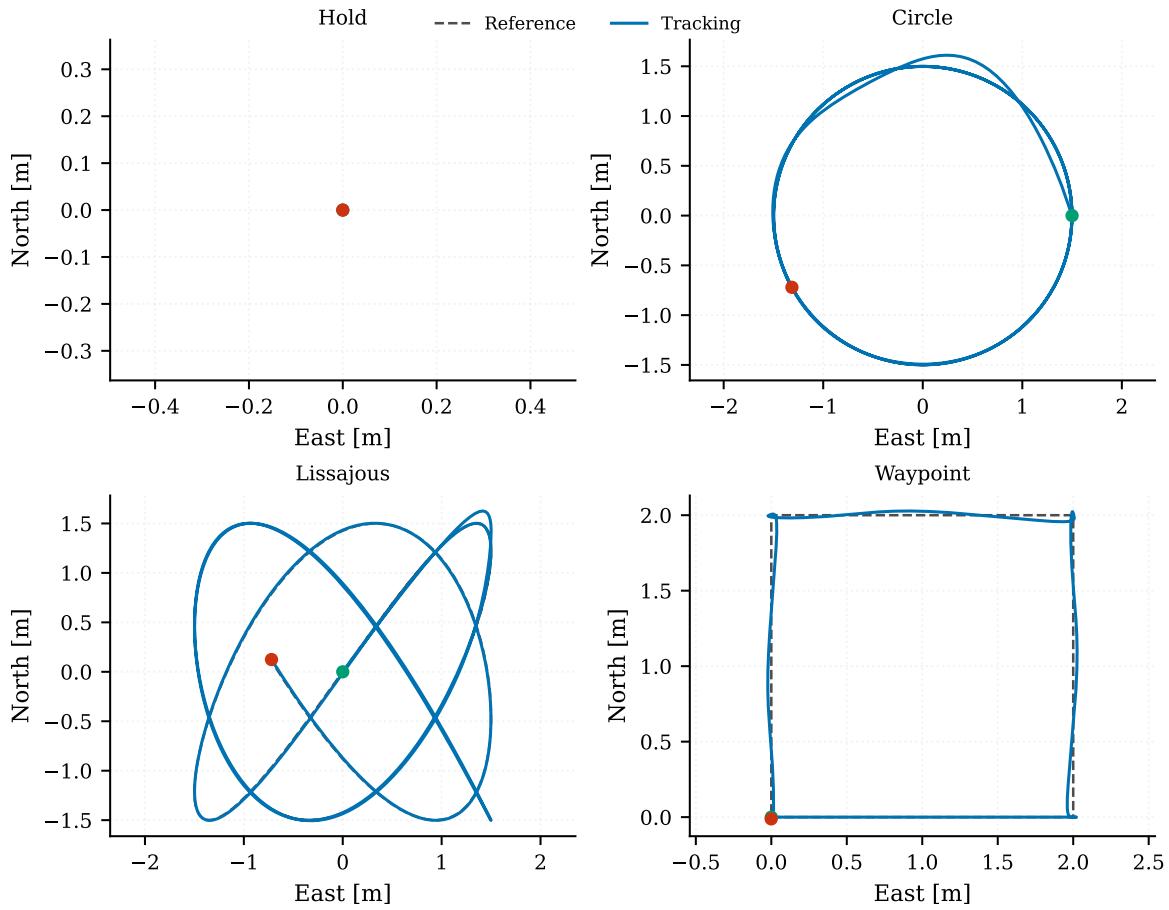


Figure 5.1. Visual sample of known families: `hold`, `circle`, `lissajous`, and `waypoint`. Source: own elaboration.

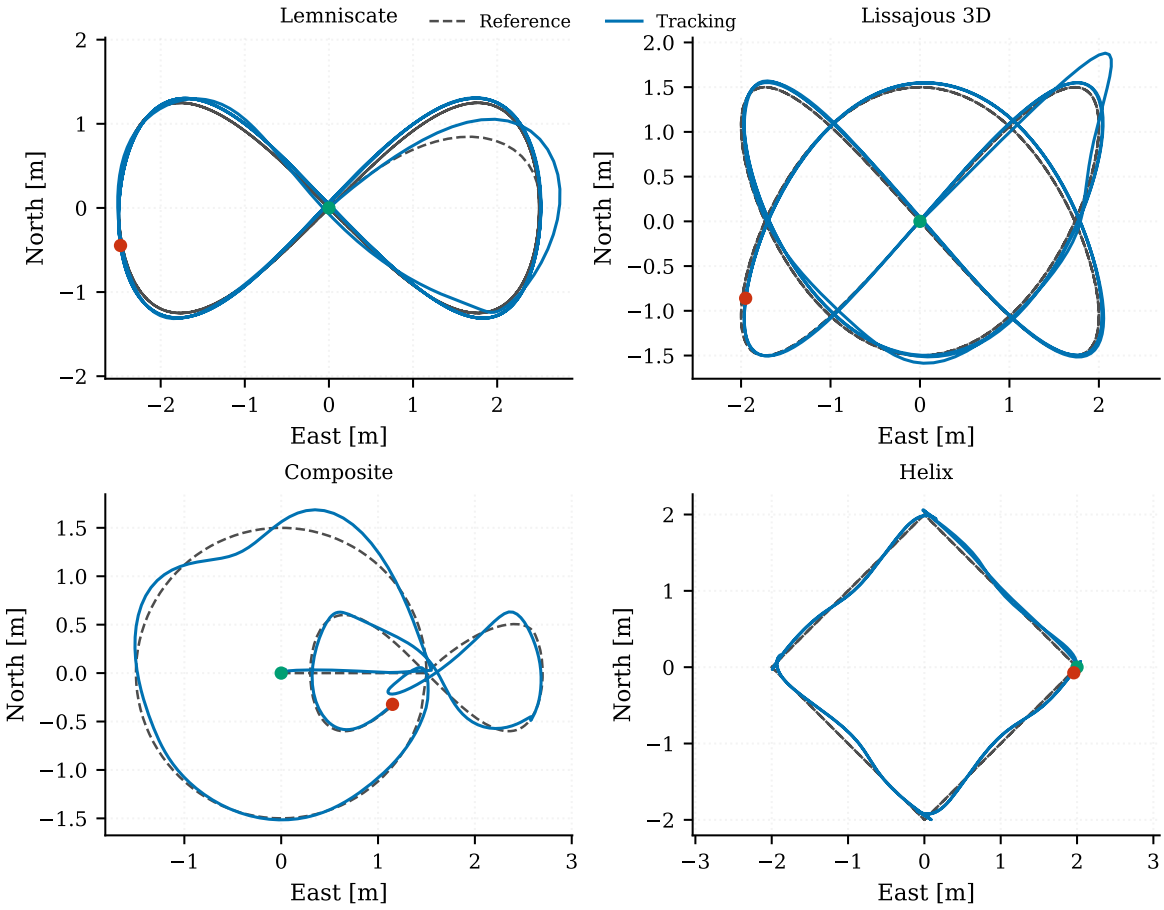


Figure 5.2. Visual sample of out-of-distribution trajectories: `lemniscate`, `lissajous` 3D, composition, and helix. Source: own elaboration.

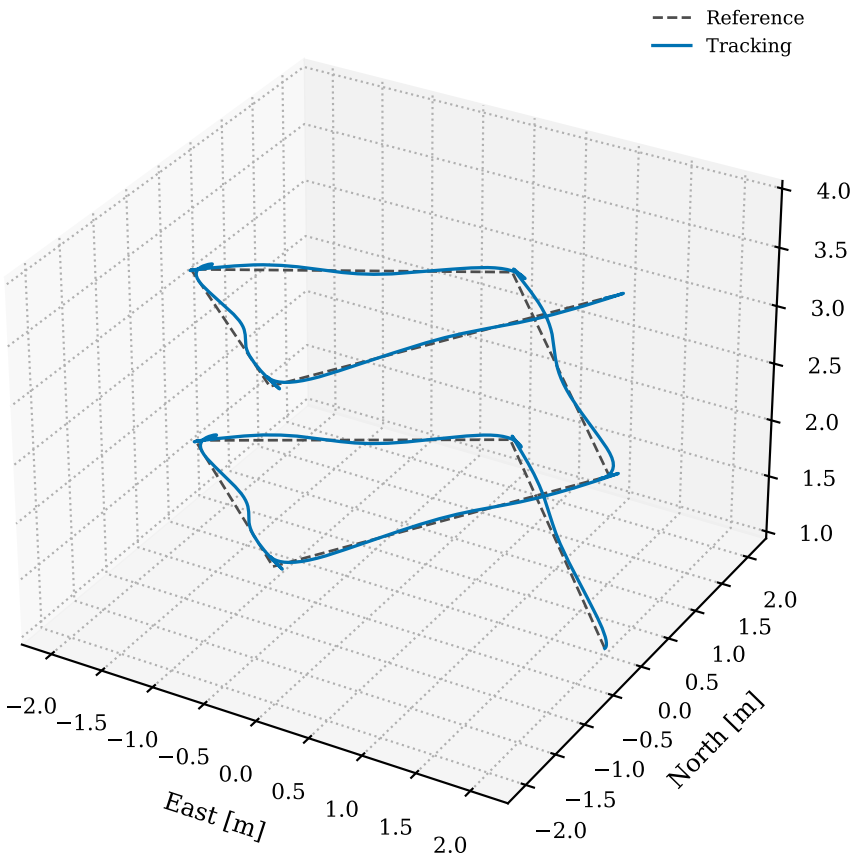


Figure 5.3. Three-dimensional view of a representative OOD helix. Source: own elaboration.

Selected Episodes

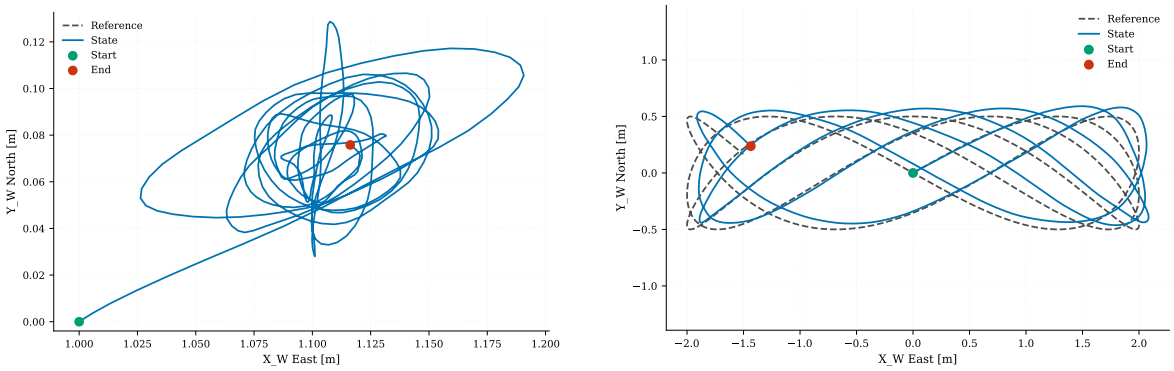
Table 5.1 summarizes the episodes included in the subsequent figures. Cases from the classic dataset's `test` split have been selected to cover different families, disturbances, and termination modes. The selection aims to show what is analyzed in a run beyond the position RMSE.

Table 5.1. Cases from the classic dataset used for the telemetry plots.

Case	Split	Reason for Inclusion
`hold_g05_P5`	`test`, PD `hold`	Hover with combined disturbance; allows observing equilibrium, correction against wind and noise, and collective thrust close to weight.
`circle_g04_P5`	`test`, PD `circle`	Curved tracking with combined disturbance and fixed-duration trajectory.
`circle_g07_P2`	`test`, PD `circle`	Circle case with East wind, useful for observing transient error and sustained attitude.
`lissajous_g07_P5`	`test`, PD `lissajous`	Multi-axis periodic trajectory with combined disturbance.
`waypoint_g03_P5`	`test`, PD `waypoint`	Finite trajectory with segments, direction changes, and completed-trajectory termination.

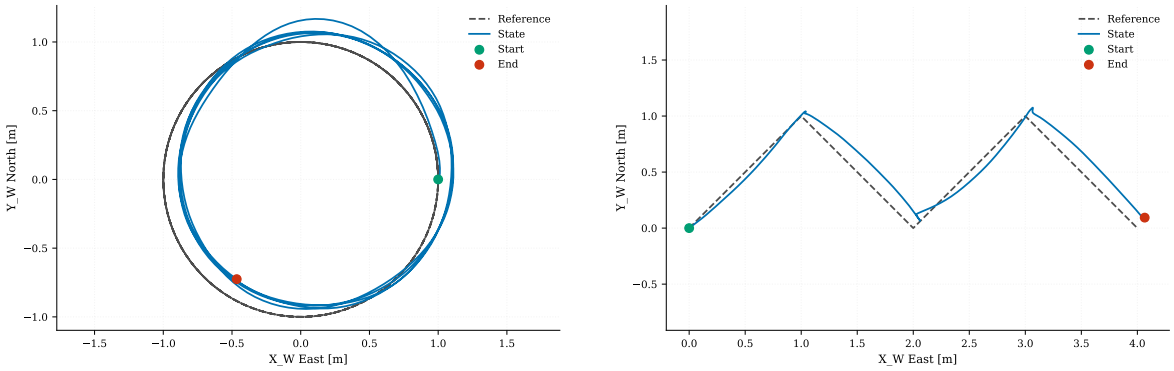
Geometric Interpretation

Figure 5.4 and Figure 5.5 compare reference and state in the horizontal plane. The first shows the contrast between a hover reference and a periodic curve, while the second compares sustained circular tracking and a waypoint trajectory, which features straight segments and direction changes.



(a). `hold_g05_P5`. (b). `lissajous_g07_P5`.

Figure 5.4. Geometric tracking of two specialized PDs from the classic dataset: perturbed hover case and multi-axis Lissajous trajectory. Source: own elaboration.

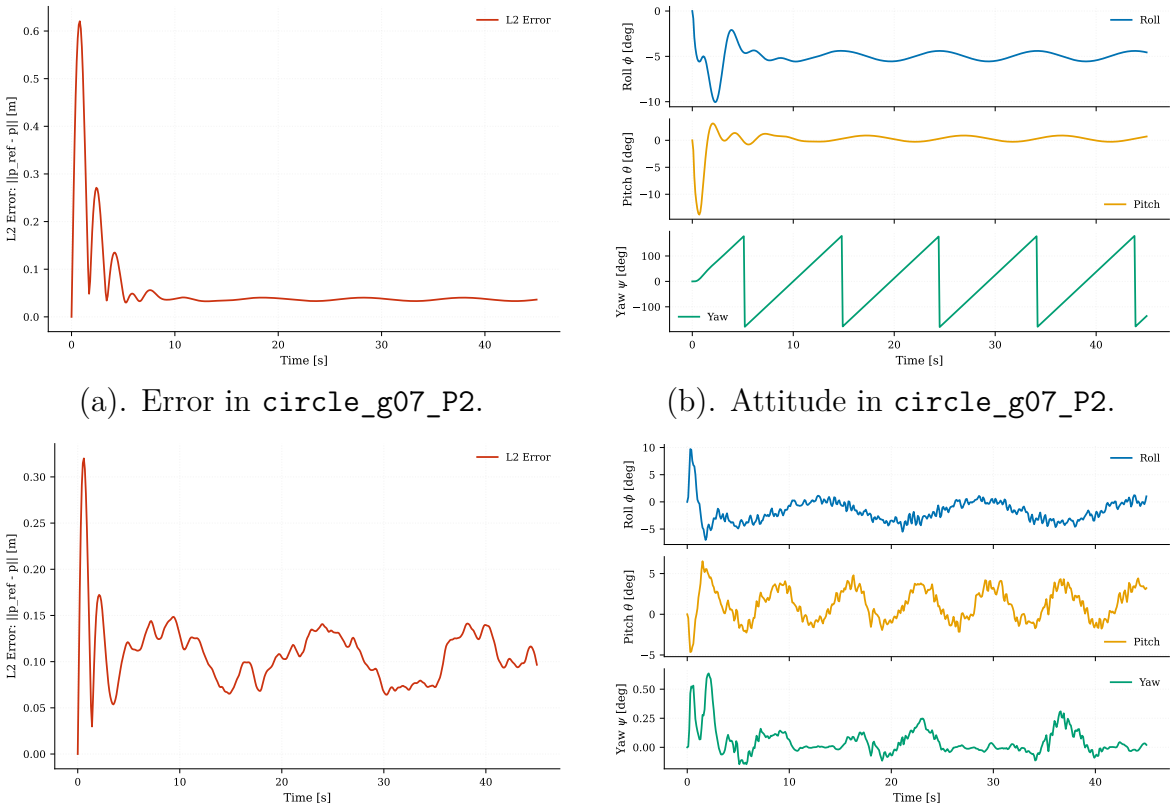


(a). `circle_g04_P5`. (b). `waypoint_g03_P5`.

Figure 5.5. Geometric tracking in a perturbed circular reference and a waypoint trajectory from the test split. Source: own elaboration.

Error and Attitude

To interpret a simulation, viewing the trajectory is not enough. Figure 5.6 combines tracking error and attitude in two episodes from the classic dataset: a circle with East wind and a Lissajous with combined disturbance. This relates transient errors to the inclination that the inner loop must stabilize.



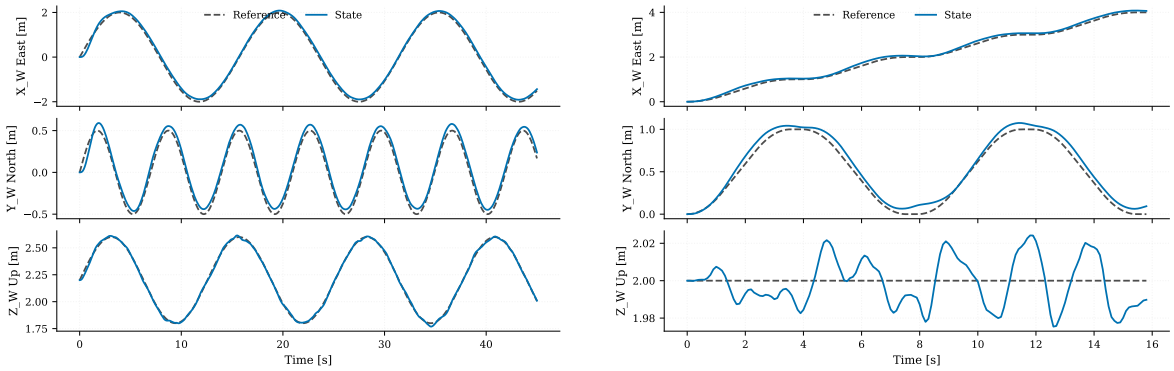
(a). Error in `circle_g07_P2`. (b). Attitude in `circle_g07_P2`.

(c). Error in `lissajous_g07_P5`. (d). Attitude in `lissajous_g07_P5`.

Figure 5.6. Visual relationship between tracking error and attitude in two classic dataset cases with disturbances. Source: own elaboration.

Time Series by Axis

Reading position axis-by-axis, as shown in Figure 5.7, is useful when the 2D plot does not reveal the full difficulty of the run. In the Lissajous trajectory, simultaneous altitude variation is visible, while in the waypoint trajectory, segments of forward motion and plateaus corresponding to arrival at each point can be distinguished.



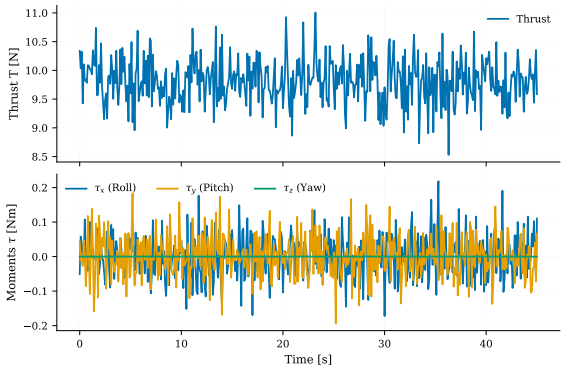
(a). Position by axis in `lissajous_g07_P5`.

(b). Position by axis in `waypoint_g03_P5`.

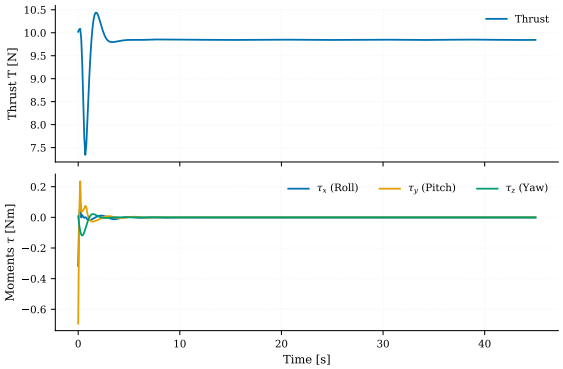
Figure 5.7. Temporal evolution of position against reference in two families with different geometric behaviors. Source: own elaboration.

Actuation

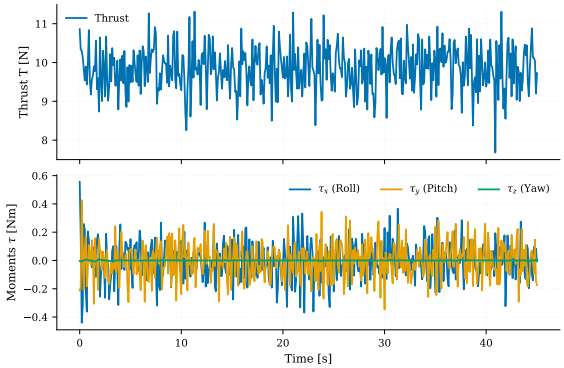
Figure 5.8 displays actuation telemetry in three cases from the classic dataset. The `hold` case shows the thrust level hovering around the vehicle weight, the circle with wind requires sustained corrections, and the combined Lissajous trajectory illustrates the distribution of thrust, moments, and rotor speeds in a multi-axis reference.



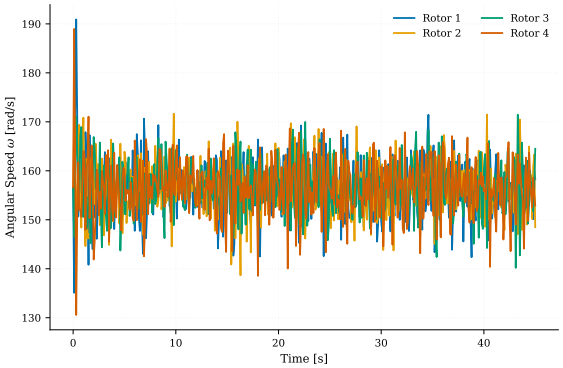
(a). Control effort in `hold_g05_P5`.



(b). Control effort in `circle_g07_P2`.



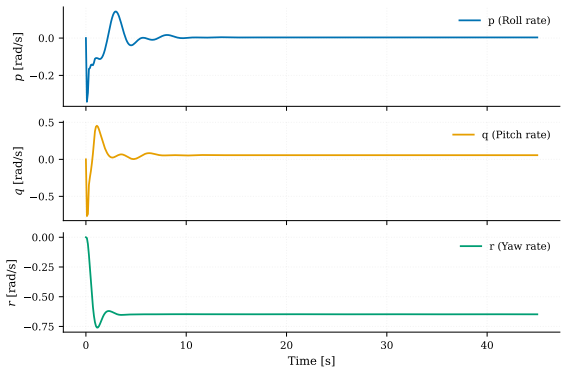
(c). Control effort in `lissajous_g07_P5`.



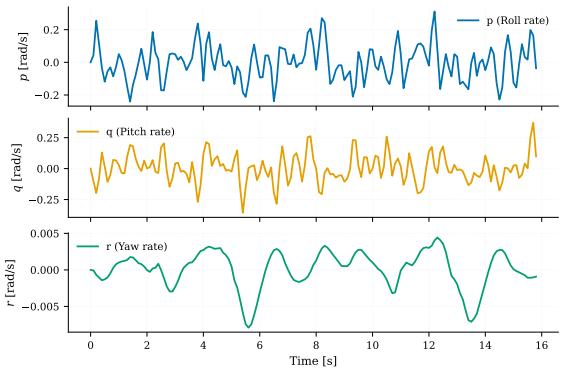
(d). Rotor speed in `lissajous_g07_P5`.

Figure 5.8. Actuation signals inspected after closed-loop simulation with native expert PD. Source: own elaboration.

Finally, Figure 5.9 completes the actuation interpretation with angular velocities in the FRD body frame. These data reveal whether an execution with acceptable position tracking relies on attitude oscillations or rapid demands from the inner loop.



(a). Angular velocities in `circle_g07_P2`.

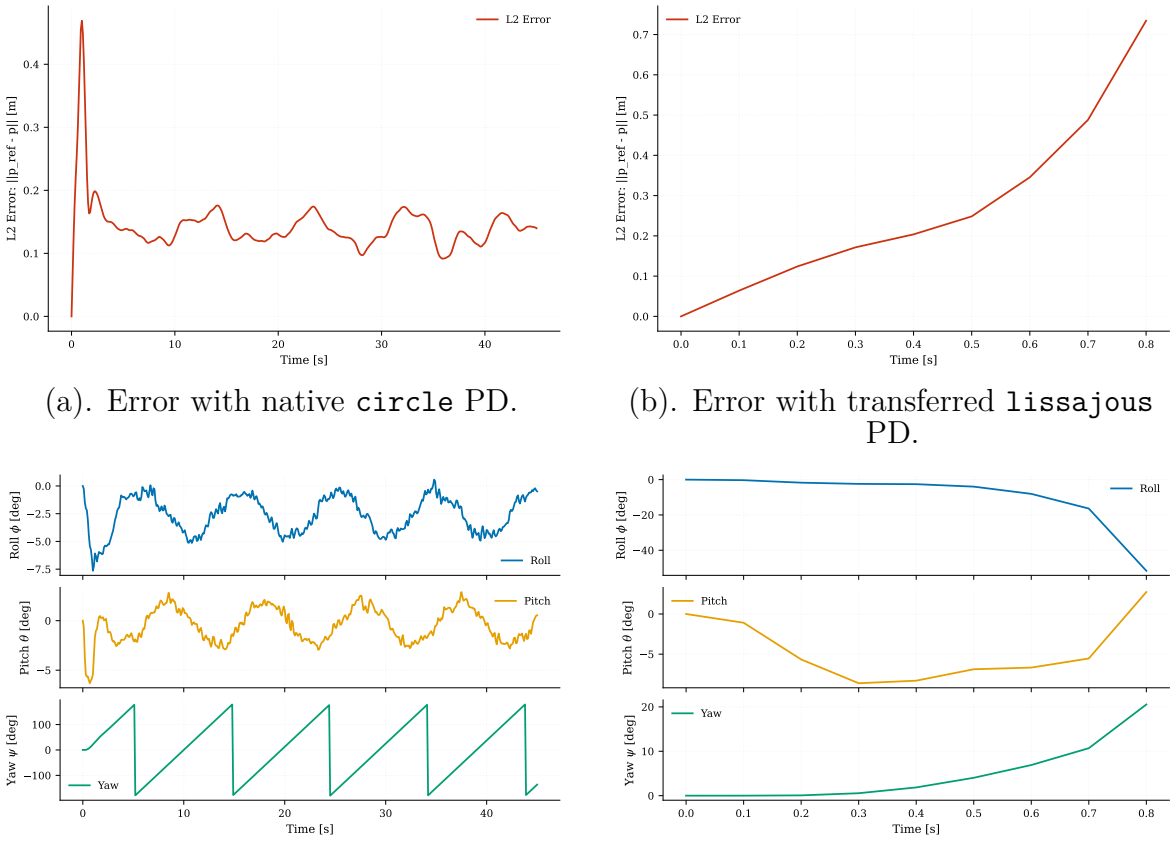


(b). Angular velocities in `waypoint_g03_P5`.

Figure 5.9. Vehicle body angular velocities for two classic dataset tracking scenarios. Source: own elaboration.

Cross-Transfer Example

The `results_transfer` dataset allows running a single scenario with PDs from other families without retuning their gains. Figure 5.10 shows the `circle_g04_P5` scenario with its native circle PD and with the Lissajous PD. The objective is to illustrate what happens, and how the Lissajous PD fails at the beginning.



(a). Error with native `circle` PD.

(b). Error with transferred `lissajous` PD.

(c). Attitude with native PD.

(d). Attitude with transferred PD.

Figure 5.10. Visual inspection of a cross-transfer in scenario `circle_g04_P5`: comparison between the native PD and a PD from another family. Source: own elaboration.